

A decorative graphic on the left side of the slide consists of a network of lines and circles. The lines are primarily dark blue, with some transitioning to a lighter blue and then to a light beige color towards the bottom. The circles are small and white with dark outlines. The overall pattern resembles a stylized circuit board or a network diagram.

058165 - PARALLEL COMPUTING

Fabrizio Ferrandi

a.a. 2022-2023

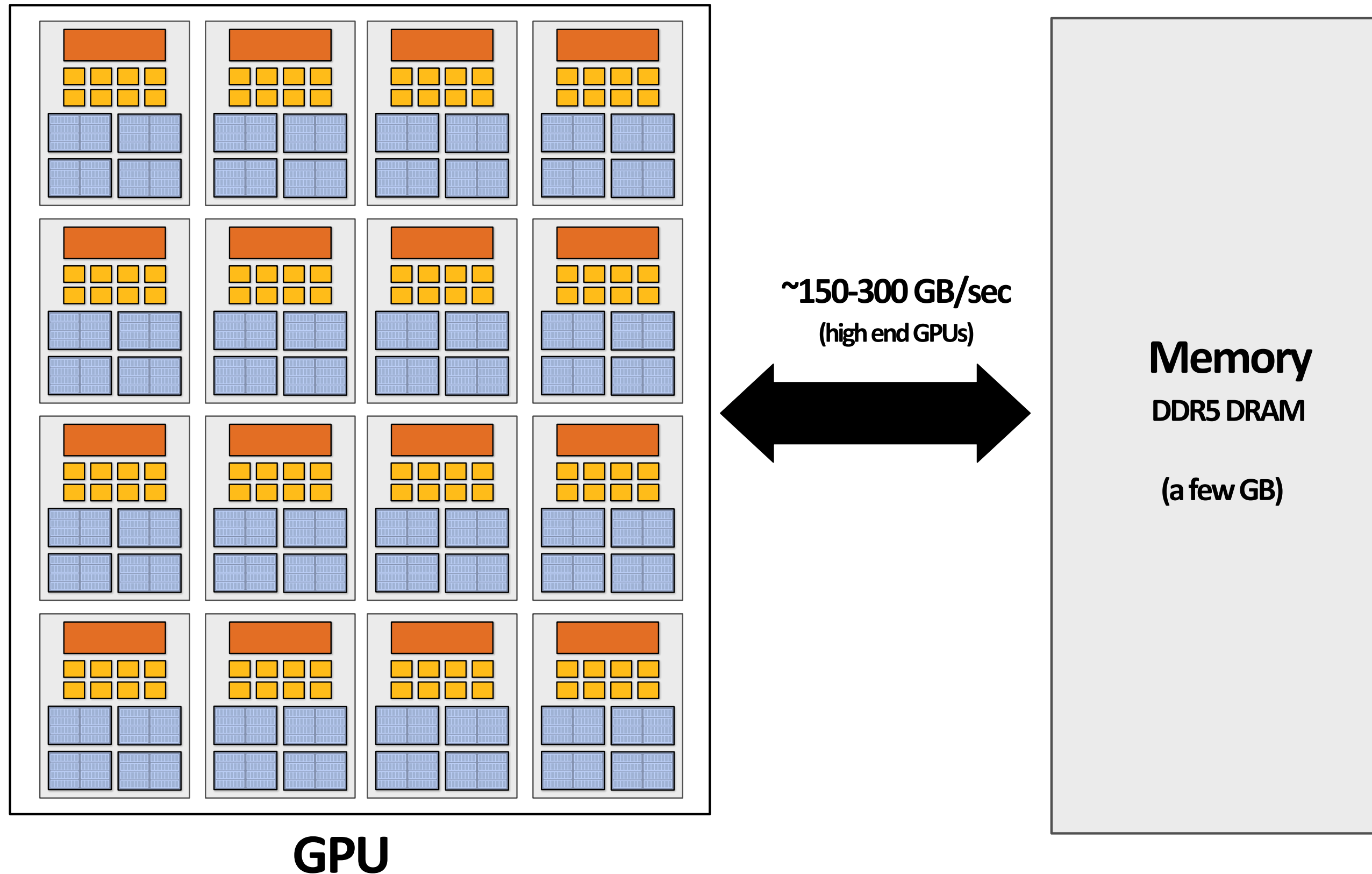
Acknowledge

Material from:

Parallel Computing lectures from prof. Kayvon and prof. Olukotun Stanford University

GPU Architecture

Basic GPU architecture



Multi-core chip

SIMD execution within a single core (many execution units performing the same instruction) Multi-threaded execution on a single core (multiple threads executed concurrently by a core)

Graphics 101 + GPU history

(for fun)

What GPUs were originally designed to do:

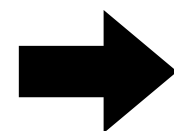
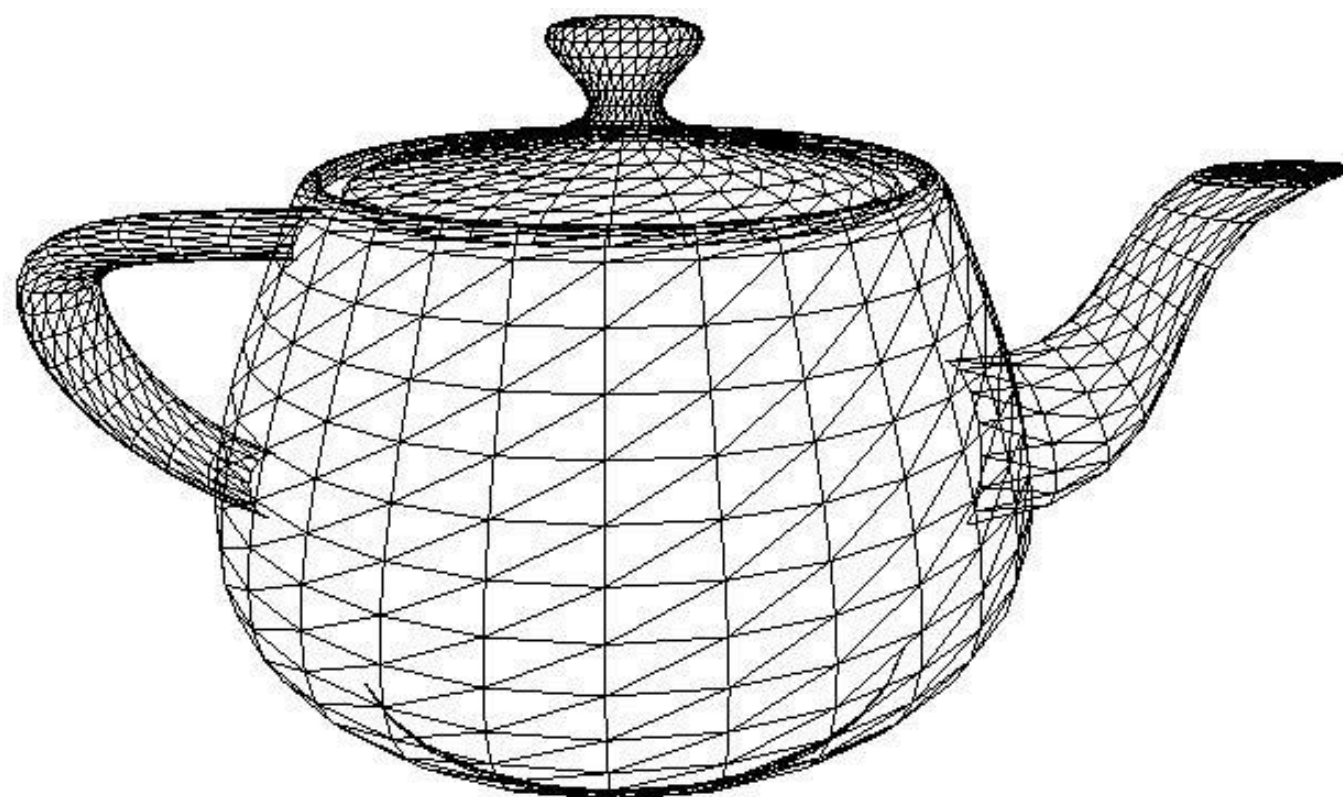


Image credit: Henrik Wann Jensen

Input: description of a scene:

3D surface geometry (e.g., triangle mesh) surface
materials, lights, camera, etc.

Output: image of the scene

**Simple definition of rendering task: computing how each triangle in 3D
mesh contributes to appearance of each pixel in the image?**

What GPUs were originally designed to do





Render high complexity 3D scenes, in real-time

Far Cry 5

Render high complexity 3D scenes, in real-time

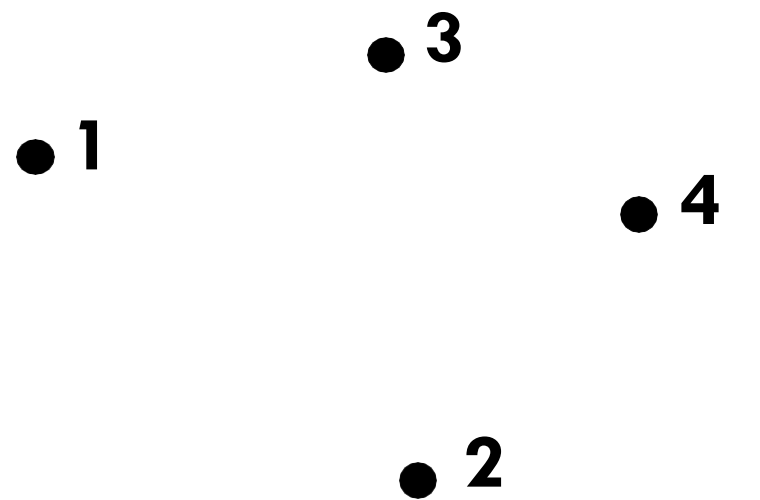
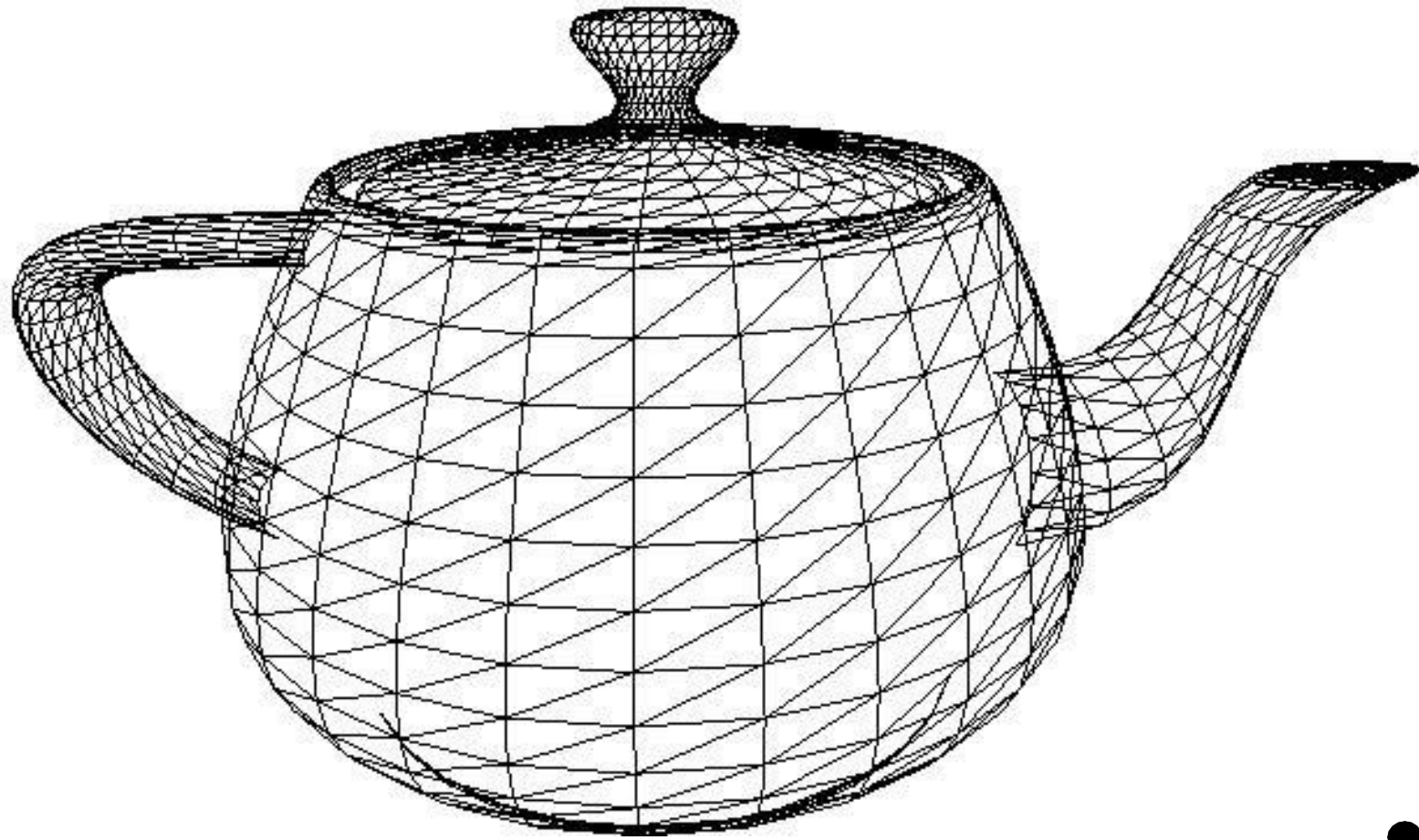


Screenshot: Red Dead Redemption

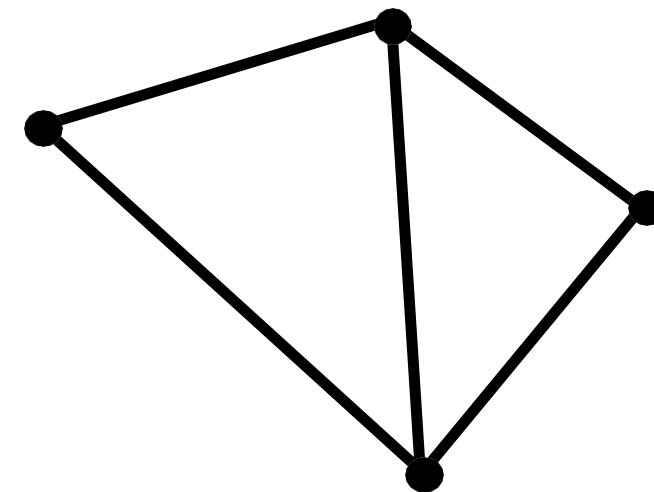
The 3D graphics workload

Real-time graphics primitives (entities)

Represent surfaces as 3D triangle meshes



Vertices (points in space)

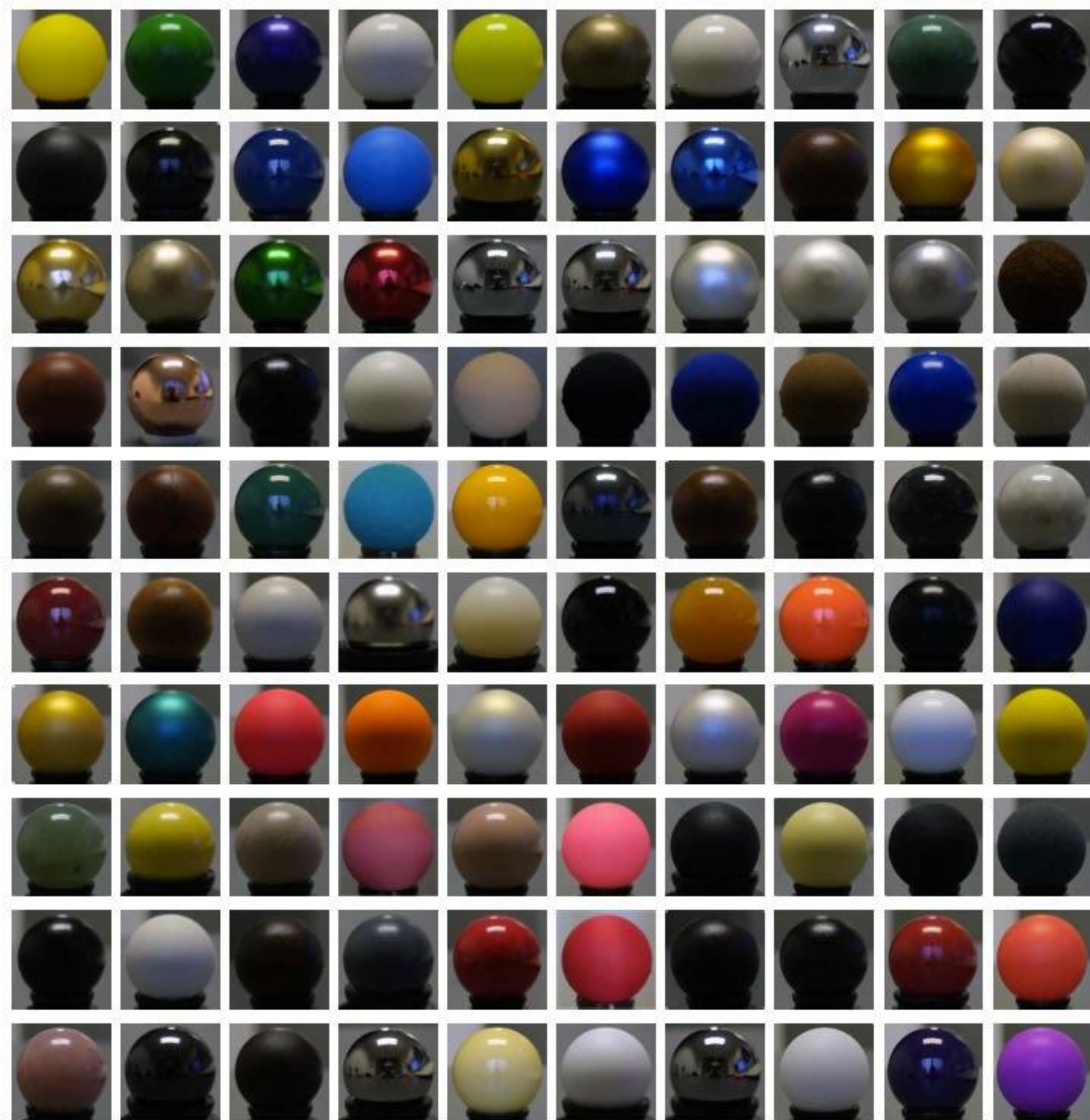


Primitives
(e.g., triangles, points, lines)

Workload in one slide

- **Given a triangle, determines where it lies on screen given the position of a virtual camera**
- **For all pixels covered by the triangle, compute the color of the surface at that pixel.**

What does the surface look like at a point?



Great diversity of materials and lights in the world!



Example “shader program” *

Run once per fragment (per pixel covered by a triangle)

OpenGL shading language (GLSL) shader program: defines behavior of fragment processing stage

```
uniform sampler2D myTexture;
uniform float3 lightDir;
varying vec3 norm;
varying vec2 uv;

vec4 myFragmentShader()
{
    vec3 kd = texture2D(myTexture, uv);
    kd *= clamp(dot(lightDir, norm), 0.0, 1.0);
    return vec4(kd, 1.0);
}
```

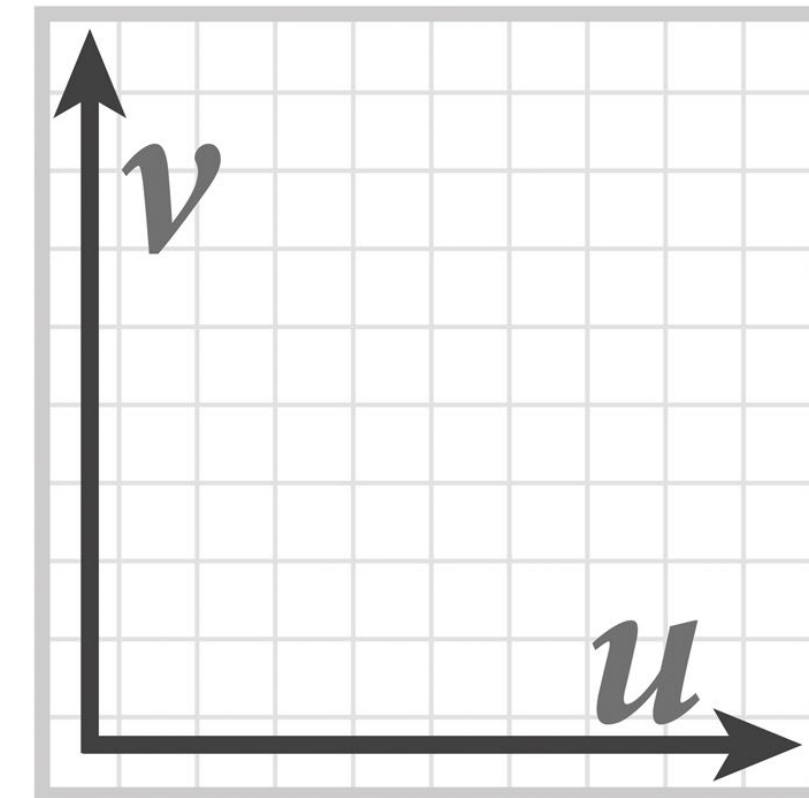
read-only global variables

per-fragment inputs

“fragment shader”
(a.k.a function invoked for each input
pixel fragment)

per-fragment output: RGBA surface color at pixel

myTexture is a texture map



* Syntax/details of this code not important

What is important is that a shader is a pure function invoked on a stream of inputs.

Shaded result

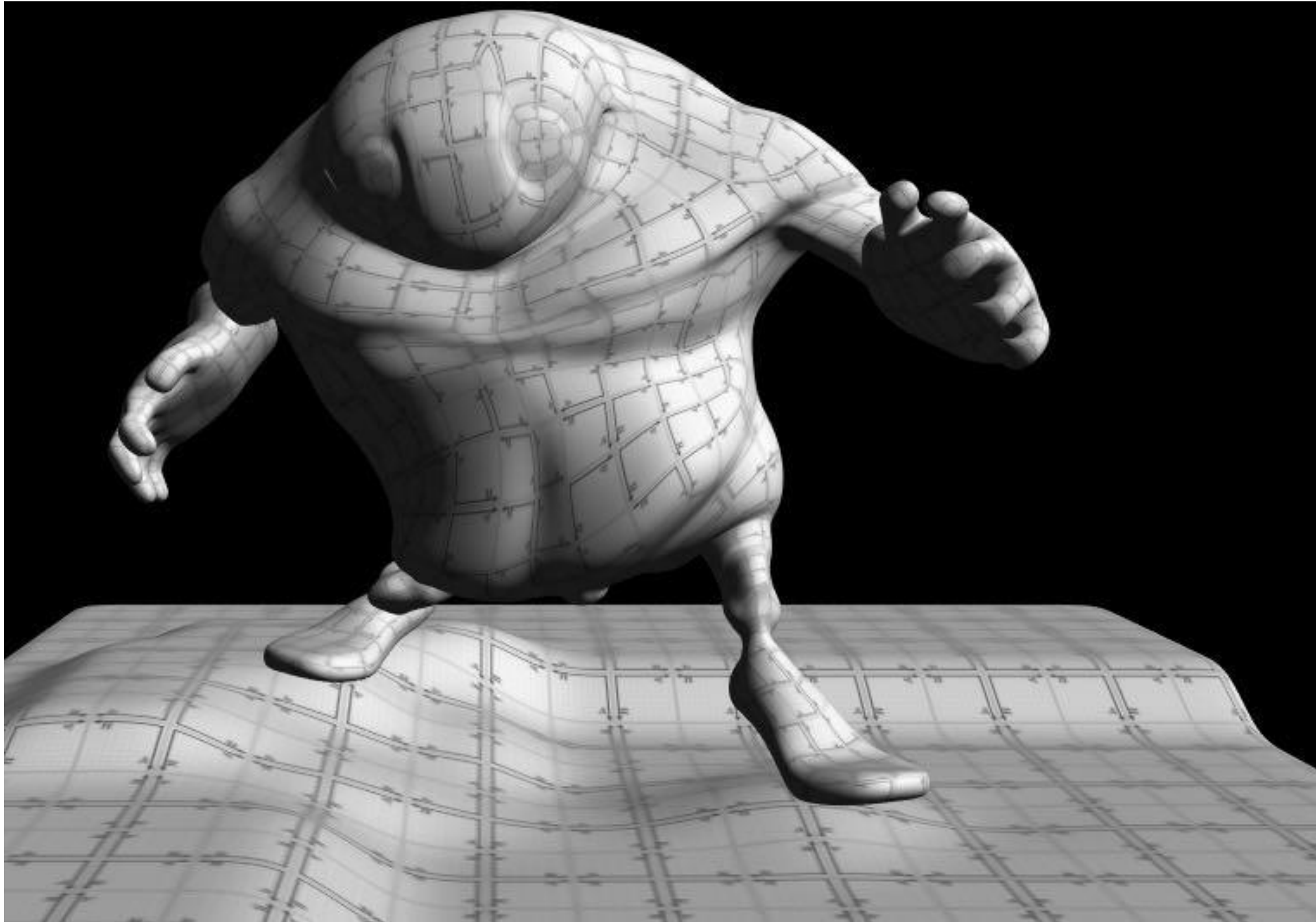
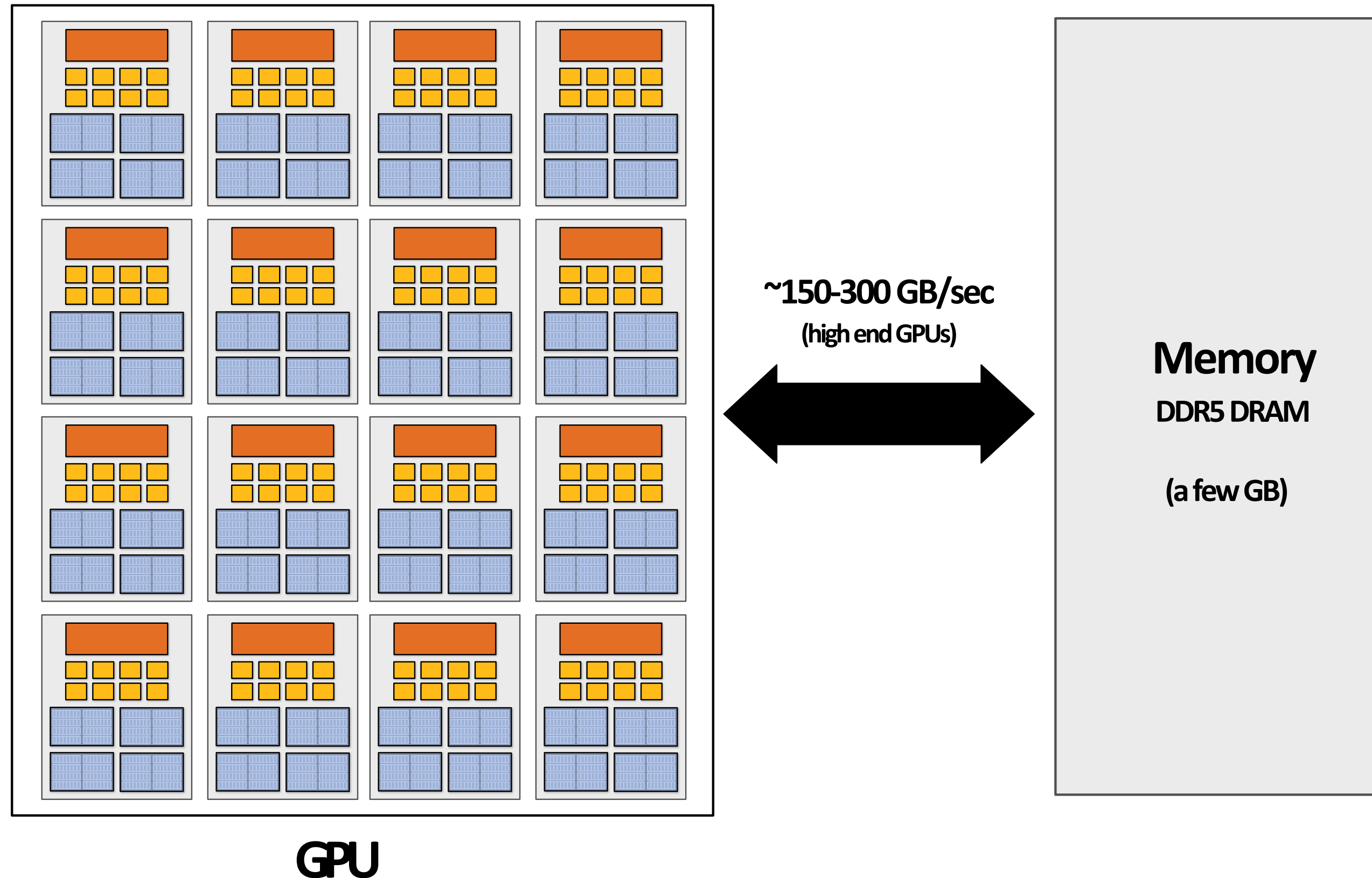


Image contains output of myFragmentShader for each pixel covered by surface (pixels covered by multiple surfaces contain output from surface closest to camera)

Why do GPU's have many high-throughput cores?

Many SIMD, multi-threaded cores provide efficient execution of shader programs

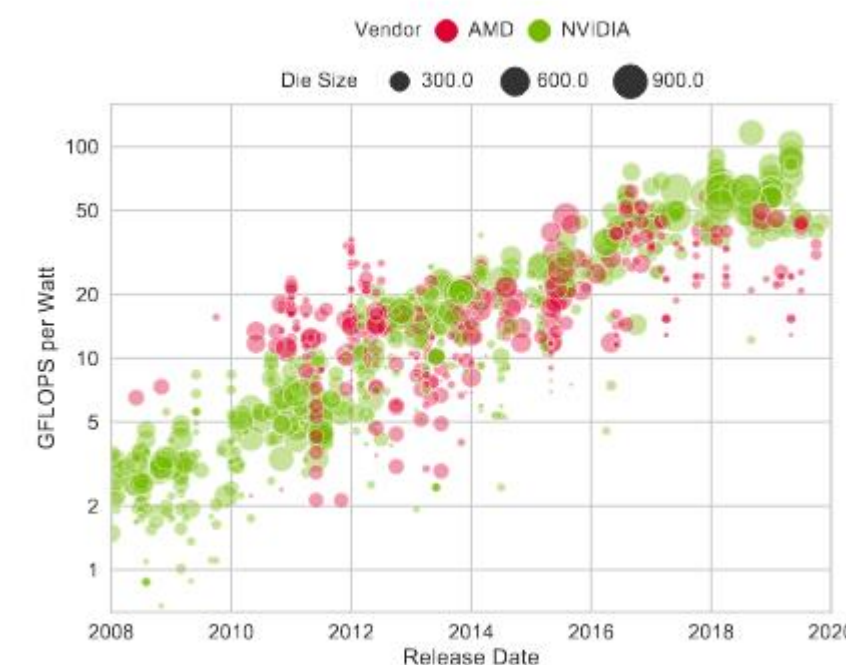
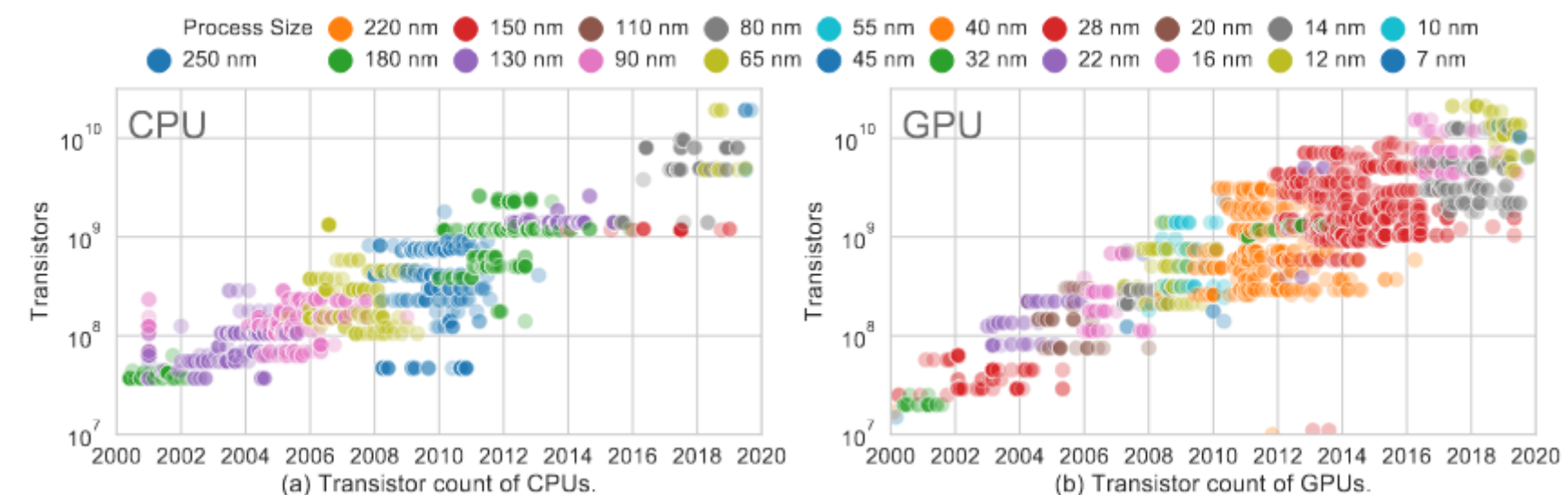


Observation circa 2001-2003

GPUs are very fast processors for performing the same computation (shader programs) in parallel on large collections of data (streams of vertices, fragments, and pixels)

Wait a minute! That sounds a lot like data-parallelism to me! I remember data-parallelism from exotic supercomputers in the 90s.

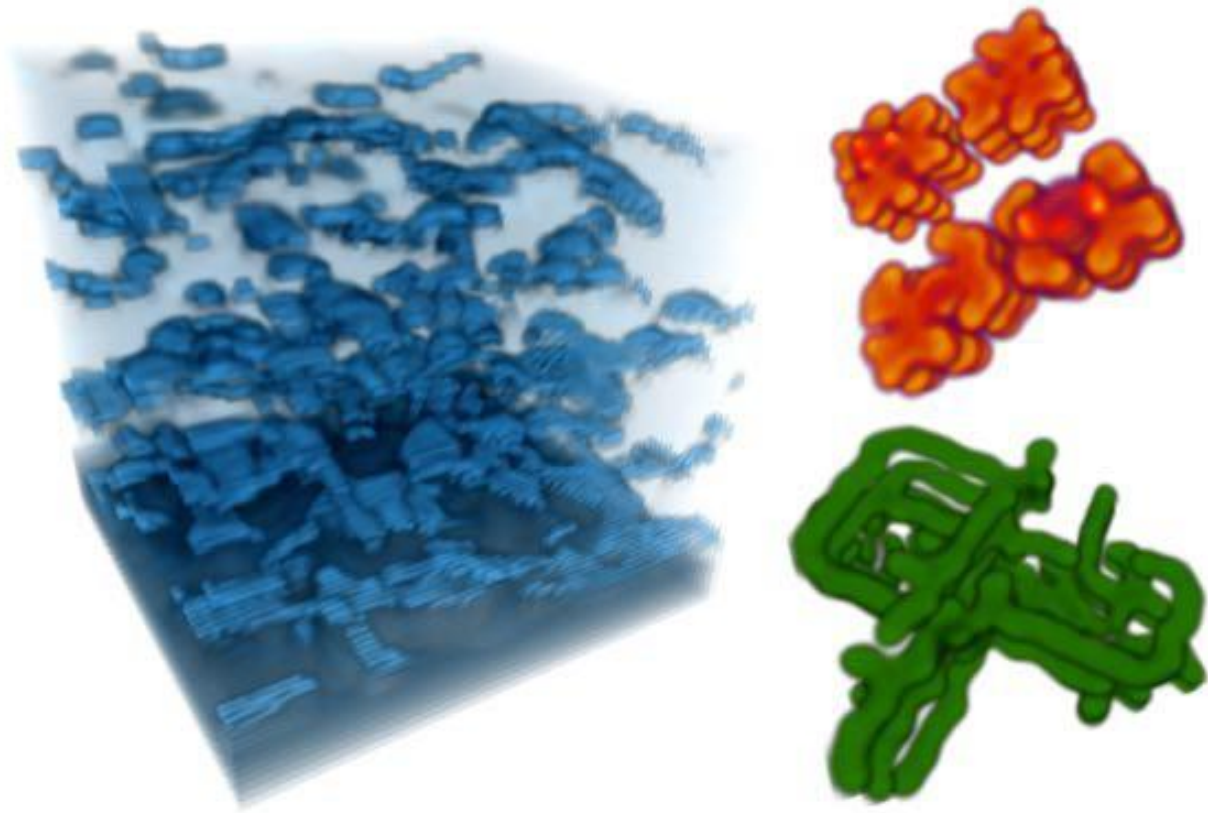
And every year GPUs are getting faster because more transistors = more parallelism.



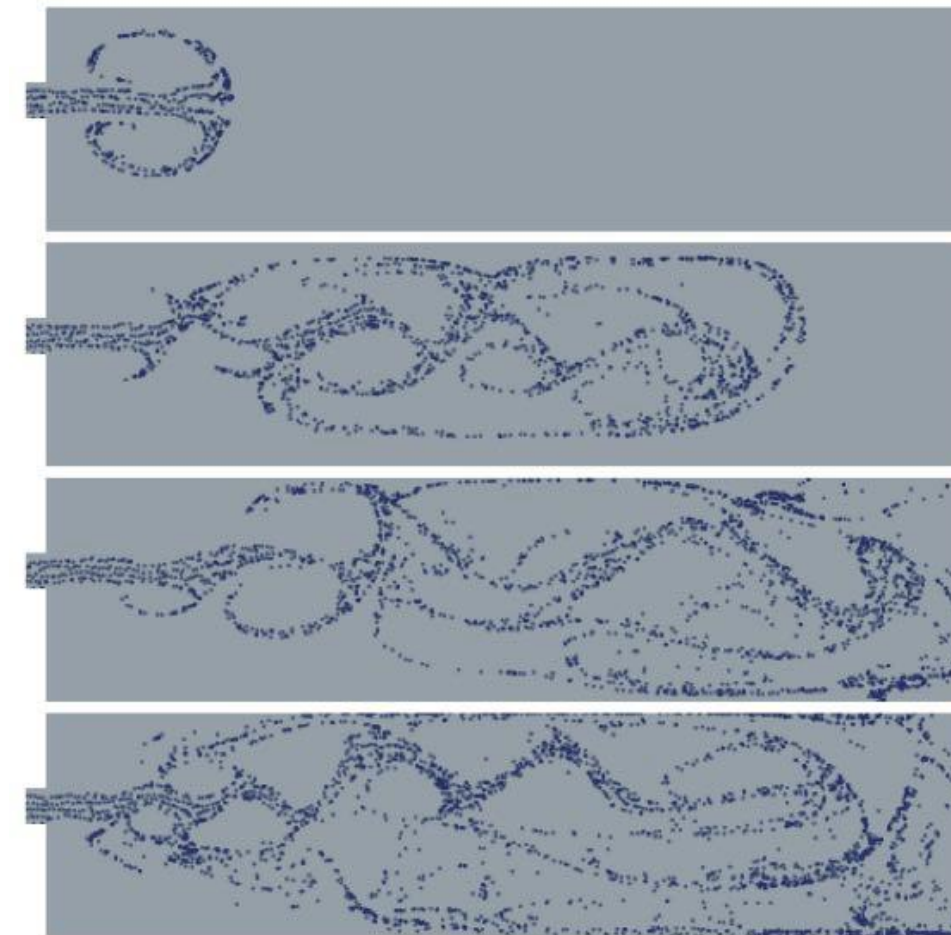
Y Sun, NB Agostini, S Dong, D Kaeli , “Summarizing CPU and GPU design trends with product data”, 2019

“GPGPU” 2002-2003

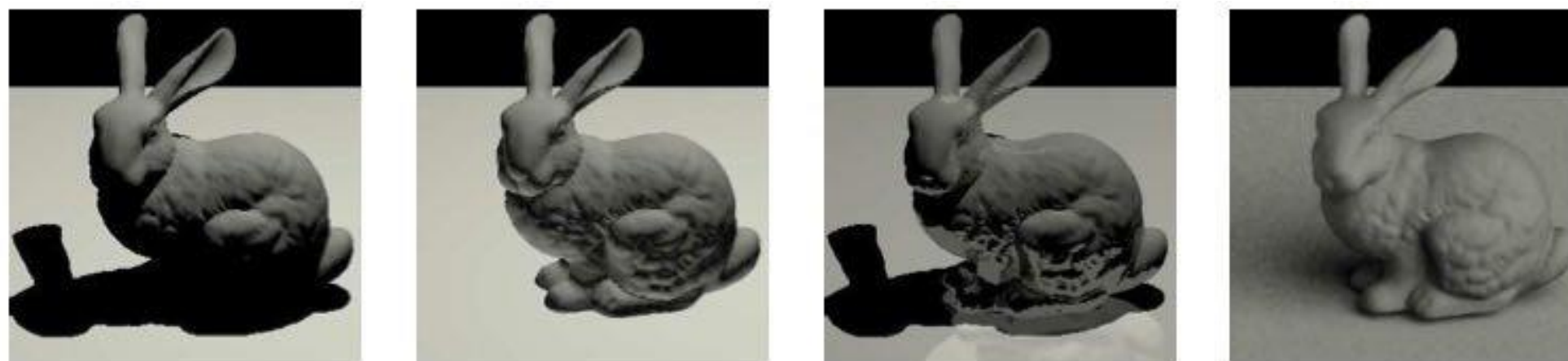
GPGPU = “general purpose” computation on GPUs



Coupled Map Lattice Simulation [Harris 02]



Sparse Matrix Solvers [Bolz 03]



Ray Tracing on Programmable Graphics Hardware [Purcell 02]

Brook stream programming language (2004)

- **Stanford graphics lab research project** [Buck 2004]
- **Abstract GPU hardware as data-parallel processor**

```
kernel void scale(float amount, float a<>, out
float b<>)
{
    b = amount * a;
}

float scale_amount;
float input_stream<1000>;    // stream declaration
float output_stream<1000>;   // stream declaration

// omitting stream element initialization...

// map kernel onto streams
scale(scale_amount, input_stream, output_stream);
```

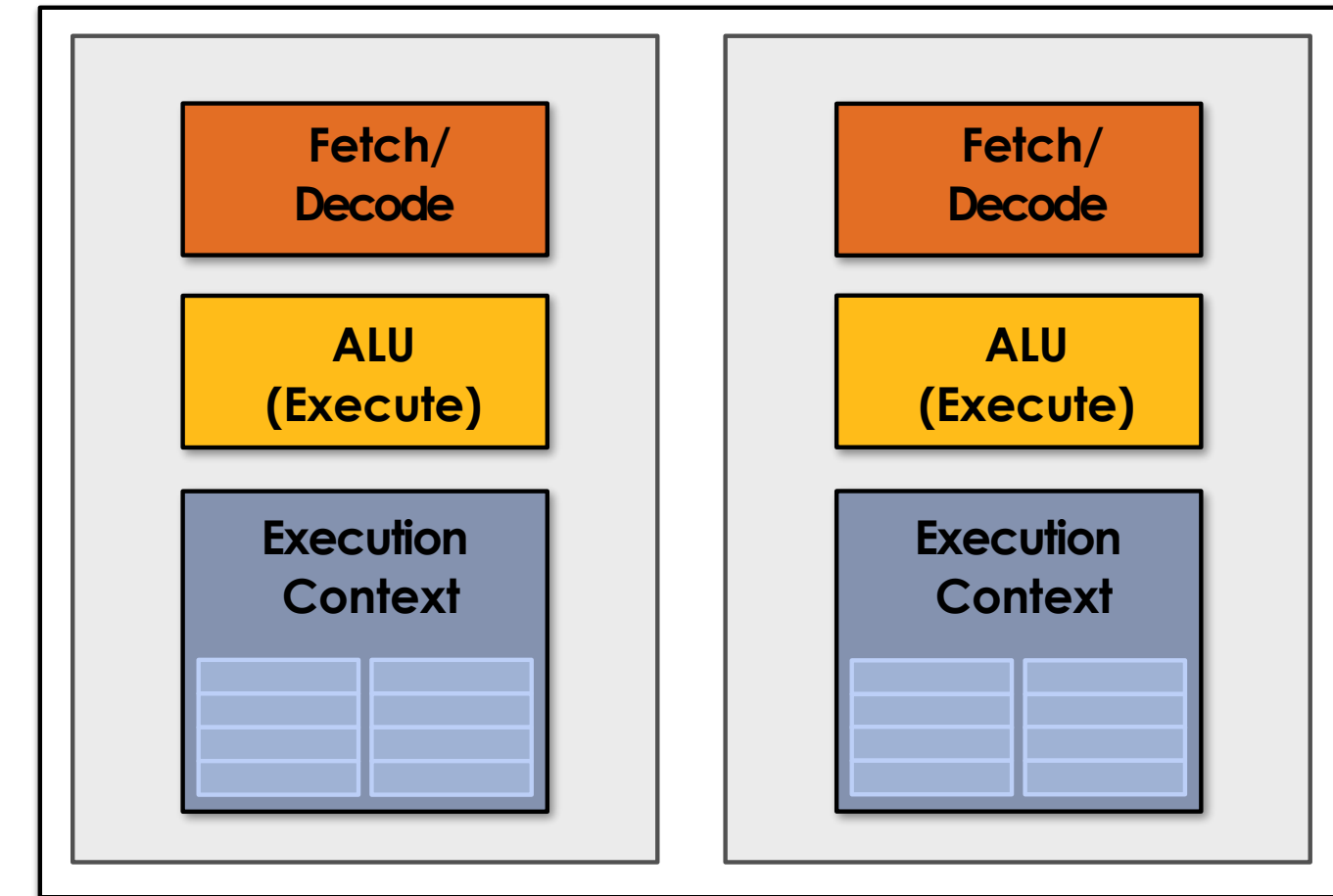
- **Brook compiler translated generic stream program into graphics commands (such as drawTriangles) and a set of graphics shader programs that could be run on GPUs of the day.**

GPU compute mode

Review: how to run code on a CPU

Lets say a user wants to run a program on a multi-core CPU...

- OS loads program text into memory OS
- selects CPU execution context
- OS interrupts processor, prepares execution context (sets contents of registers, program counter, etc. to prepare execution context)
- Go!
- Processor begins executing instructions from within the environment maintained in the execution context.



Multi-core CPU

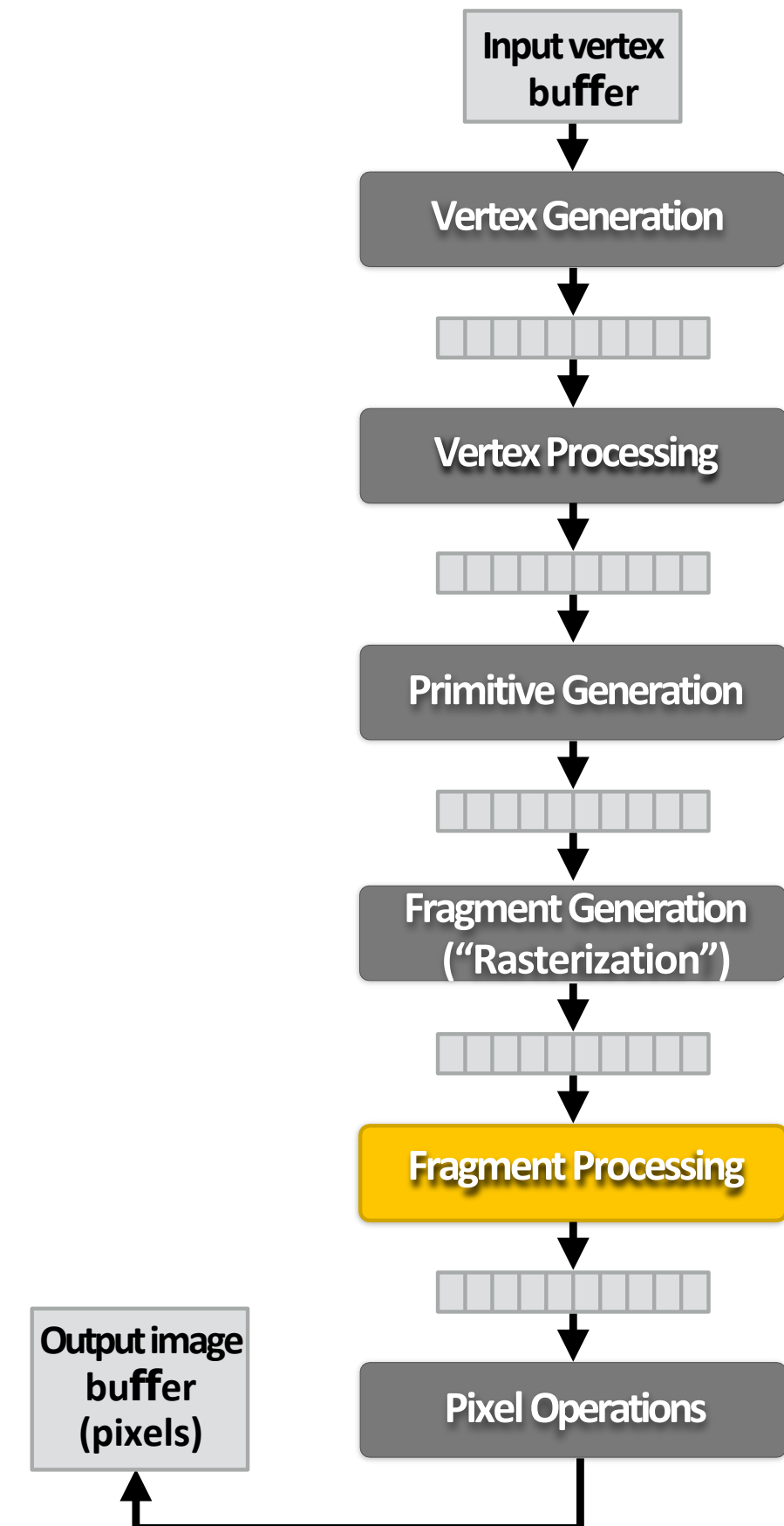
How to run code on a GPU (prior to 2007)

Let's say a user wants to draw a picture using a GPU...

- Application (via graphics driver) provides GPU shader program binaries
- Application sets graphics pipeline parameters (e.g., output image size)
- Application provides GPU a buffer of vertices
- Application sends GPU a "draw" command:
`drawPrimitives(vertex_buffer)`

This was the only interface to GPU hardware.

GPU hardware could only execute graphics pipeline computations.

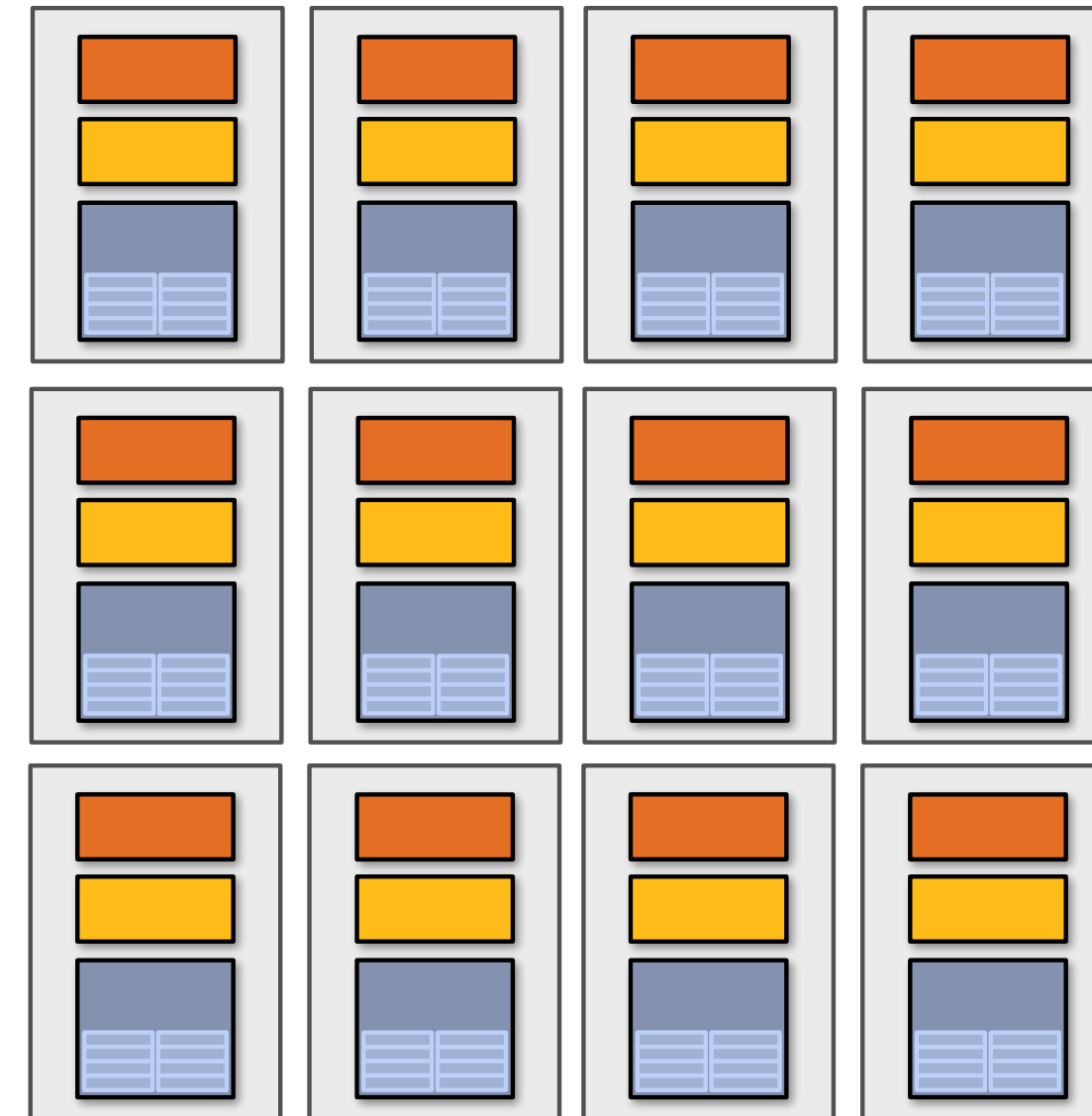


NVIDIA Tesla architecture (2007)

First alternative, non-graphics-specific (“compute mode”) interface to GPU hardware

Let’s say a user wants to run a non-graphics program on the GPU’s programmable cores...

- Application can allocate buffers in GPU memory and copy data to/from buffers
- Application (via graphics driver) provides GPU a single kernel program binary
- Application tells GPU to run the kernel in an SPMD fashion (“run N instances of this kernel”)
`launch(myKernel, N)`



Interestingly, this is a far simpler operation than the graphics operation `drawPrimitives()`

CUDA programming language

- Introduced in 2007 with NVIDIA Tesla architecture
- “C-like” language to express programs that run on GPUs using the compute-mode hardware interface
- Relatively low-level: CUDA’s abstractions closely match the capabilities/performance characteristics of modern GPUs (design goal: maintain low abstraction distance)
- Note: OpenCL is an open standards version of CUDA
 - CUDA only runs on NVIDIA GPUs
 - OpenCL runs on CPUs and GPUs from many vendors
 - Almost everything I say about CUDA also holds for OpenCL
 - CUDA is better documented

The plan

1. **CUDA programming abstractions**
2. **CUDA implementation on modern GPUs**
3. **More detail on GPU architecture**

Things to consider throughout this lecture:

- **Is CUDA a data-parallel programming model?**
- **Is CUDA an example of the shared address space model?**
- **Or the message passing model?**
- **Can you draw analogies to ISPC instances and tasks?**

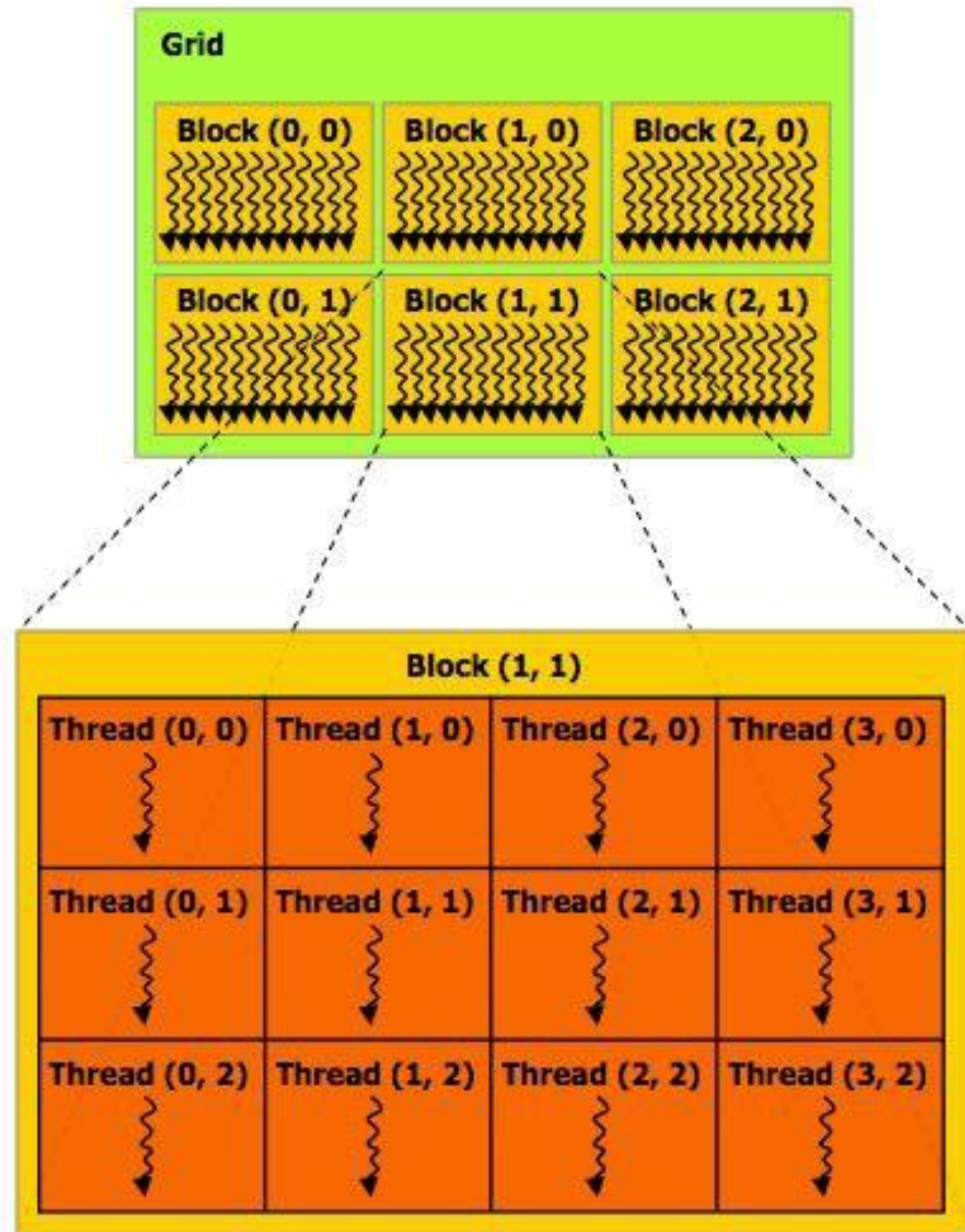
Clarification

- I am going to describe CUDA abstractions using CUDA terminology
- Specifically, be careful with the use of the term CUDA thread. A CUDA thread presents a similar abstraction as a pthread in that both correspond to logical threads of control, but the implementation of a CUDA thread is very different
- We will discuss these differences at the end of the lecture

CUDA programs consist of a hierarchy of concurrent threads

Thread IDs can be up to 3-dimensional (2D example below)

Multi-dimensional thread ids are convenient for problems that are naturally N-D



Regular application thread running on CPU (the “host”)

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will launch 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Basic CUDA syntax

“Host” code : serial execution

Running as part of normal C/C++ application on CPU

Bulk launch of many CUDA threads “launch a grid of CUDA thread blocks”

Call returns when all threads have terminated

Regular application thread running on CPU (the “host”)

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/
threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will launch 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

SPMD execution of device kernel function:

“CUDA device” code: kernel function (`__global__` denotes a CUDA kernel function) runs on GPU

Each thread computes its overall grid thread id from its position in its block (threadIdx) and its block’s position in the grid (blockIdx)

CUDA kernel definition

```
// kernel definition (runs on GPU)
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + B[j][i];
}
```

Clear separation of host and device code

Separation of execution into host and device code is performed statically by the programmer

“Host” code : serial execution on CPU

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAddDoubleB<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

“Device” code (SPMD execution on GPU)

```
__device__ float doubleValue(float x)
{
    return 2 * x;
}

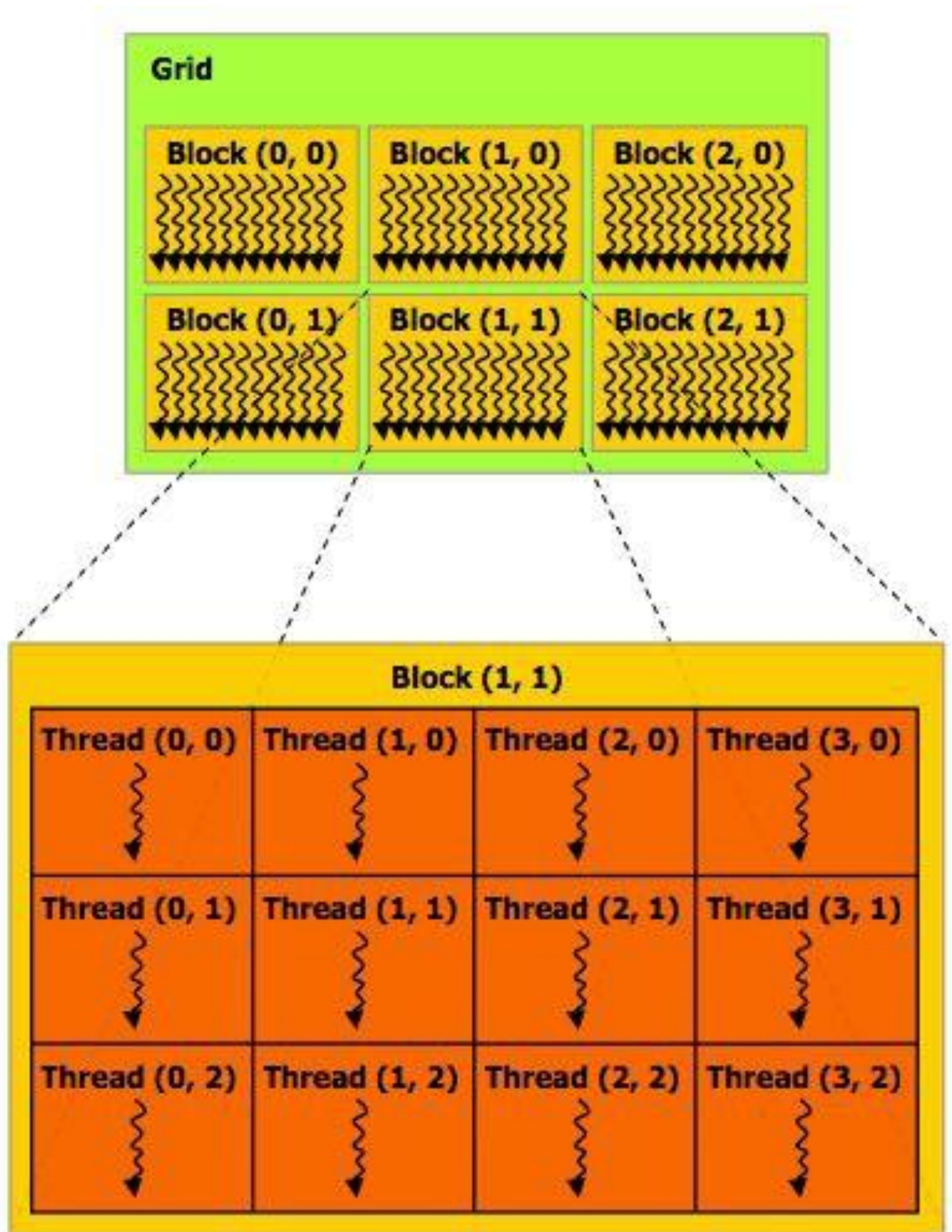
// kernel definition
__global__ void matrixAddDoubleB(float A[Ny][Nx],
                                float B[Ny][Nx],
                                float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + doubleValue(B[j][i]);
}
```

Number of SPMD “CUDA threads” is explicit in the program

Number of kernel invocations is not determined by size of data collection

(a kernel launch is not specified by `map(kernel, collection)` as was the case with graphics shader programming)



Regular application thread running on CPU (the “host”)

```
const int Nx = 11; // not a multiple of threadsPerBlock.x
const int Ny = 5;  // not a multiple of threadsPerBlock.y

dim3 threadsPerBlock(4, 3);
dim3 numBlocks((Nx+threadsPerBlock.x-1)/threadsPerBlock.x,
               (Ny+threadsPerBlock.y-1)/threadsPerBlock.y);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

CUDA kernel definition

```
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // guard against out of bounds array access
    if (i < Nx && j < Ny)
        C[j][i] = A[j][i] + B[j][i];
}
```

CUDA execution model

Host
(serial execution)



Implementation: CPU

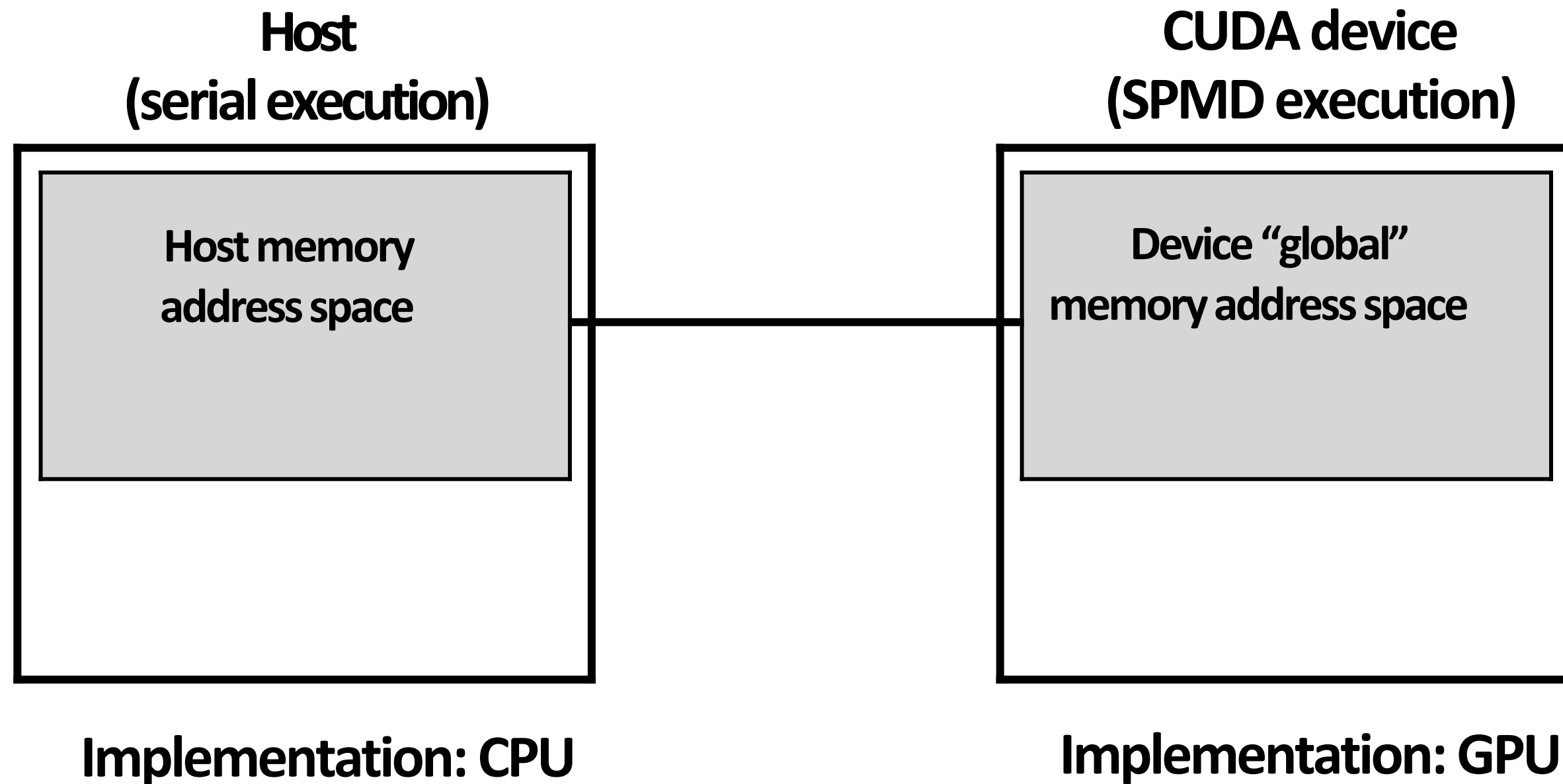
CUDA device
(SPMD execution)



Implementation: GPU

CUDA memory model

Distinct host and device address spaces



memcpy primitive

Move data between address spaces

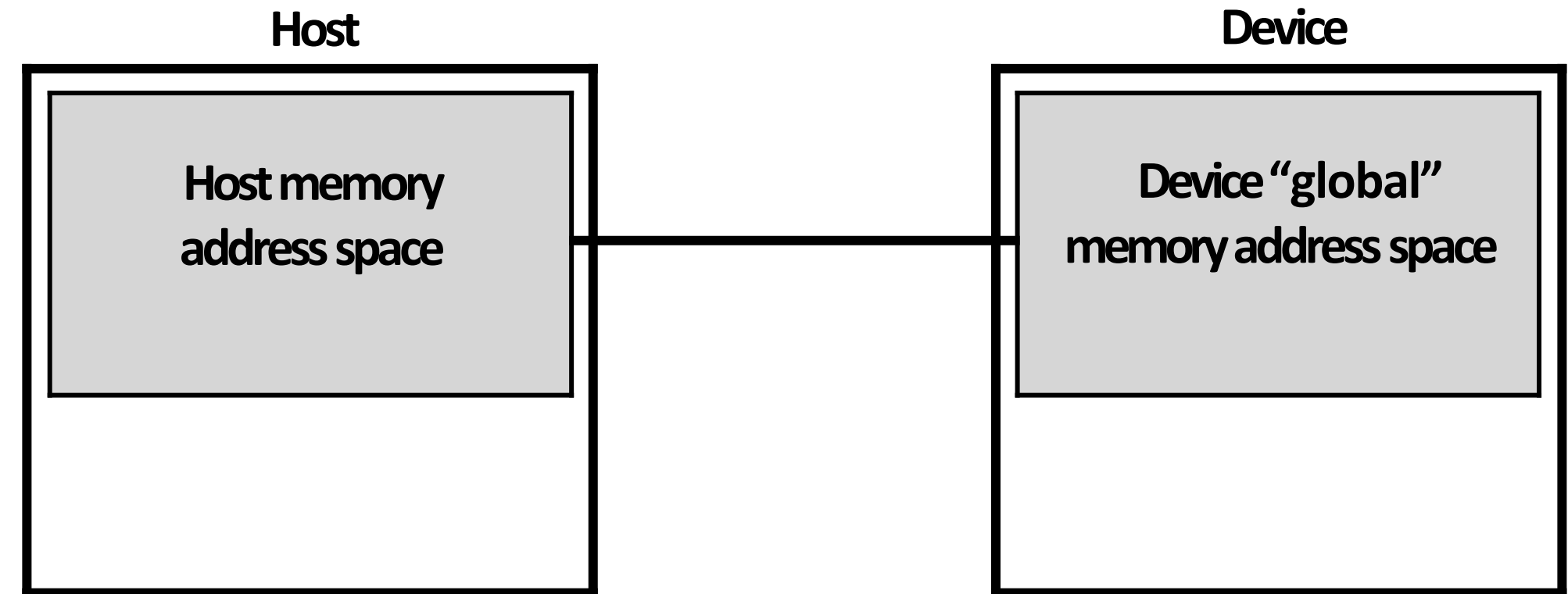
```
float* A = new float[N]; // allocate buffer in host mem

// populate host address space pointer A
for (int i=0 i<N; i++)
    A[i] = (float)i;

int bytes = sizeof(float) * N;
float* deviceA;           // allocate buffer in
cudaMalloc(&deviceA, bytes); // device address space

// populate deviceA
cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);

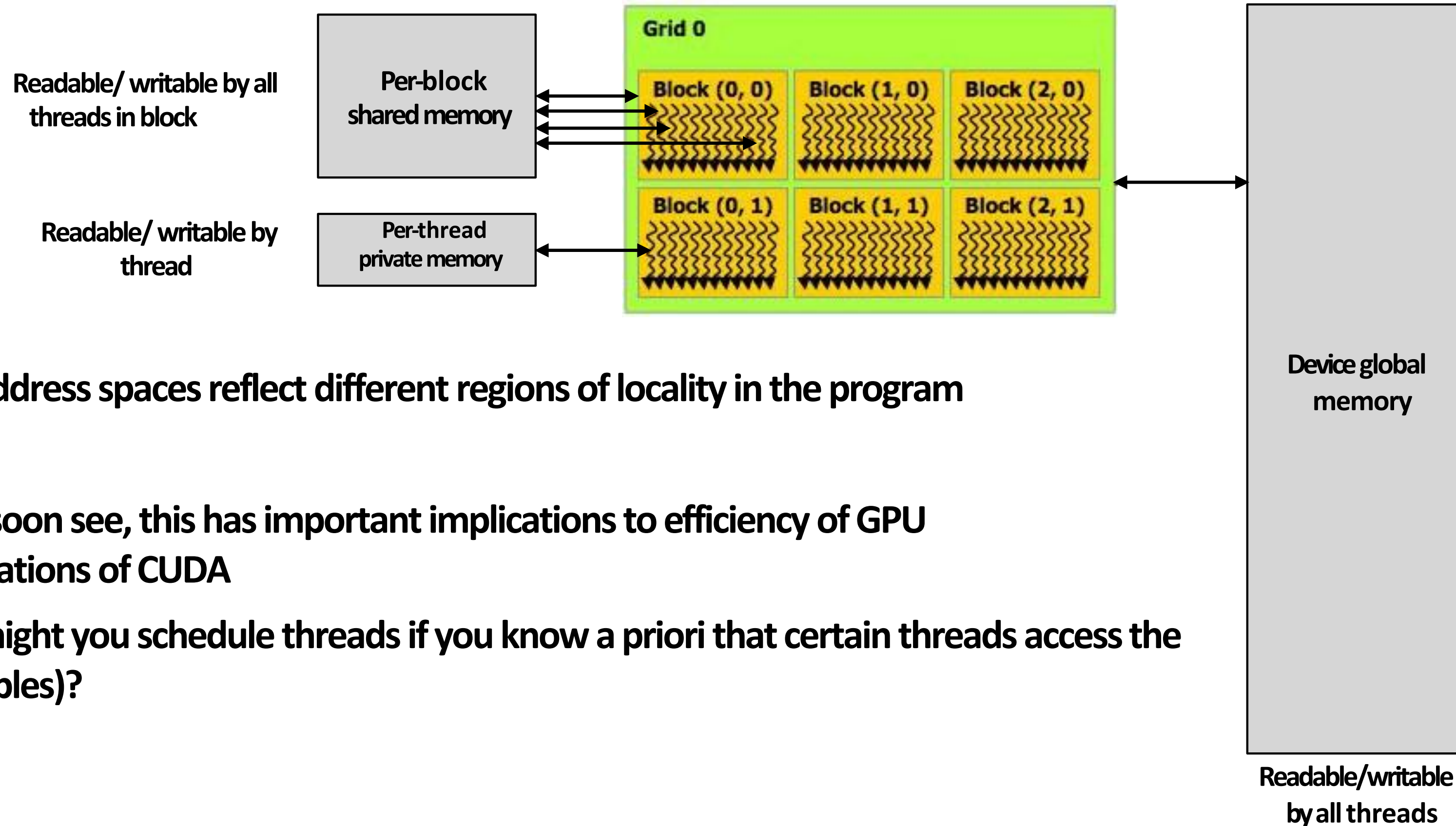
// note: directly accessing deviceA[i] is an invalid
// operation here (cannot manipulate contents of deviceA
// directly from host, since deviceA is not a pointer
// into the host's address space)
```



What does cudaMemcpy remind you of?

CUDA device memory model

Three distinct types of address spaces visible to kernels

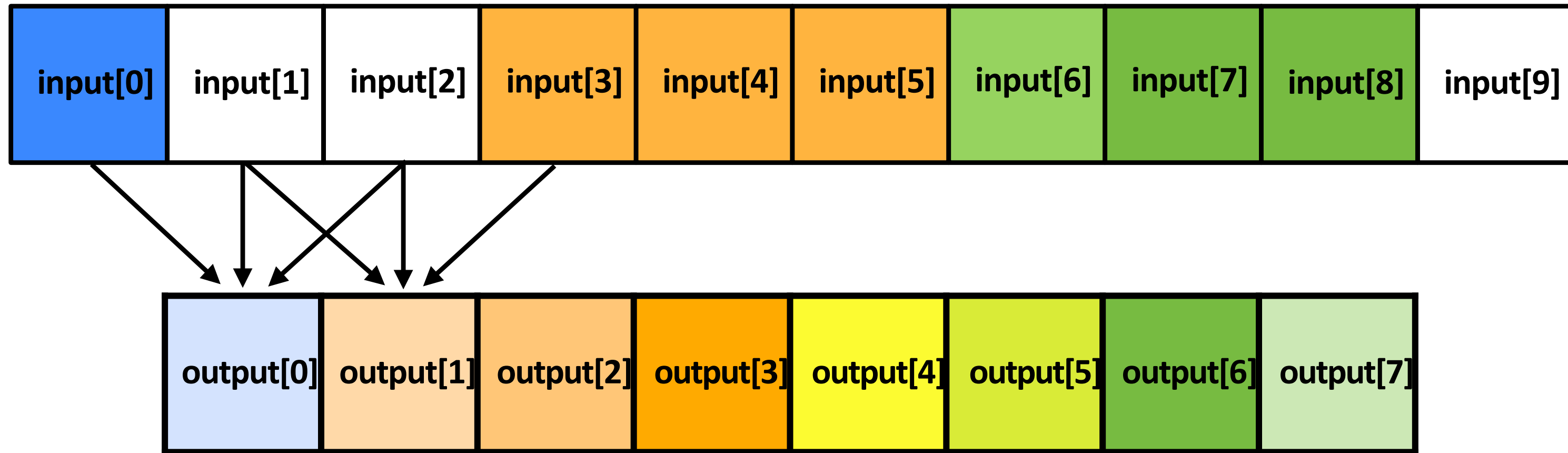


Different address spaces reflect different regions of locality in the program

As we will soon see, this has important implications to efficiency of GPU implementations of CUDA

e.g., how might you schedule threads if you know a priori that certain threads access the same variables)?

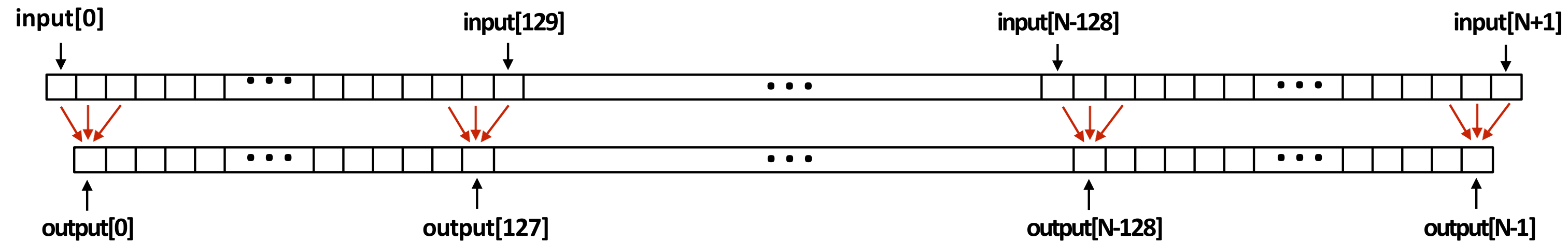
CUDA example: 1D convolution



```
output[i] = (input[i] + input[i+1] + input[i+2]) / 3.f;
```

1D convolution in CUDA (version 1)

One thread per output element



CUDAKernel

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += input[index + i];

    output[index] = result / 3.f;
}
```

each thread computes result
for one element

each thread writes result to
global memory

Host code

```
int N = 1024 * 1024;
cudaMalloc(&devInput, sizeof(float) * (N+2) ); // allocate input array in device memory
cudaMalloc(&devOutput, sizeof(float) * N);      // allocate output array in device memory

// properly initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>>(N, devInput, devOutput);
```

1D convolution in CUDA (version 2)

One thread per output element: stage input data in per-block shared memory

CUDA Kernel

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output)
{
    __shared__ float support[THREADS_PER_BLK+2]; // per-block allocation
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK + threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result / 3.f;
}
```

All threads cooperatively load
block's support region from
global memory into shared
memory

(total of 130 load instructions instead
of 3 * 128 load instructions)

barrier (all threads in block)

each thread computes
result for one element

write result to global
memory

Host code

```
int N = 1024 * 1024
cudaMalloc(&devInput, sizeof(float) * (N+2) ); // allocate array in device memory
cudaMalloc(&devOutput, sizeof(float) * N);      // allocate array in device memory

// property initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

CUDA synchronization constructs

- syncthreads()
 - Barrier: wait for all threads in the block to arrive at this point
- Atomic operations
 - e.g., `float atomicAdd(float* addr, float amount)`
 - CUDA provides atomic operations on both global memory addresses and per-block shared memory addresses
- Host/device synchronization
 - Implicit barrier across all threads at return of kernel

Summary: CUDA abstractions

- Execution: thread hierarchy
 - Bulk launch of many threads (this is imprecise... I'll clarify later)
 - Two-level hierarchy: threads are grouped into thread blocks
- Distributed address space
 - Built-in memcpy primitives to copy between host and device address spaces
 - Three different types of device address spaces
 - Per thread, per block (“shared”), or per program (“global”)
- Barrier synchronization primitive for threads in thread block
- Atomic primitives for additional synchronization (shared and global variables)

CUDA semantics

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    __shared__ float support[THREADS_PER_BLK+2]; // per-block allocation
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local var

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result / 3.f;
}

// host code ////////////////////////////////////////
int N = 1024 * 1024;
cudaMalloc(&devInput, N+2); // allocate array in device memory
cudaMalloc(&devOutput, N); // allocate array in device memory

// property initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>>(N, devInput, devOutput);
```

Consider implementation of call to `pthread_create()` or `std::thread()`:

Allocate thread state:

- Stack space for thread
- Allocate control block so OS can schedule thread

Will running this CUDA program create 1 million instances of local variables/per-thread stack?

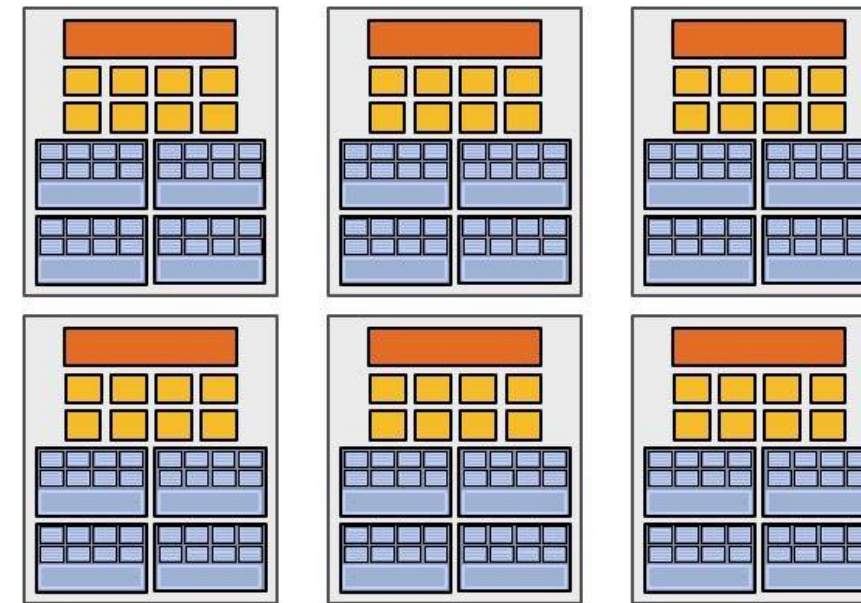
8K instances of shared variables? (support)

Launch over 1 million CUDA threads (over 8K thread blocks)

Assigning work



High-end GPU
(16 cores)



Mid-range GPU
(6 cores)

Desirable for CUDA program to run on all of these GPUs without modification

Note: there is no concept of `num_cores` in the CUDA programs I have shown you. (CUDA thread launch is similar in spirit to a `forall` loop in data parallel model examples)

CUDA compilation

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    __shared__ float support[THREADS_PER_BLK+2]; // per block allocation
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local var

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result;
}
```

```
int N = 1024 * 1024;
cudaMalloc(&devInput, N+2); // allocate array in device memory
cudaMalloc(&devOutput, N); // allocate array in device memory

// property initialize contents of devInput here ...

convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

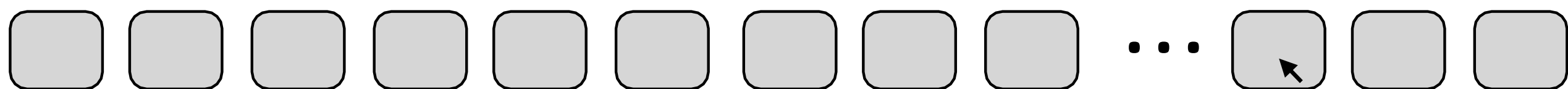
A compiled CUDA device binary includes:

Program text (instructions) Information
about required resources:

- 128 threads per block
- B bytes of local data per thread
- 128+2=130 floats (520 bytes) of shared space per thread block

———— launch 8K thread blocks

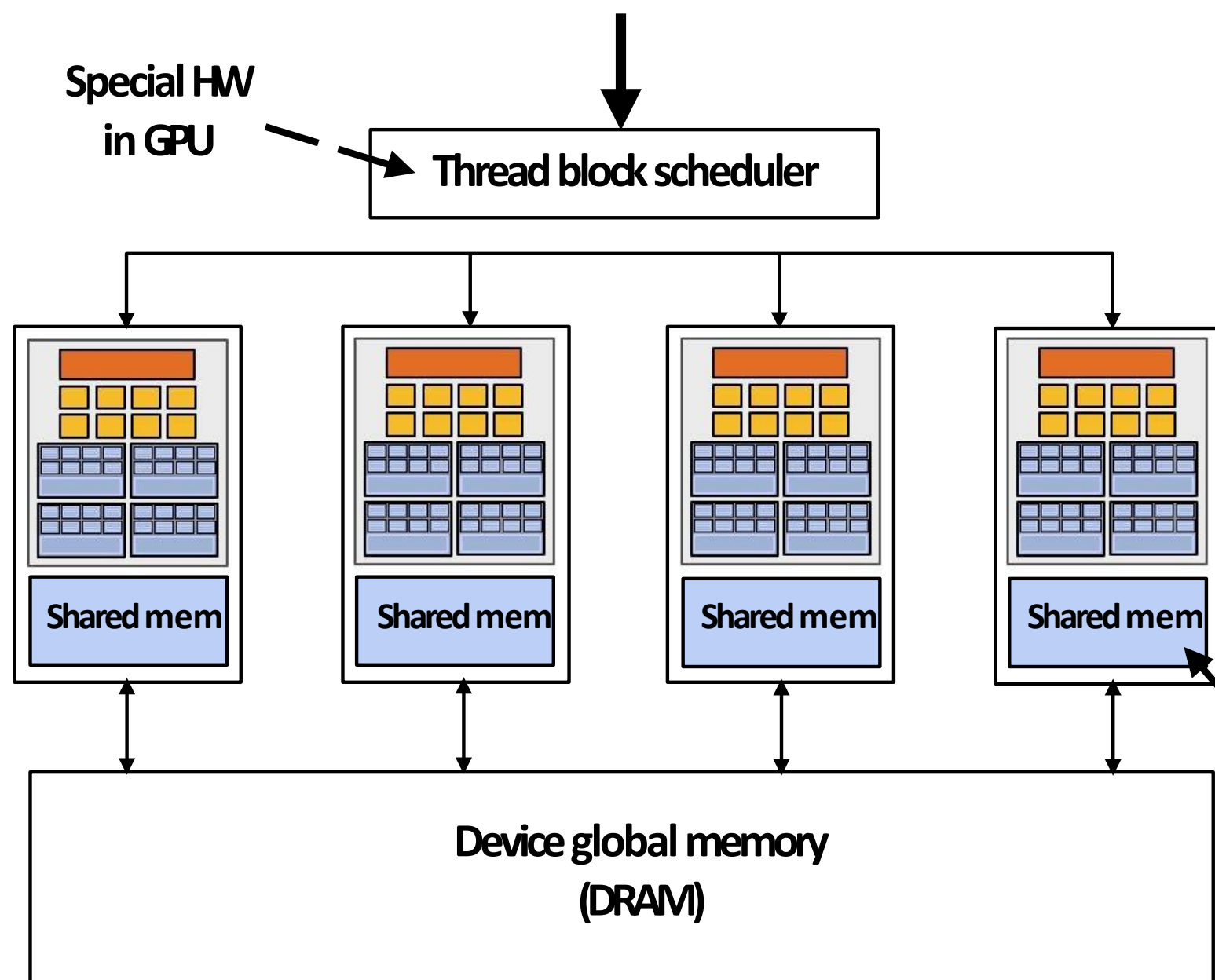
CUDA thread-block assignment



Grid of 8K convolve thread blocks (specified by kernel launch)

Block resource requirements:
(contained in compiled kernel binary)
128 threads
520 bytes of shared mem
(128 x B) bytes of local mem

Kernel launch command from host
`launch(blockDim, convolve)`

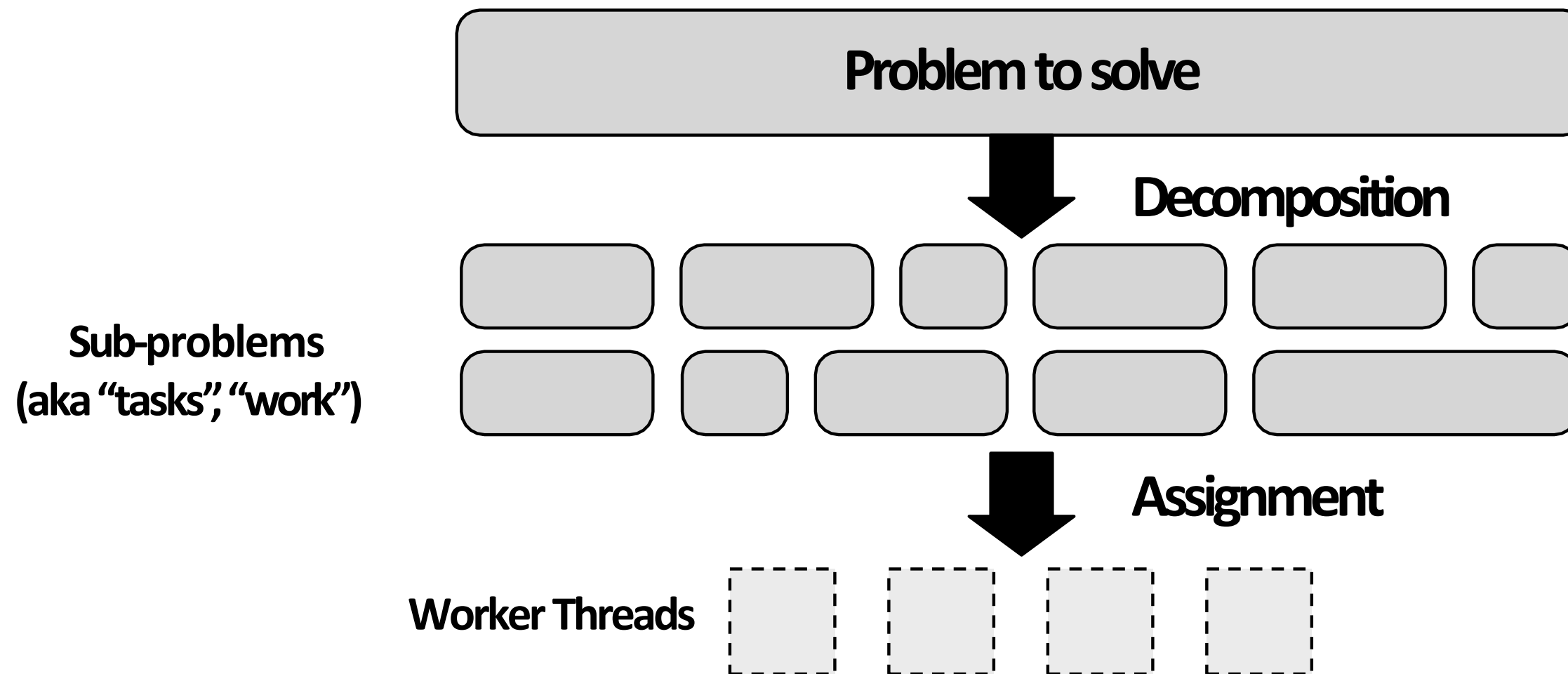


Major CUDA assumption: thread block execution can be carried out in any order (no dependencies between blocks)

GPU implementation maps thread blocks (“work”) to cores using a dynamic scheduling policy that respects resource requirements

Shared mem is fast
on-chip memory

Another instance of our common design pattern: a pool of worker “threads”



Other examples:

- ISPC’s implementation of launching tasks
 - Creates one pthread for each hyper-thread on CPU. Threads kept alive for remainder of program
- Thread pool in a web server
 - Number of threads is a function of number of cores, not number of outstanding requests
 - Threads spawned at web server launch, wait for work to arrive

NVIDIA V100 SM “sub-core”

 = SIMD fp32 functional unit,
control shared across 16 units
(16 x MUL-ADD per clock *)

 = SIMD int functional unit, control
shared across 16 units (16 x
MUL/ADD per clock *)

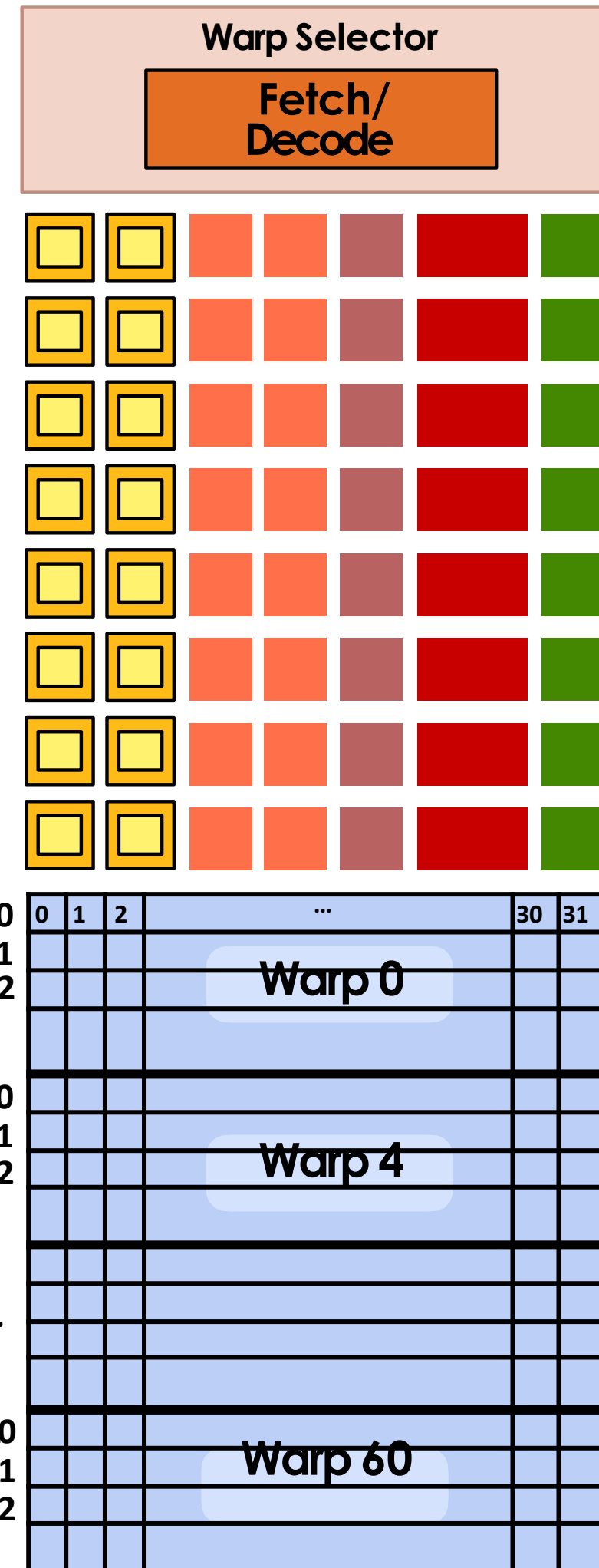
 = SIMD fp64 functional unit,
control shared across 8 units
(8 x MUL/ADD per clock **)

 = Tensor core unit

 = Load/store unit

* one 32-wide SIMD operation every two clocks

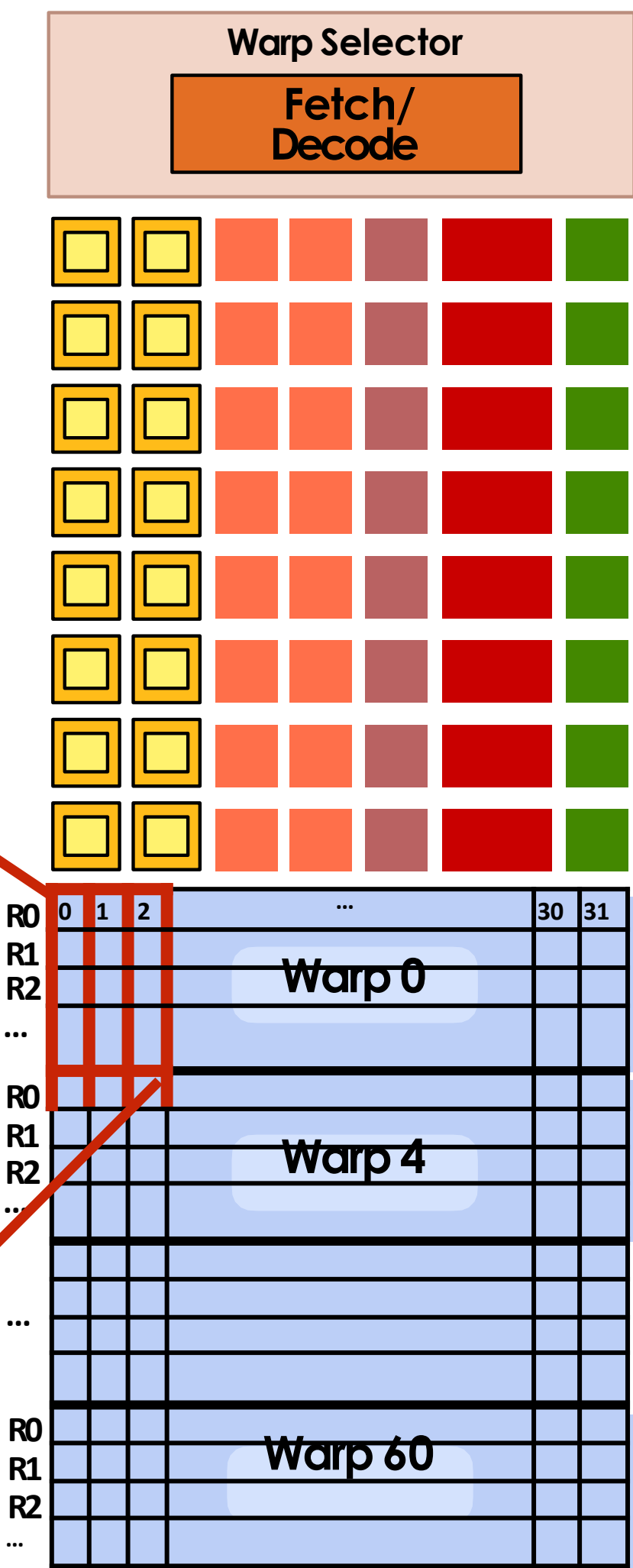
** one 32-wide SIMD operation every four clocks



NVIDIA V100 SM “sub-core”

Scalar registers for one CUDA thread: R0, R1, etc...

Scalar registers for another CUDA thread: R0, R1, etc...

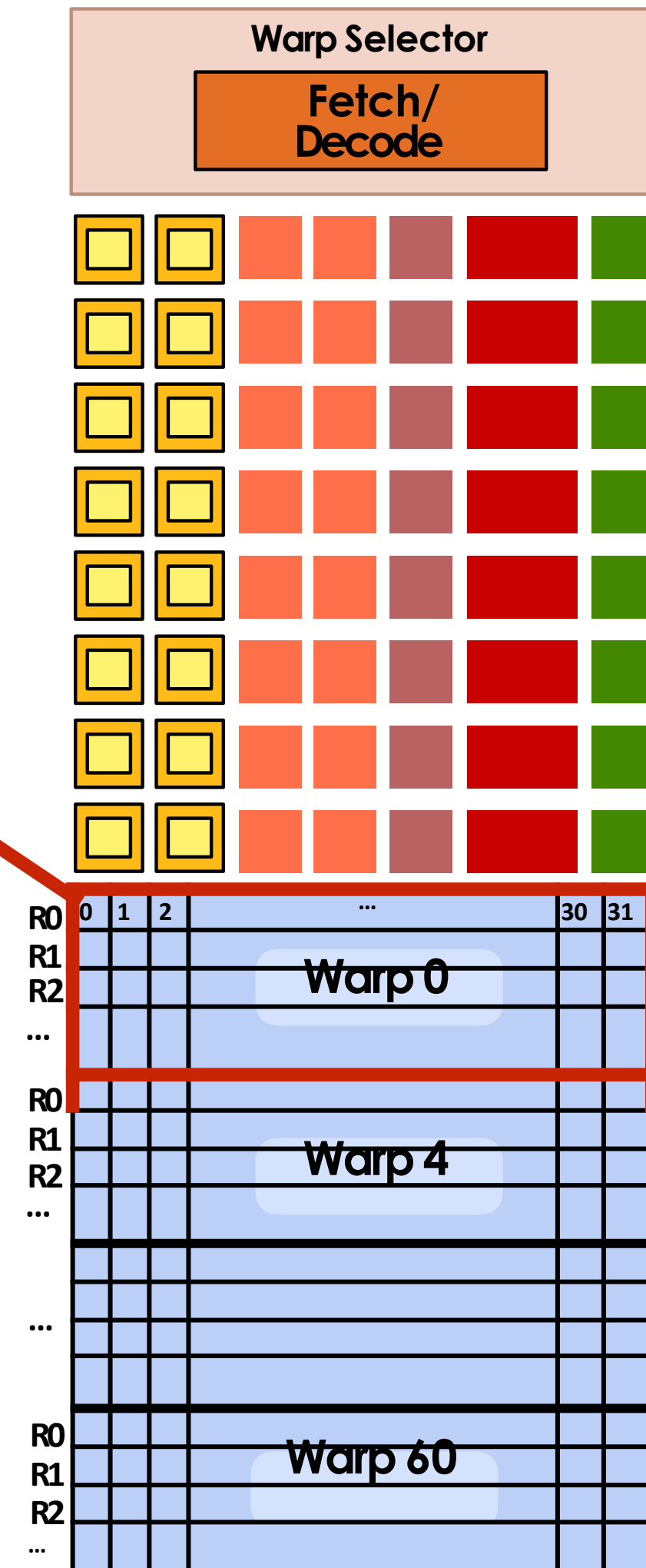


NVIDIA V100 SM “sub-core”

Scalar registers for 32 threads in the same “warp”

A group of 32 threads in thread block is called a warp.

- In a thread block, threads 0-31 fall into the same warp (so do threads 32-63, etc.)
- Therefore, a thread block with 256 CUDA threads is mapped to 8 warps.
- Each sub-core in the V100 is capable of scheduling and interleaving execution of up to 16 warps



NVIDIA V100 SM “sub-core”

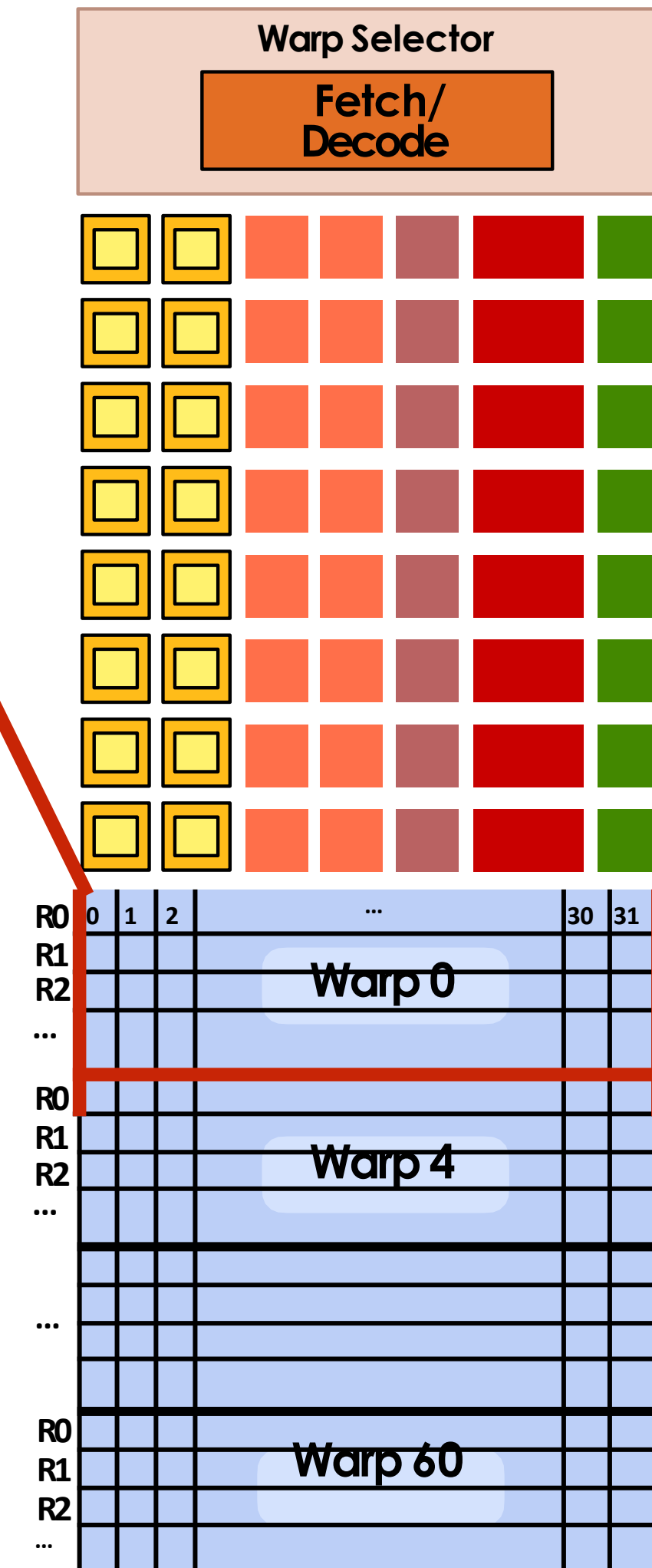
Scalar registers for 32 threads in the same “warp”

Threads in a warp are executed in a SIMD manner

if they share the same instruction

- If the 32 CUDA threads do not share the same instruction, performance can suffer due to divergent execution.
- This mapping is similar to how ISPC runs program instances in a gang *

A warp is not part of CUDA, but is an important CUDA implementation detail on modern NVIDIA GPUs

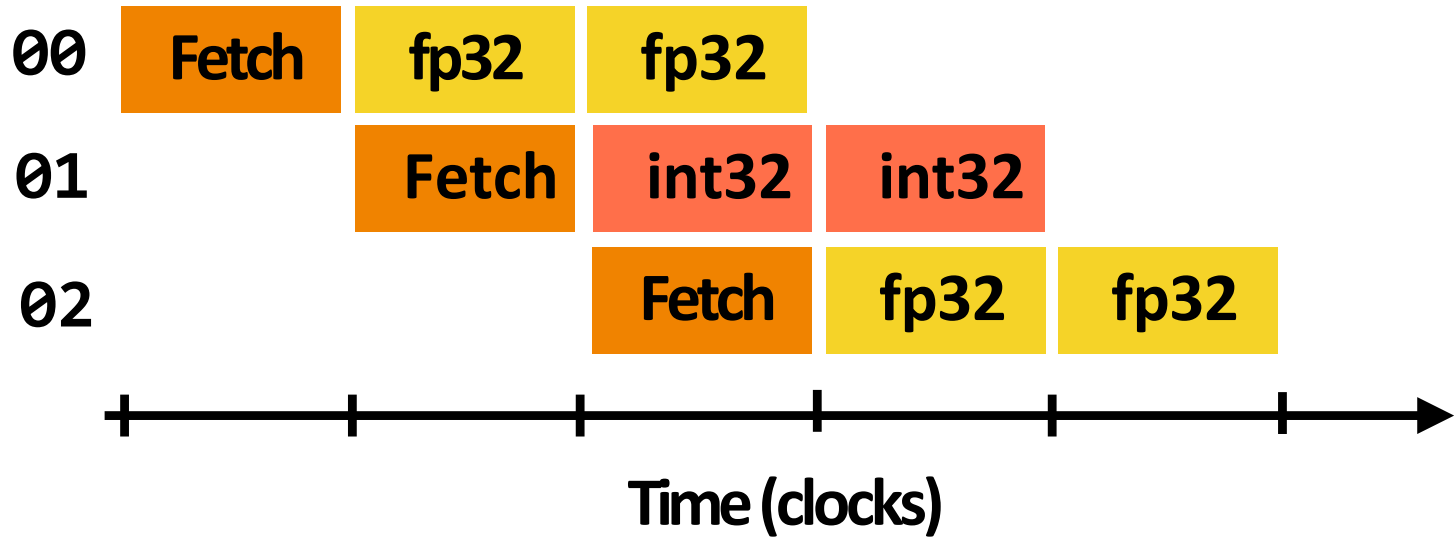


* But GPU hardware is dynamically checking whether 32 independent CUDA threads share an instruction, and if this is true, it executes all 32 threads in a SIMD manner. The CUDA program is not compiled to SIMD instructions like ISPC gangs.

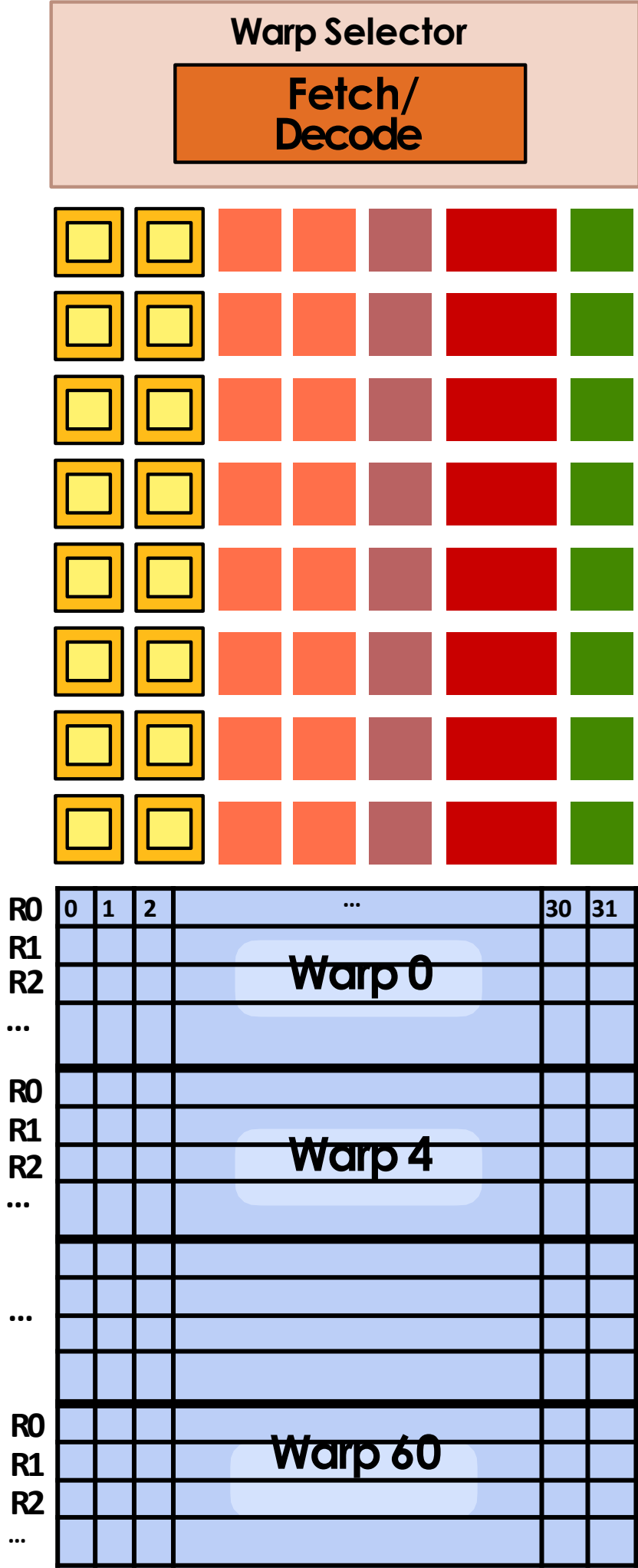
Instruction execution

Instruction stream for CUDA threads in a warp...
(note in this example all instructions are independent)

```
00  fp32  mul  r0 r1 r2
01  int32 add  r3 r4 r5
02  fp32  mul  r6 r7 r8
...
```

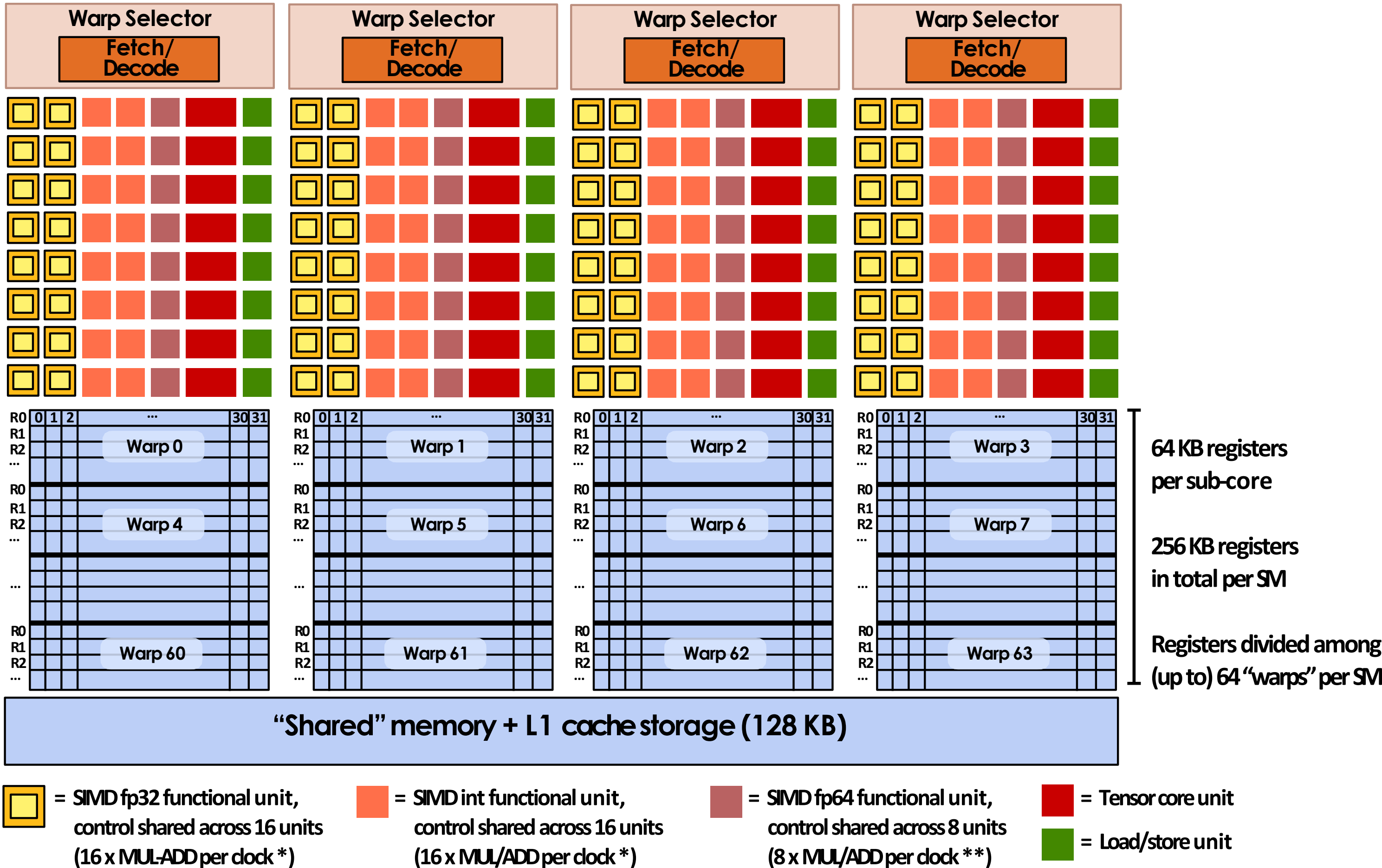


Remember, entire warp of CUDA threads is running this instruction stream. So each instruction is run by all 32 CUDA threads in the warp. Since there are 16 ALUs, running the instruction for the entire warp takes two clocks.



NVIDIA V100 GPU SM

This is one NVIDIA V100 streaming multi-processor (SM) unit

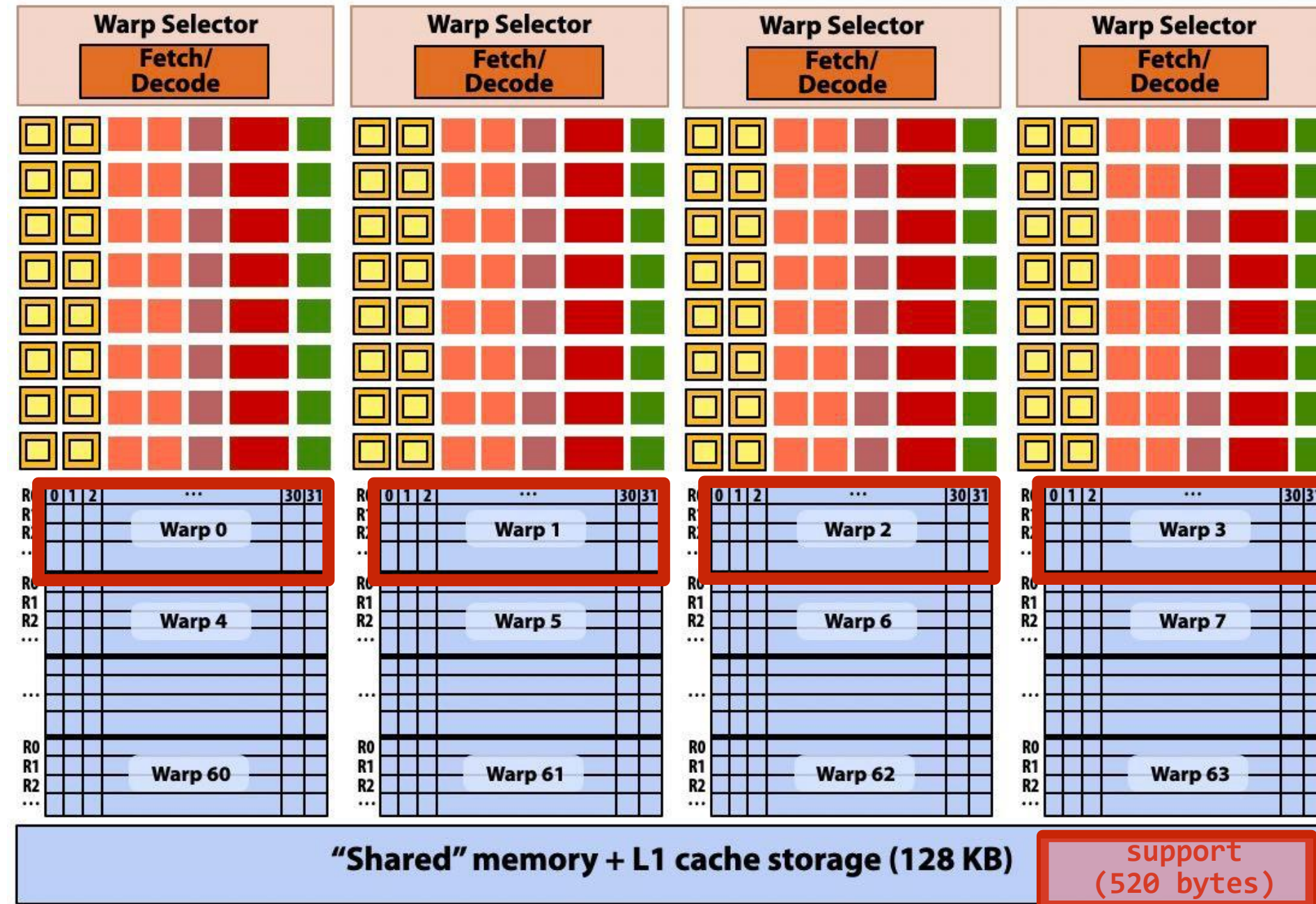


64 KB registers
per sub-core

256 KB registers
in total per SM

Registers divided among
(up to) 64 “warps” per SM

Running a thread block on a V100 SM



A `convolve` thread block is executed by 4 warps
(4 warps x 32 threads/warp = 128 CUDA threads per block)

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input,
                        float* output)
{
    __shared__ float support[THREADS_PER_BLK+2];
    int index = blockIdx.x * blockDim.x +
                threadIdx.x;

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x]
            = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

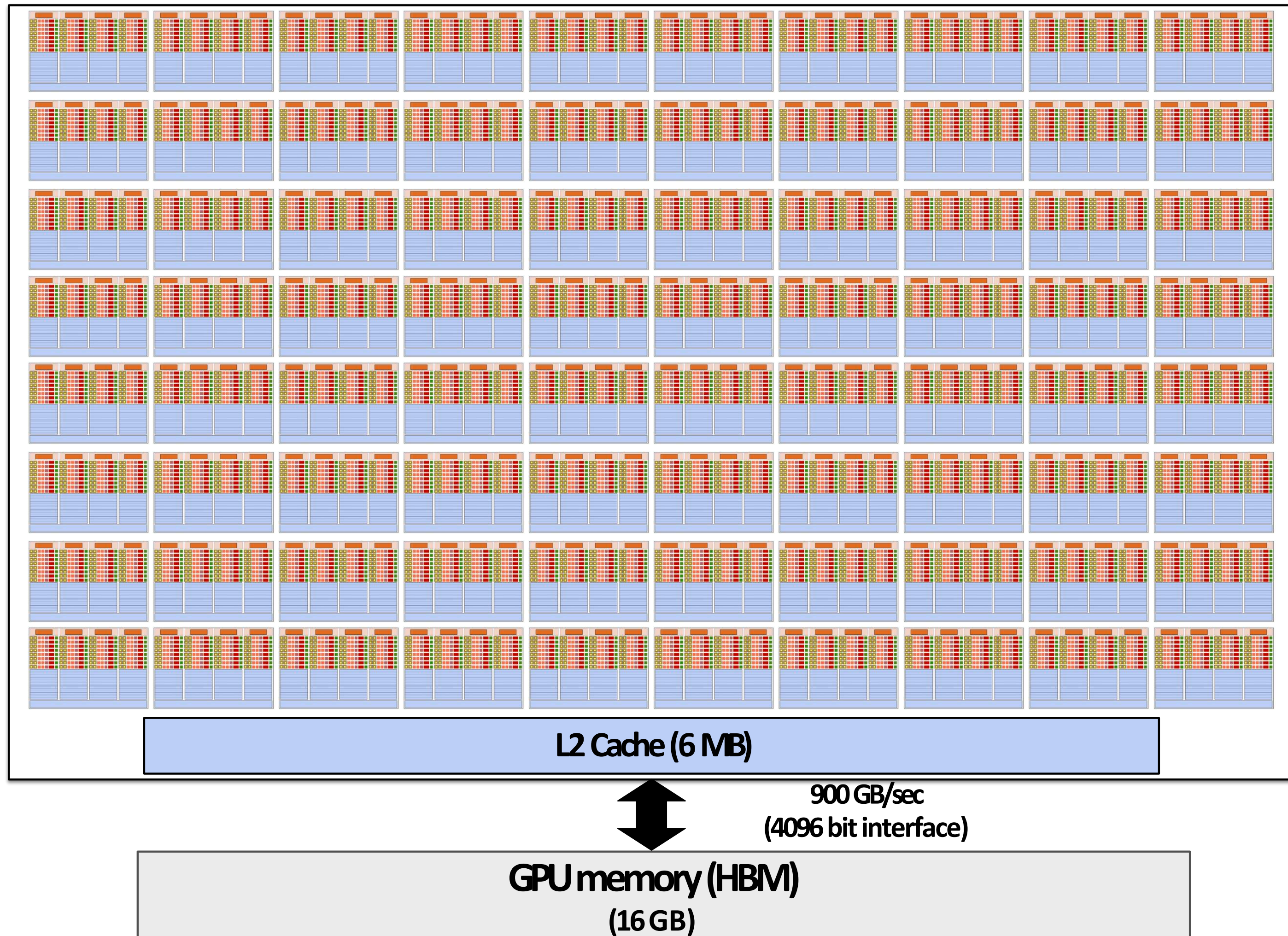
    float result = 0.0f; // thread-local
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result;
}
```

SM core operation each clock:

- Each sub-core selects one runnable warp (from the 16 warps in its partition)
- Each sub-core runs next instruction for the CUDA threads in the warp (this instruction may apply to all or a subset of the CUDA threads in a warp depending on divergence)

NVIDIA V100 GPU (80 SMs)



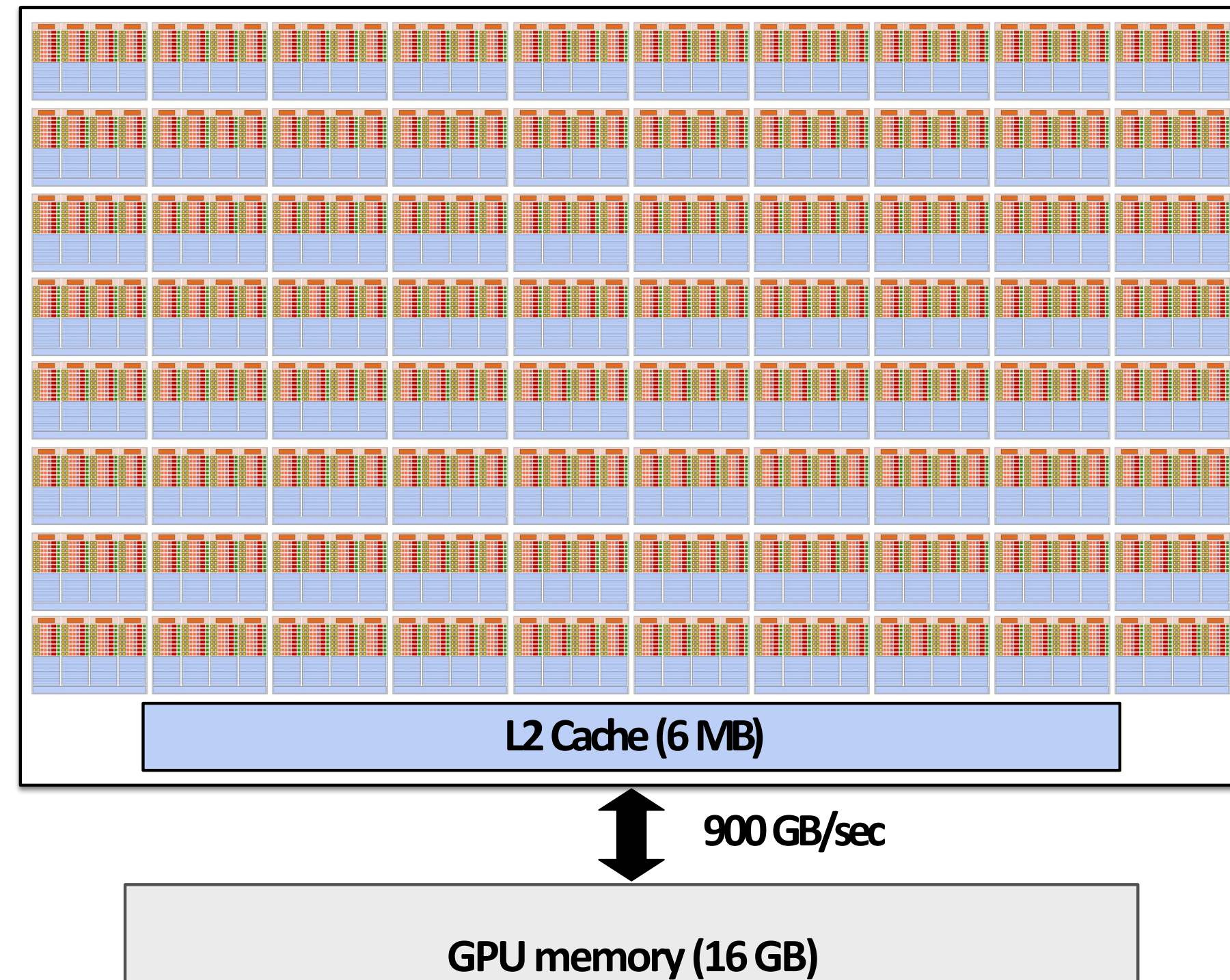
Summary: geometry of the V100 GPU

1.245 GHz clock

80 SM cores per chip

$80 \times 4 \times 16 = 5,120$ fp32 mul-add ALUs
= 12.7 TFLOPs *

Up to $80 \times 64 = 5120$ interleaved warps per chip
(163,840 CUDA threads/chip)



* mul-add counted as 2 flops:

Running a CUDA program on a GPU

Running the convolve kernel

convolve kernel's execution requirements:

Each thread block must execute 128 CUDA threads

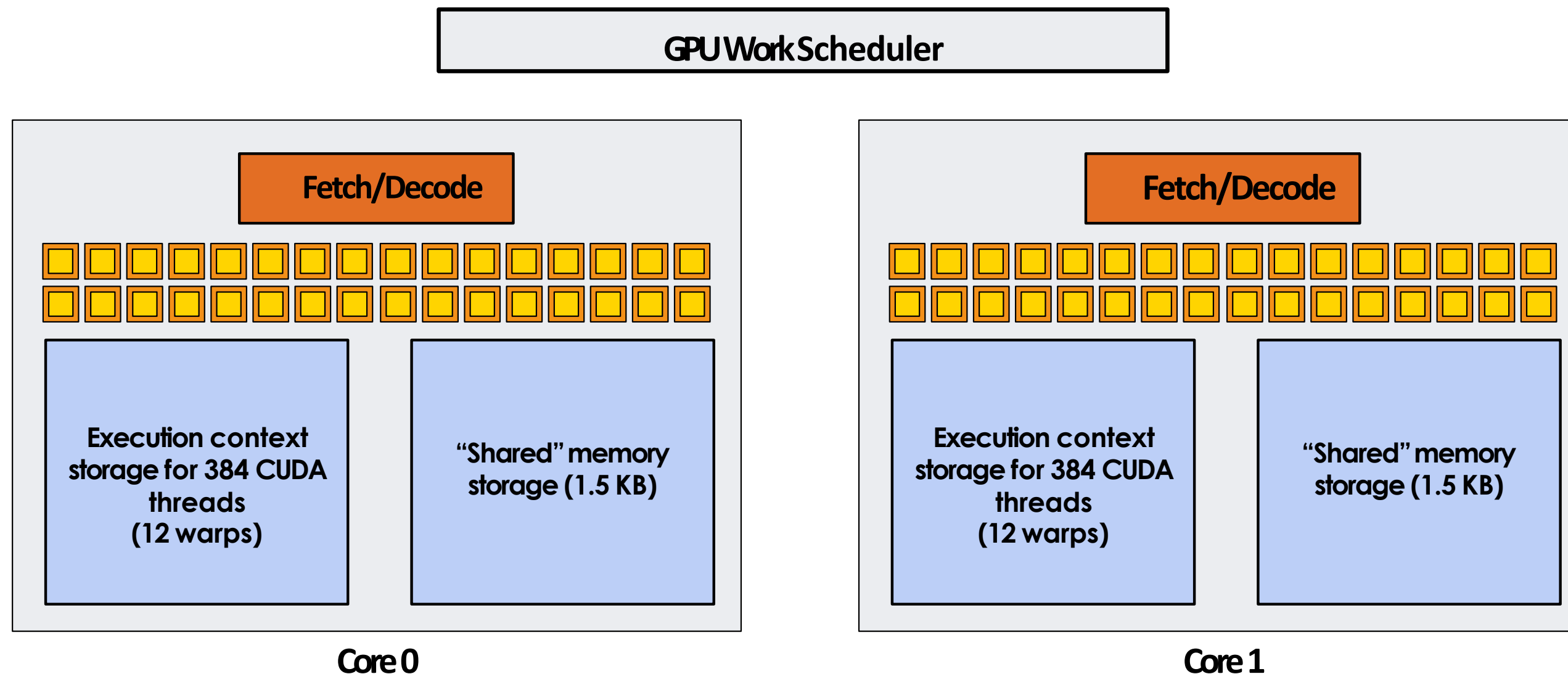
Each thread block requires $130 \times \text{sizeof}(\text{float}) = 520$ bytes of shared memory

Let's assume array size N is very large, so the host-side kernel launch generates thousands of thread blocks.

```
#define THREADS_PER_BLK 128  
convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>(N, input_array, output_array);
```

Let's run this program on the fictitious two-core GPU below.

(Note: the fictitious cores are much “smaller” than the V100 SM cores discussed earlier in lecture: they have fewer execution units, support for fewer active warps, less shared memory, etc.)



Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

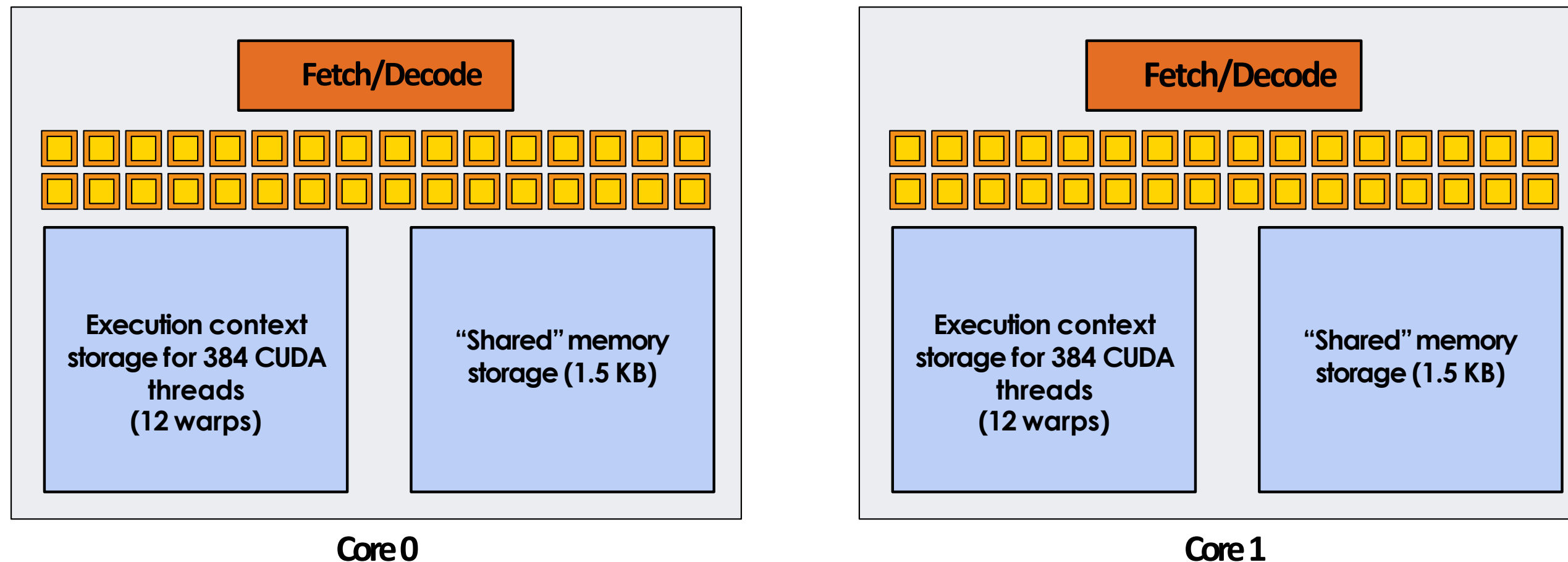
Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 1: host sends CUDA device (GPU) a command (“execute this kernel”)



EXECUTE: convolve
ARGS: N, input_array, output_array
NUM_BLOCKS: 1000

GPU WorkScheduler



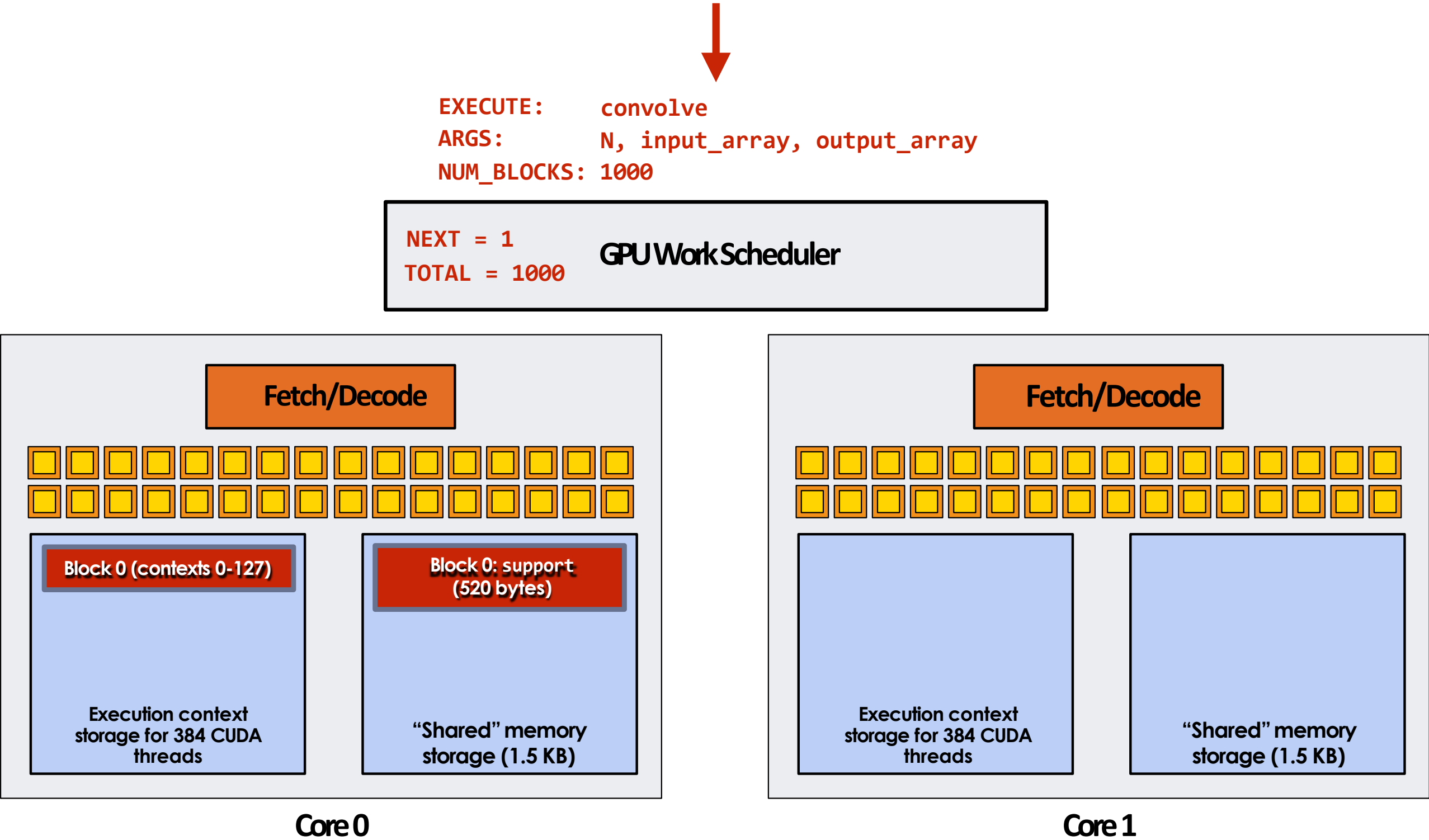
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 2: scheduler maps block 0 to core 0 (reserves execution contexts for 128 threads and 520 bytes of shared storage)



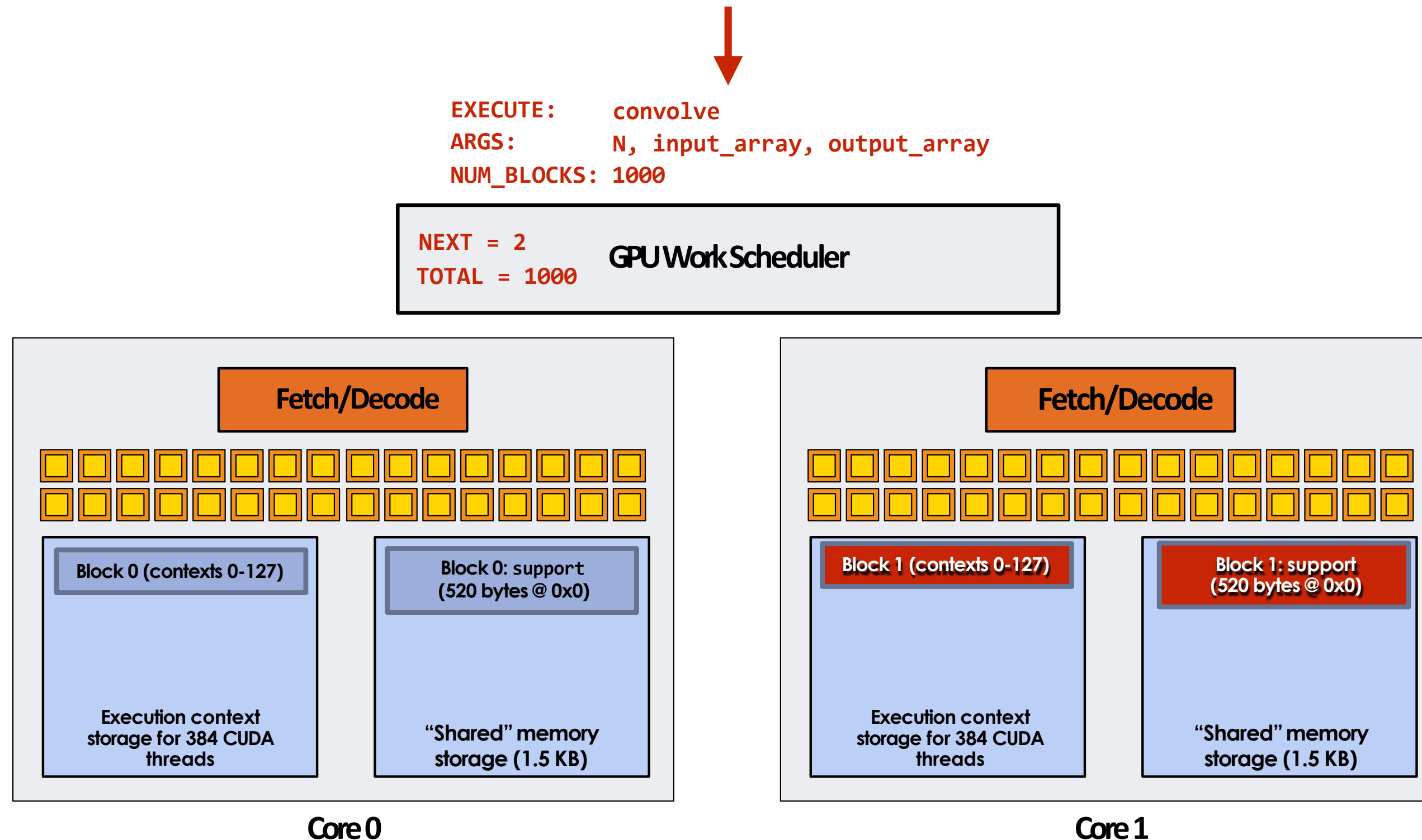
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 3: scheduler continues to map blocks to available execution contexts (interleaved mapping shown)



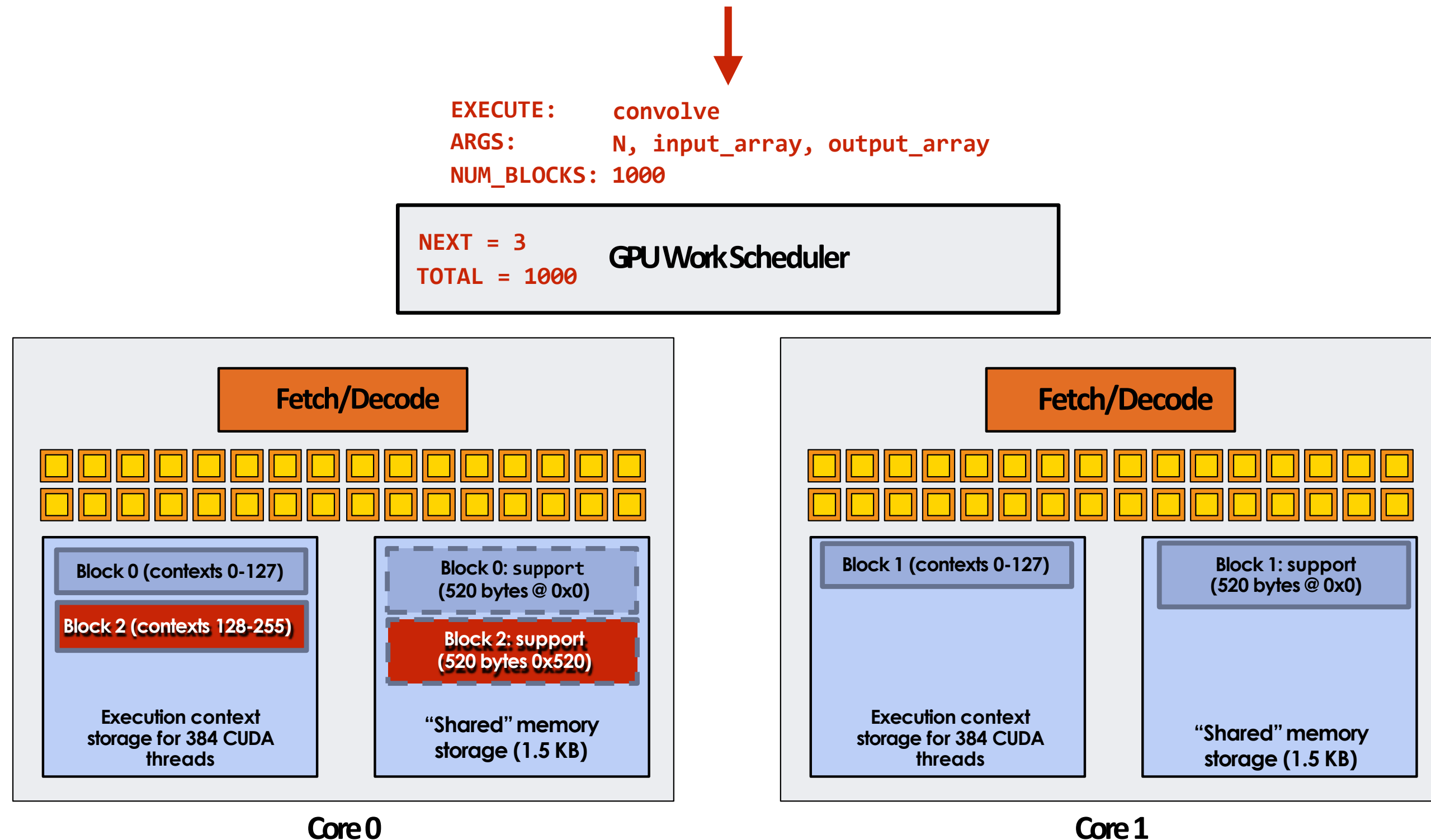
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof}(\text{float}) = 520$ bytes of shared memory

Step 3: scheduler continues to map blocks to available execution contexts (interleaved mapping shown)



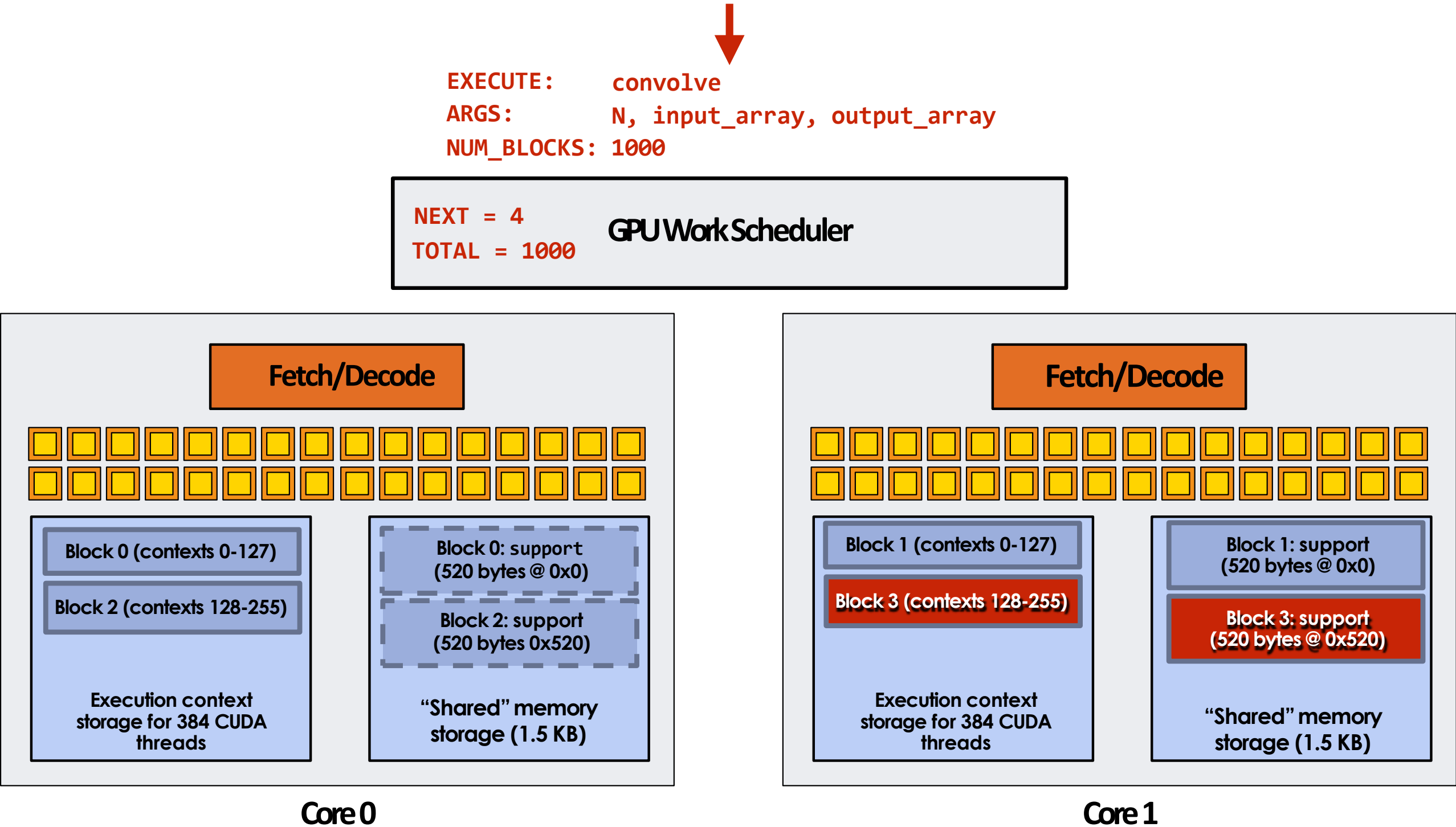
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 3: scheduler continues to map blocks to available execution contexts (interleaved mapping shown). Only two thread blocks fit on a core (third block won't fit due to insufficient shared storage $3 \times 520 \text{ bytes} > 1.5 \text{ KB}$)



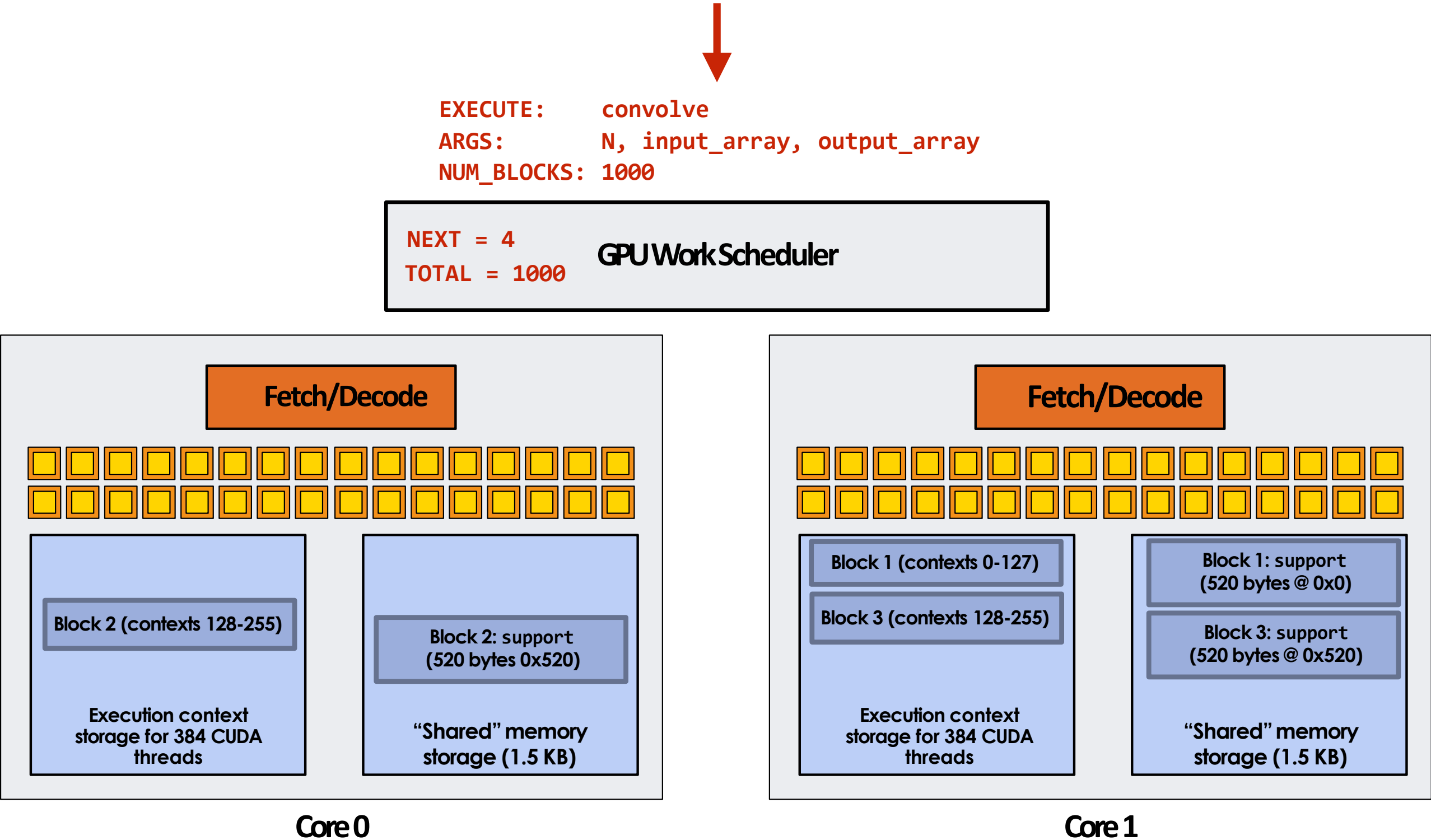
Running the CUDA kernel

Kernel's execution requirements:

- Each thread block must execute 128 CUDA threads

- Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 4: thread block 0 completes on core 0



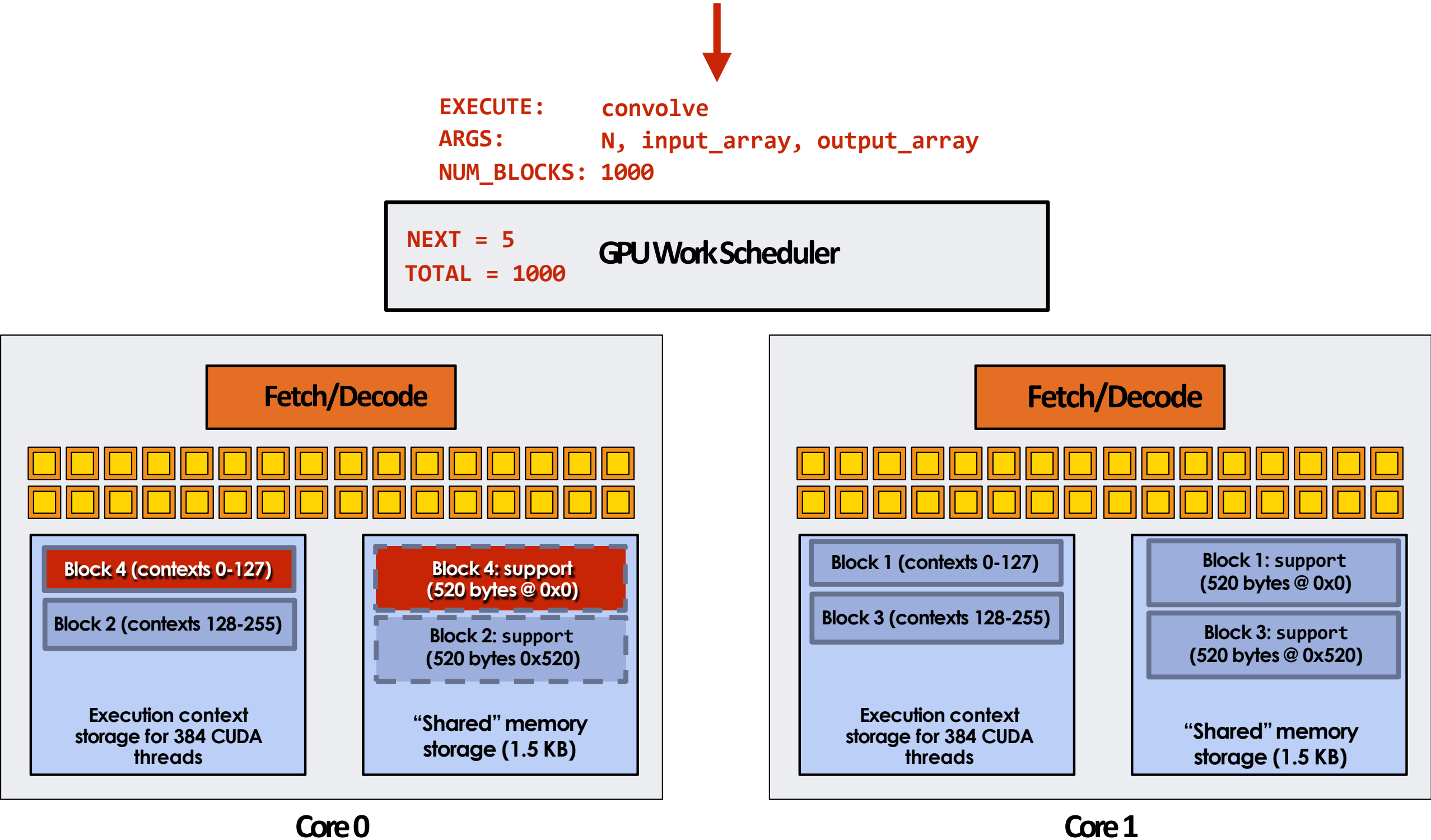
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 5: block 4 is scheduled on core 0 (mapped to execution contexts 0-127)



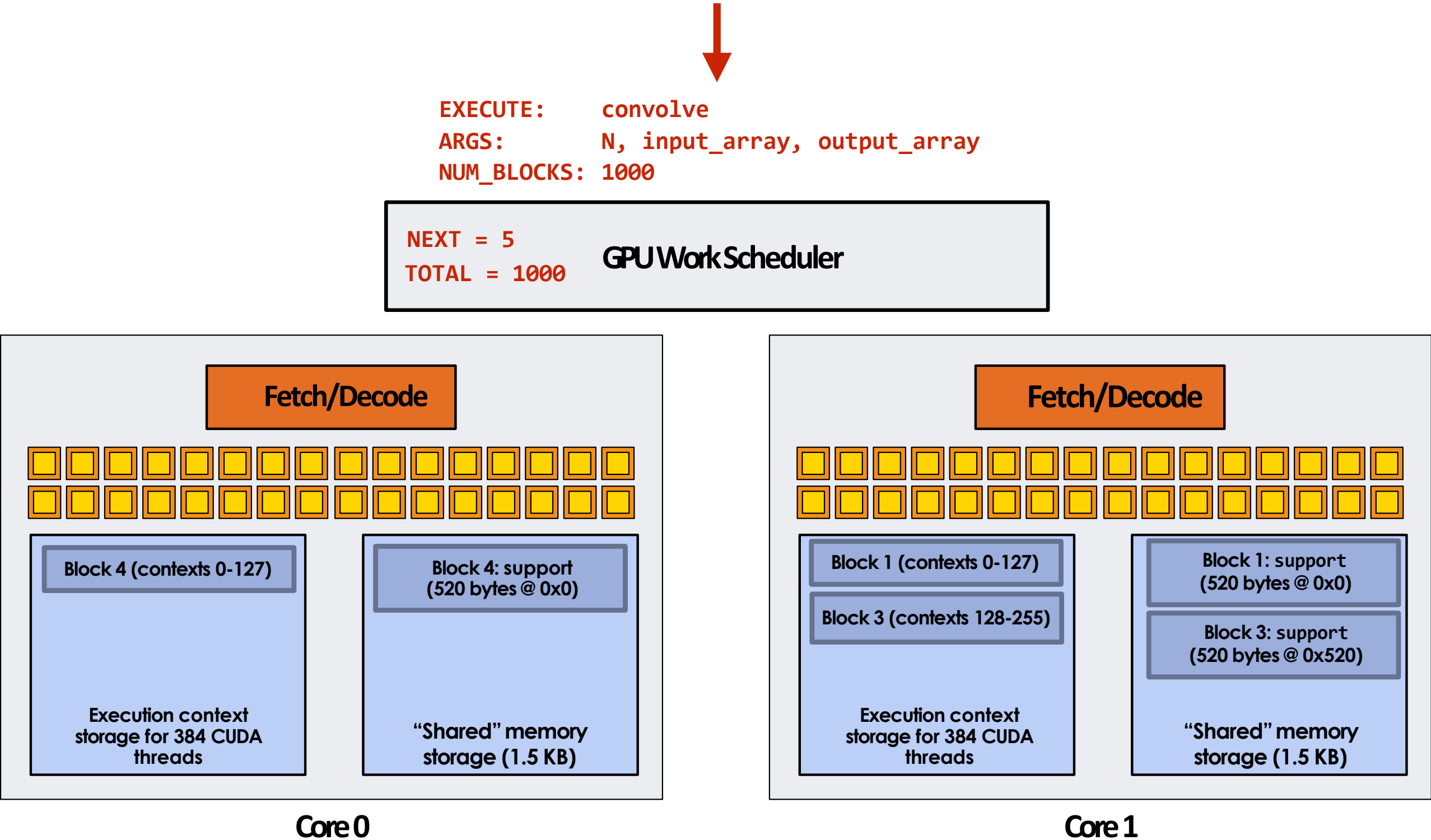
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

Step 6: thread block 2 completes on core 0



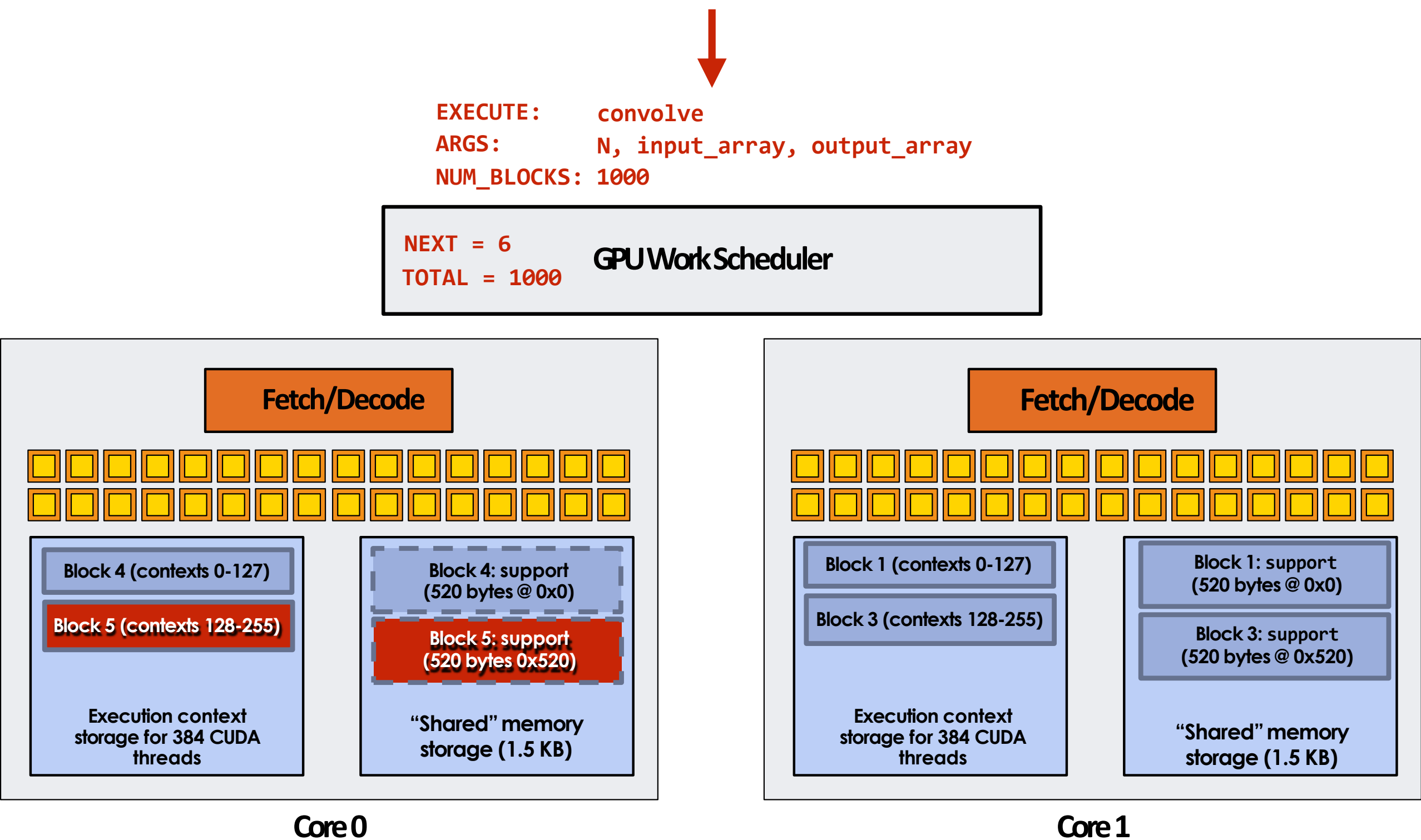
Running the CUDA kernel

Kernel's execution requirements:

Each thread block must execute 128 CUDA threads

Each thread block must allocate $130 \times \text{sizeof(float)} = 520$ bytes of shared memory

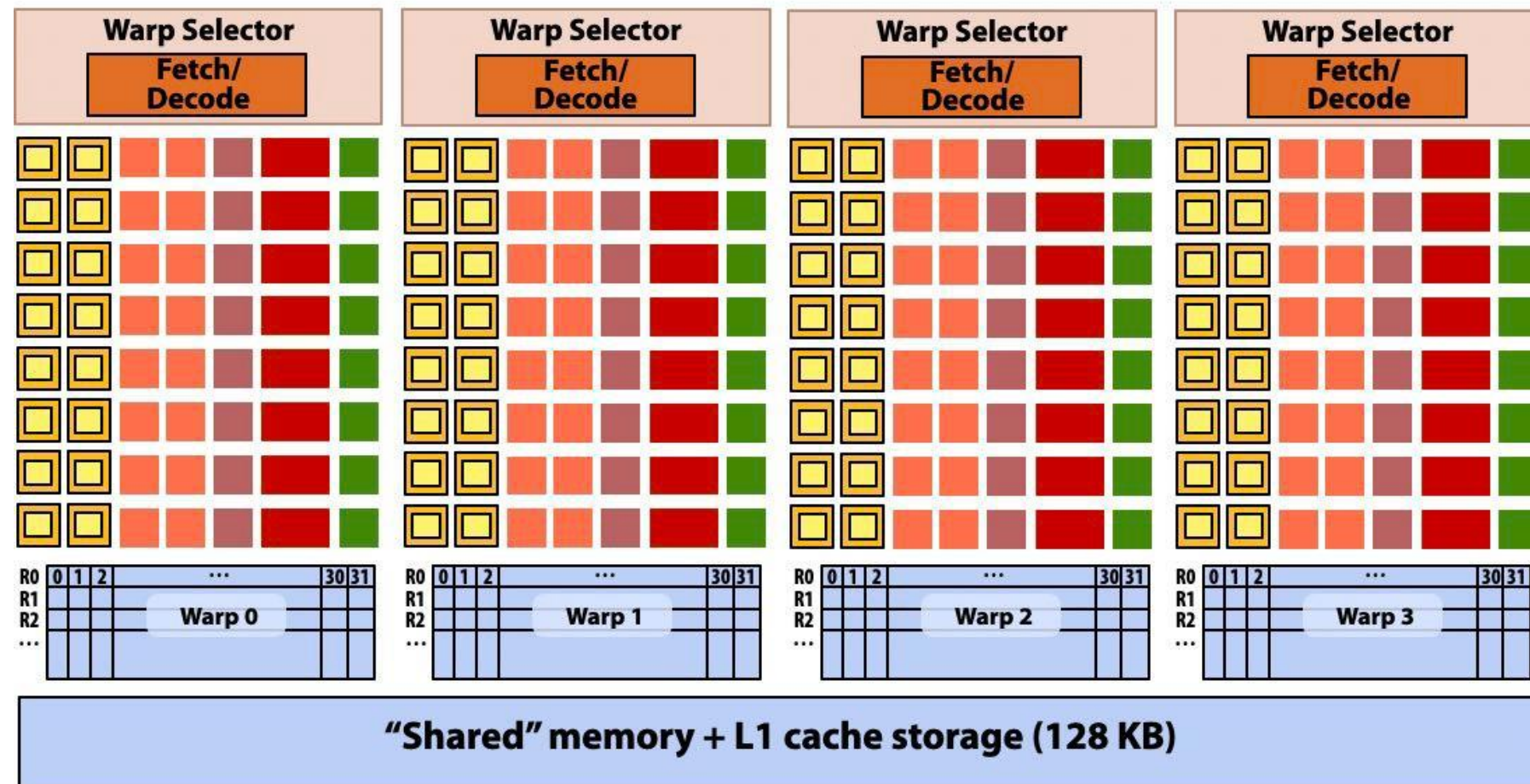
Step 7: thread block 5 is scheduled on core 0 (mapped to execution contexts 128-255)



More advanced scheduling questions:

(If you understand the following examples you really understand how CUDA programs run on a GPU, and also have a good handle on the work scheduling issues we've discussed in the course up to this point.)

Why must CUDA allocate execution contexts for all threads in a block?



Imagine a thread block with **256 CUDA threads**
(see code, top-right)

Assume a fictitious SM core (shown above) with only four warps worth of parallel execution in HW

Why not just run four warps (threads 0-127) to completion then run next four warps (threads 128-255) to completion in order to execute the entire thread block?

```
#define THREADS_PER_BLK 256

__global__ void convolve(int N, float* input,
                        float* output)
{
    __shared__ float support[THREADS_PER_BLK+2];
    int index = blockIdx.x * blockDim.x +
                threadIdx.x;

    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK+threadIdx.x]
            = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result;
}
```

CUDA kernels may create dependencies between threads in a block

Simplest example is `__syncthreads()`

Threads in a block cannot be executed by the system in any order when dependencies exist.

CUDA semantics: threads in a block ARE running concurrently. If a thread in a block is runnable it will eventually be run! (no deadlock)

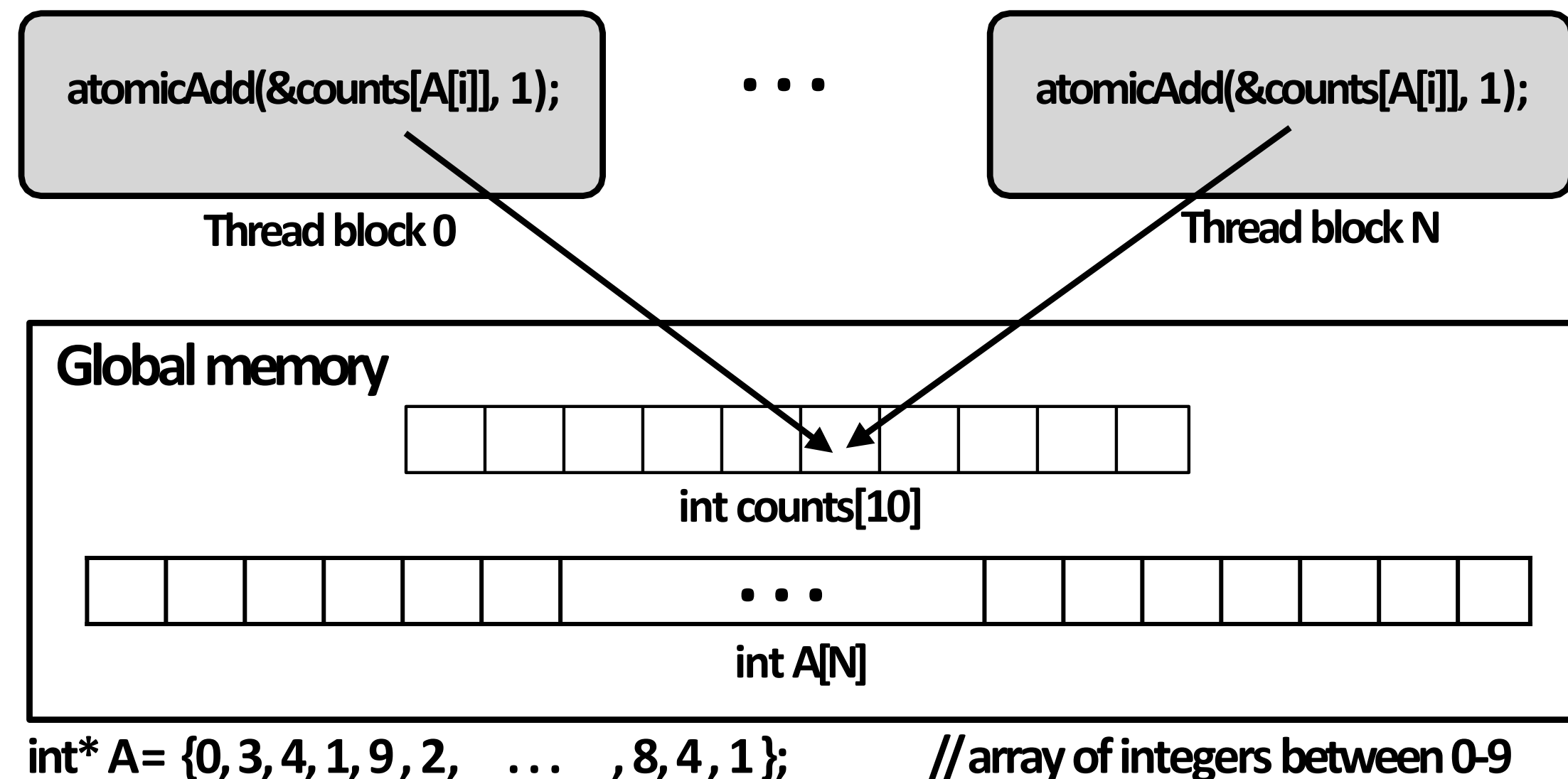
Implementation of CUDA abstractions

- **Thread blocks can be scheduled in any order by the system**
 - System assumes no dependencies between
 - blocks Logically concurrent
 - A lot like ISPC tasks, right?
- **CUDA threads in same block run concurrently (live at same time)**
 - When block begins executing, all threads exist and have register state allocated (these semantics impose a scheduling constraint on the system)
 - A CUDA thread block is itself an SPMD program (like an ISPC gang of program instances)
 - Threads in thread block are concurrent, cooperating “workers”
- **CUDA implementation:**
 - A NVIDIA GPU warp has performance characteristics akin to an ISPC gang of instances (but unlike an ISPC gang, the warp concept does not exist in the programming model*)
 - All warps in a thread block are scheduled onto the same SM, allowing for high-BW/low latency communication through shared memory variables
 - When all threads in block complete, block resources (shared memory allocations, warp execution contexts) become available for next block

* Exceptions to this statement include intra-warp builtin operations like swizzle and vote

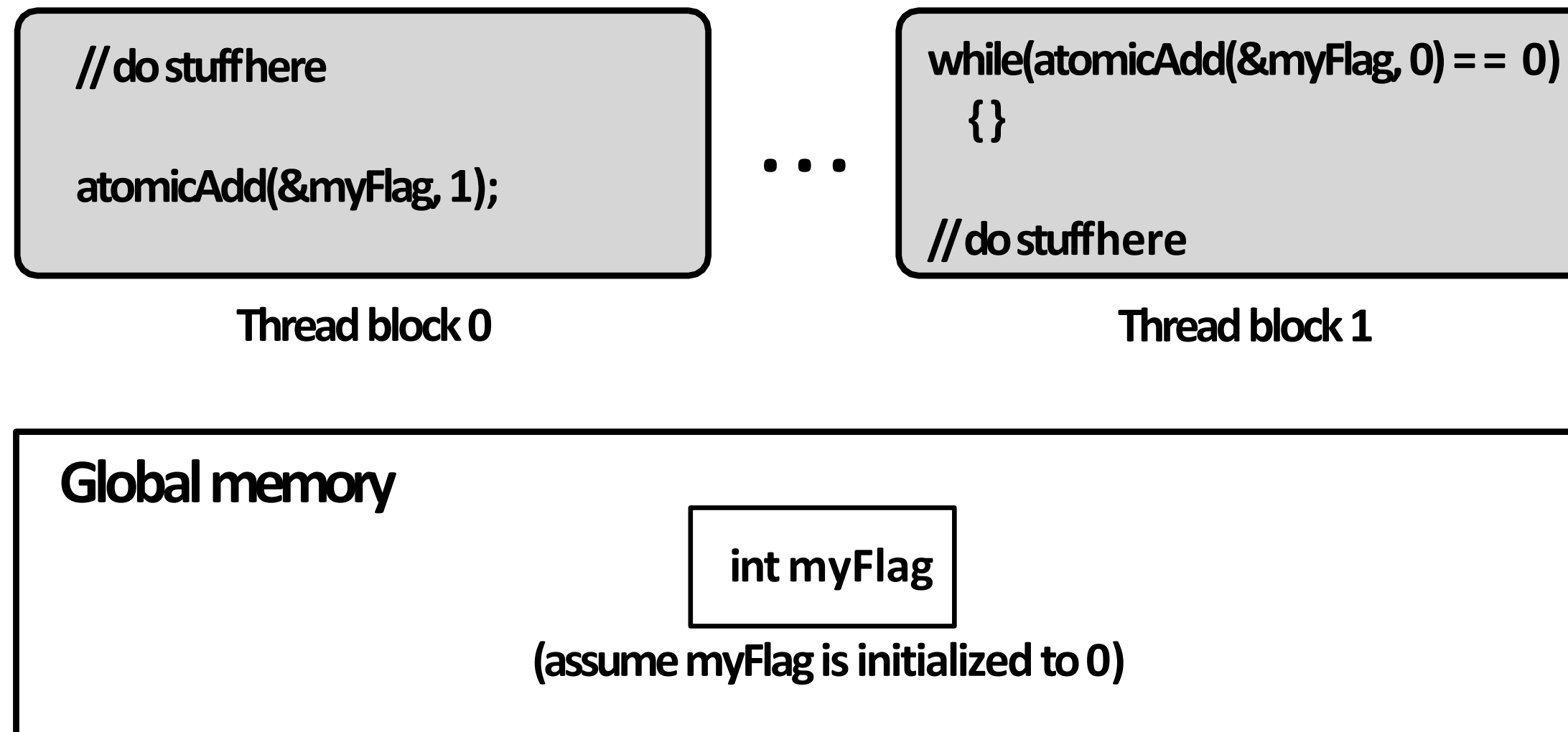
Consider a program that creates a histogram:

- This example: build a histogram of values in an array
 - All CUDA threads atomically update shared variables in global memory
- Notice I have never claimed CUDA thread blocks were guaranteed to be independent. I only stated CUDA reserves the right to schedule them in any order.
- This is valid code! This use of atomics does not impact implementation's ability to schedule blocks in any order (atomics used for mutual exclusion, and nothing more)



But is this reasonable CUDA code?

- Consider implementation of on a single SM GPU with resources for only one CUDA thread block per SM
 - What happens if the CUDA implementation runs block 0 first?
 - What happens if the CUDA implementation runs block 1 first?



“Persistent thread” CUDA programming style

```
#define THREADS_PER_BLK 128
#define BLOCKS_PER_CHIP 80 * (32*64/128) // specific to V100 GPU

__device__ int workCounter = 0;           // global mem variable

__global__ void convolve(int N, float* input, float* output) {
    __shared__ int startingIndex;
    __shared__ float support[THREADS_PER_BLK+2]; // shared across block
    while (1) {

        if (threadIdx.x == 0)
            startingIndex = atomicInc(workCounter, THREADS_PER_BLK);
        __syncthreads();
        if (startingIndex >= N)
            break;

        int index = startingIndex + threadIdx.x; // thread local
        support[threadIdx.x] = input[index];
        if (threadIdx.x < 2)
            support[THREADS_PER_BLK+threadIdx.x] = input[index+THREADS_PER_BLK];

        __syncthreads();

        float result = 0.0f; // thread-local variable
        for (int i=0; i<3; i++)
            result += support[threadIdx.x + i];
        output[index] = result;

        __syncthreads();
    }
}

// host code //////////////////////////////////////
int N = 1024 * 1024;
cudaMalloc(&devInput, N+2); // allocate array in device memory
cudaMalloc(&devOutput, N);   // allocate array in device memory
// properly initialize contents of devInput here ...

convolve<<<BLOCKS_PER_CHIP, THREADS_PER_BLK>>>(N, devInput, devOutput);
```

Idea: write CUDA code that requires knowledge of the number of cores and blocks per core that are supported by underlying GPU implementation.

Programmer launches exactly as many thread blocks as will fill the GPU

(Program makes assumptions about GPU implementation: that GPU will in fact run all blocks concurrently. Ugg!)

Now, work assignment to blocks is implemented entirely by the application

(circumvents GPU’s thread block scheduler)

Now the programmer’s mental model is that **all** CUDA threads are concurrently running on the GPU at once.

CUDA summary

- Execution semantics
 - Partitioning of problem into thread blocks is in the spirit of the data-parallel model (intended to be machine independent: system schedules blocks onto any number of cores)
 - Threads in a thread block actually do run concurrently (they have to, since they cooperate)
 - Inside a single thread block: SPMD shared address space programming
 - There are subtle, but notable differences between these models of execution. Make sure you understand it. (And ask yourself what semantics are being used whenever you encounter a parallel programming system)
- Memory semantics
 - Distributed address space: host/device memories
 - Thread local/block shared/global variables within device memory
 - Loads/stores move data between them (so it is correct to think about local/shared/global memory as being distinct address spaces)
- Key implementation details:
 - Threads in a thread block are scheduled onto same GPU core to allow fast communication through shared memory
 - Threads in a thread block are are grouped into warps for SIMD execution on GPU hardware

One last point...

- **In this lecture, we talked about writing CUDA programs for the programmable cores in a GPU**
 - Work (described by a CUDA kernel launch) was mapped onto the cores via a hardware work scheduler
- **Remember, there is still the graphics pipeline interface for driving GPU execution**
 - And much of the interesting non-programmable functionality of the GPU is present to accelerate execution of graphics pipeline operations
 - It's more or less "turned off" when running CUDA programs
- **How the GPU implements the graphics pipeline efficiently is a topic for a graphics class...**

And...

- **We didn't even talk about the hundreds of teraflops available in the tensor cores in the SM (for deep learning)**
- **A topic for a later class...**

Advanced thread scheduling

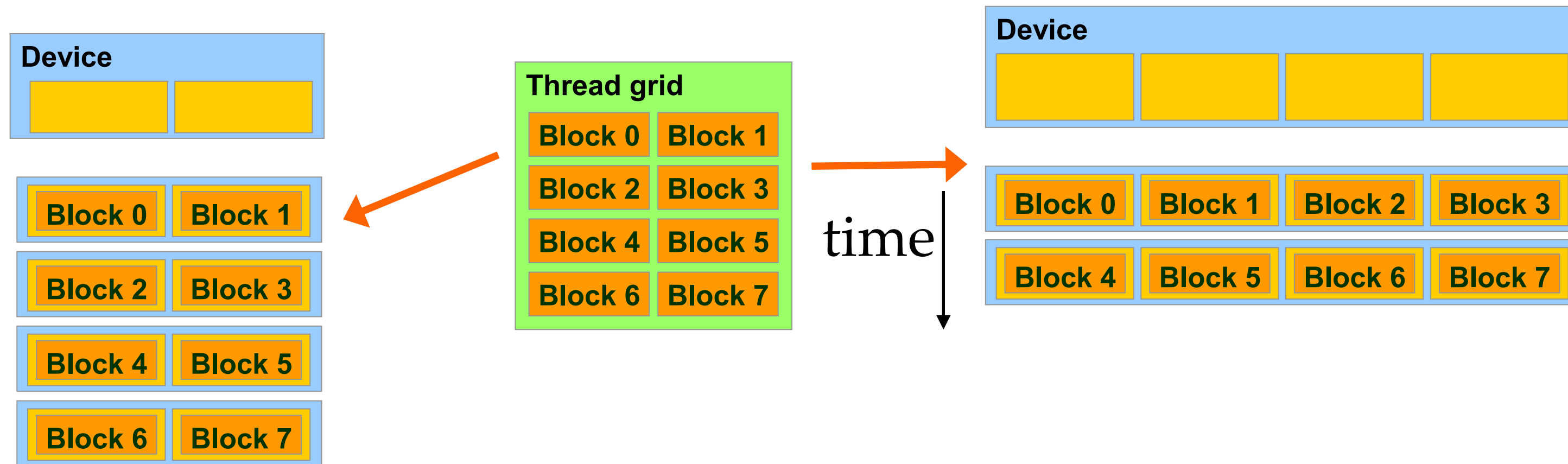
Material from The GPU Teaching Kit .

The GPU Teaching Kit is licensed by NVIDIA and the University of Illinois under the [Creative Commons Attribution-NonCommercial 4.0 International License.](#)

Objective

- To learn how a CUDA kernel utilizes hardware execution resources
 - Assigning thread blocks to execution resources
 - Capacity constraints of execution resources
 - Zero-overhead thread scheduling

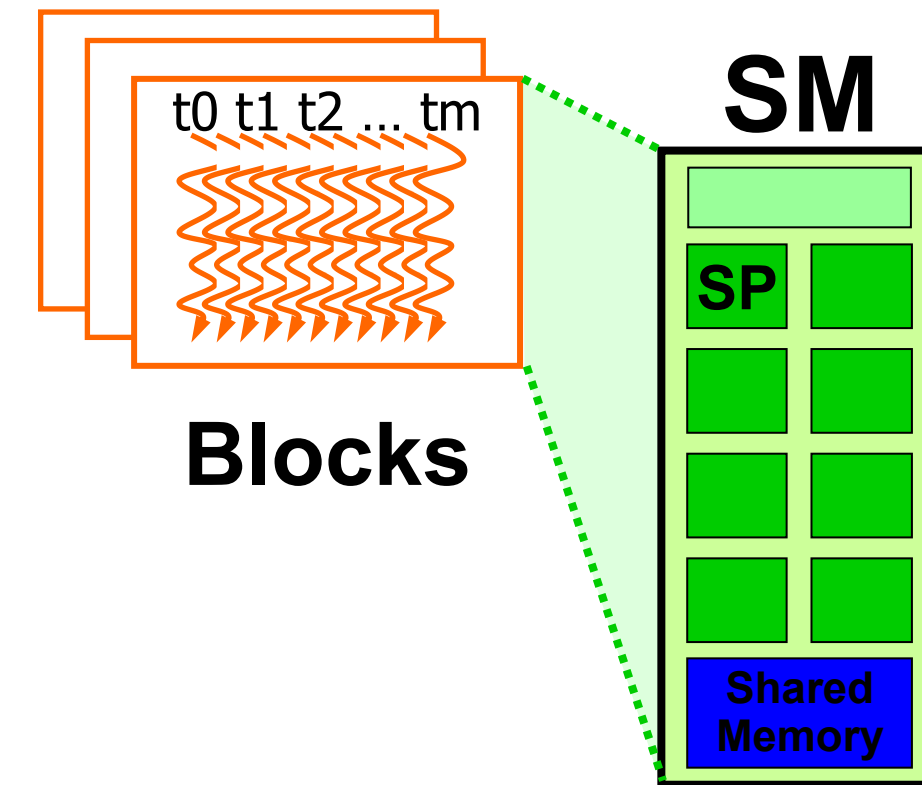
Transparent Scalability



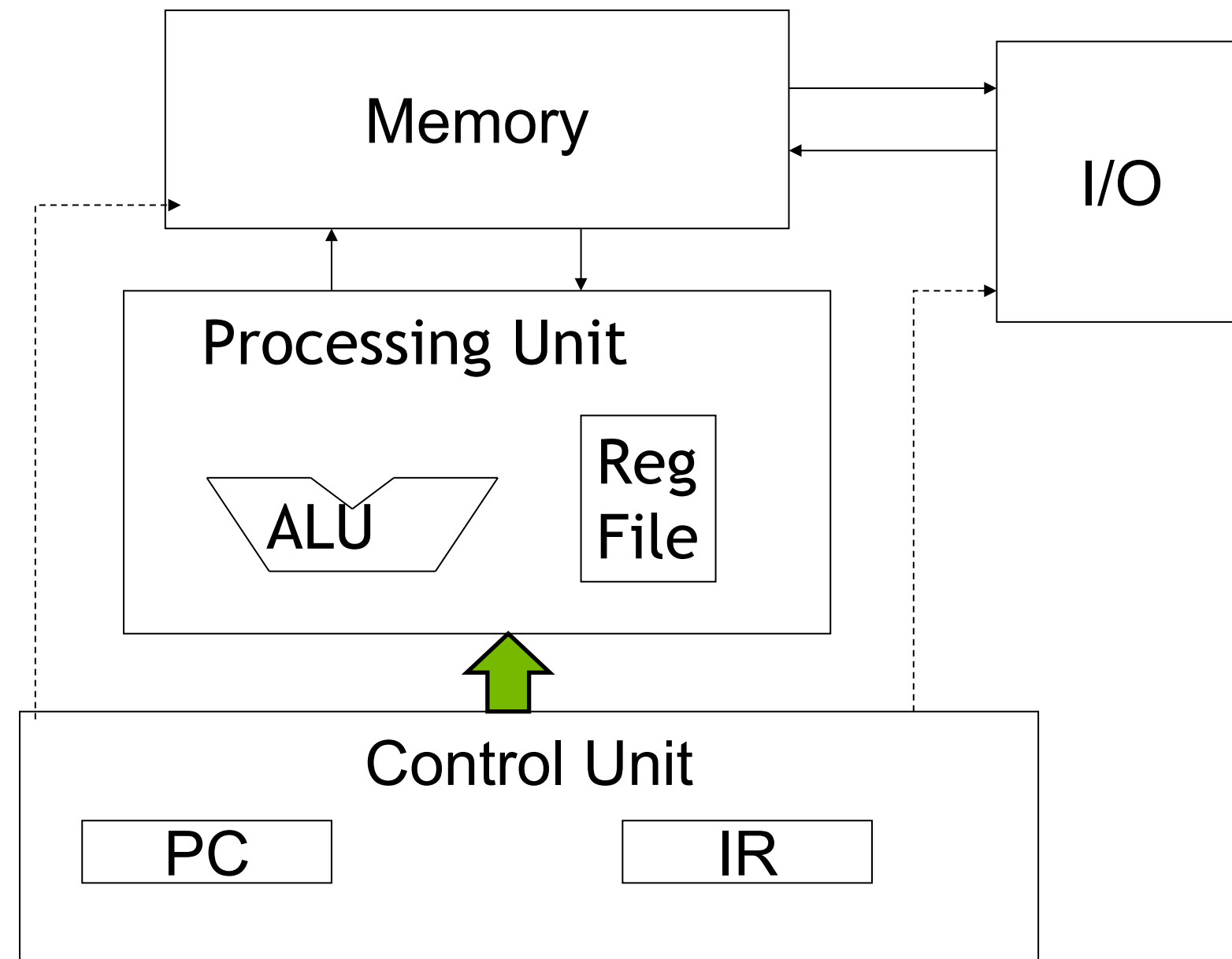
- Each block can execute in any order relative to others.
- Hardware is free to assign blocks to any processor at any time
 - A kernel scales to any number of parallel processors

Example: Executing Thread Blocks

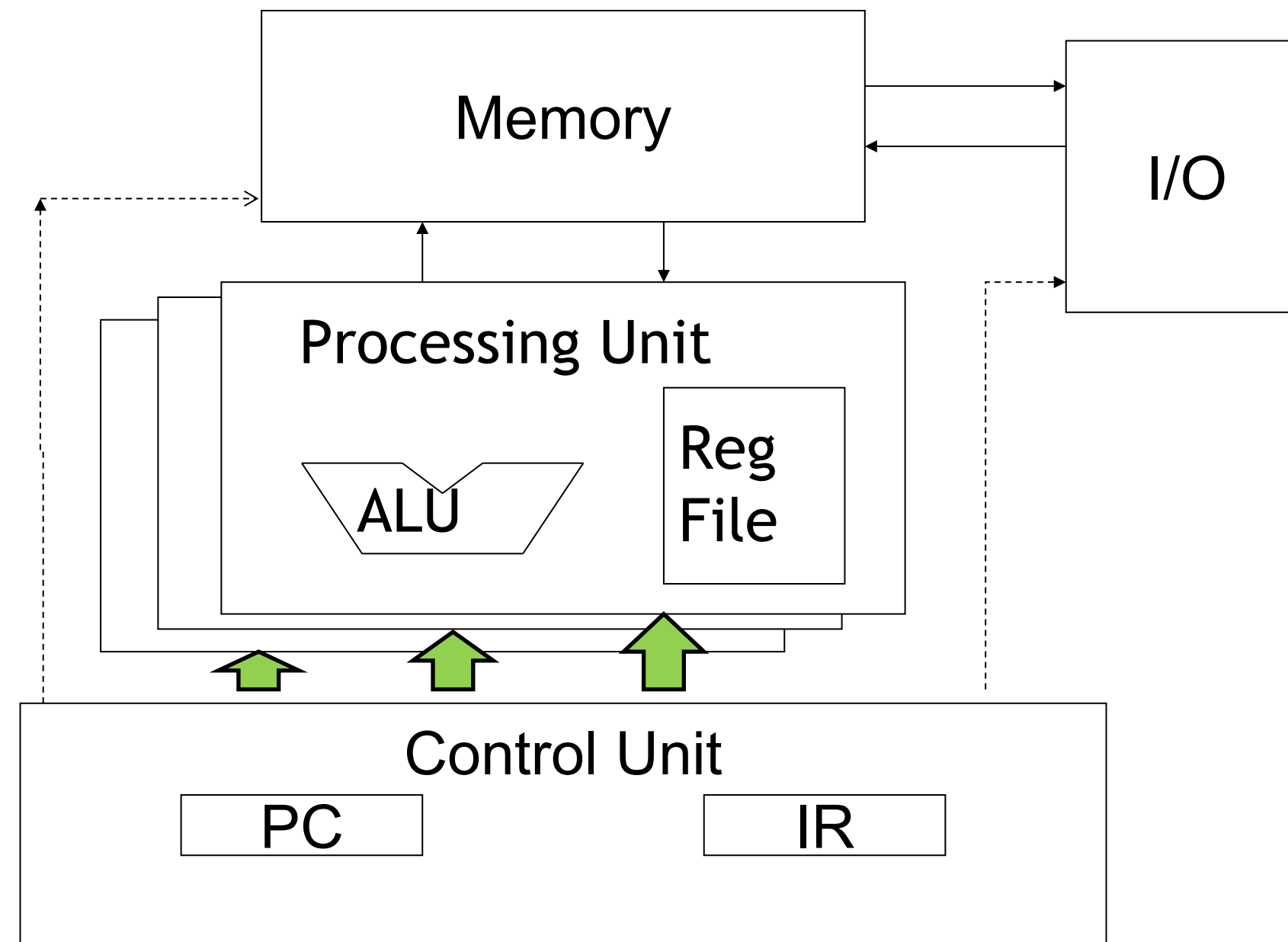
- Threads are assigned to **Streaming Multiprocessors (SM)** in block granularity
 - Up to 32 blocks to each SM as resource allows
 - Volta SM can take up to **2048** threads
 - Could be 256 (threads/block) * 8 blocks
 - Or 512 (threads/block) * 4 blocks, etc.
- SM maintains thread/block idx #s
- SM manages/schedules thread execution



The Von-Neumann Model



The Von-Neumann Model with SIMD units



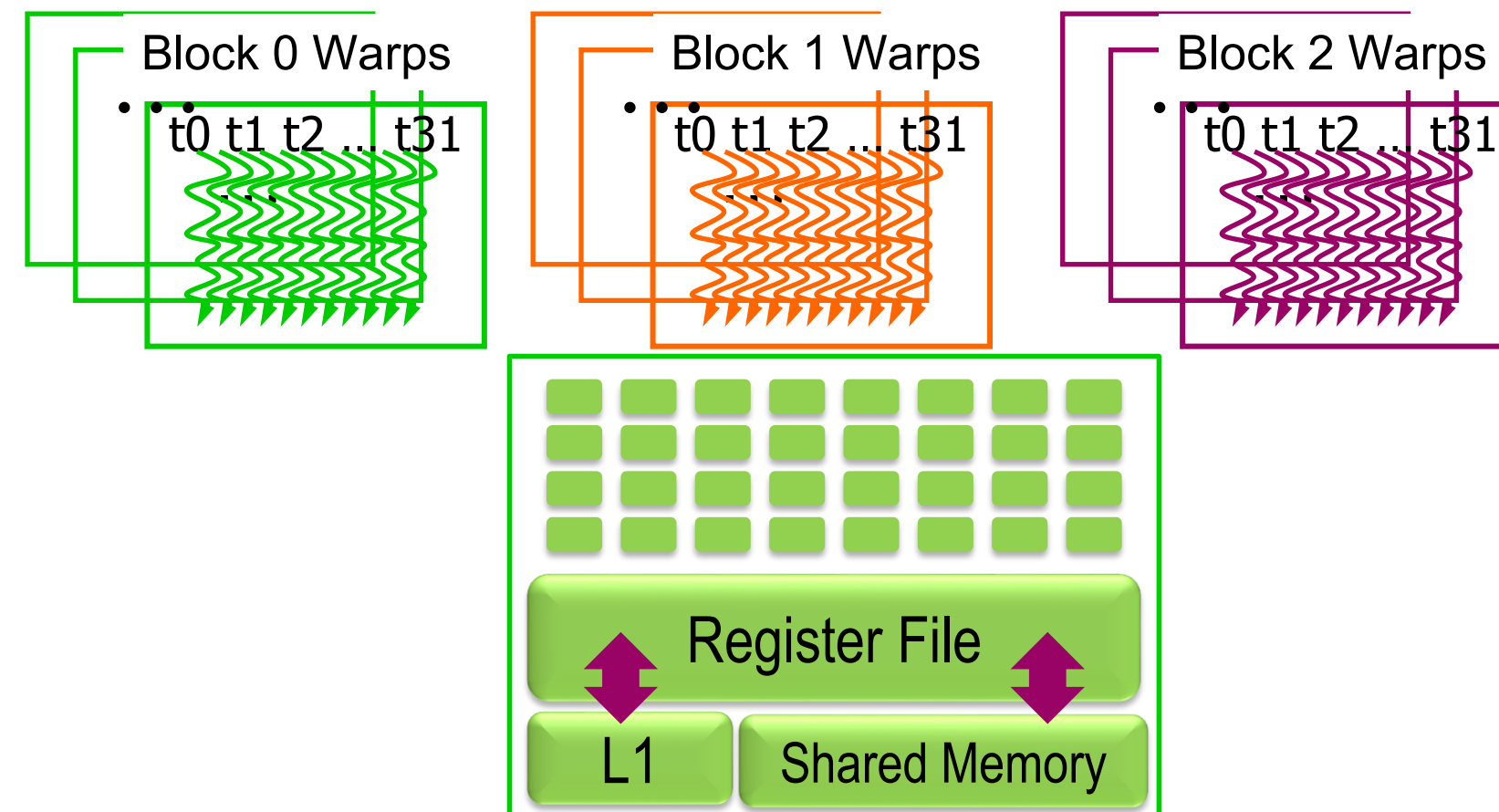
Single Instruction Multiple Data
(SIMD)

Warps as Scheduling Units

- Each Block is executed as 32-thread Warps
 - An implementation decision, not part of the CUDA programming model
 - Warps are scheduling units in SM
 - Threads in a warp execute in SIMD
 - Future GPUs may have different number of threads in each warp

Warp Example

- If 3 blocks are assigned to an SM and each block has 256 threads, how many Warps are there in an SM?
 - Each Block is divided into $256/32 = 8$ Warps
 - There are $8 * 3 = 24$ Warps



Example: Thread Scheduling (Cont.)

- SM implements zero-overhead warp scheduling
 - Warps whose next instruction has its operands ready for consumption are eligible for execution
 - Eligible Warps are selected for execution based on a prioritized scheduling policy
 - All threads in a warp execute the same instruction when selected

Block Granularity Considerations

- For Matrix Multiplication using multiple blocks, should each block have 4X4, 8X8 or 30X30 threads for Volta?
- For 4X4, we have 16 threads per Block. Each SM can take up to 2048 threads, which translates to 128 Blocks. However, each SM can only take up to 32 Blocks, so only 512 threads will go into each SM!
- For 8X8, we have 64 threads per Block. Since each SM can take up to 2048 threads, it can take up to 32 Blocks and achieve full capacity unless other resource considerations overrule.
- For 30X30, we would have 900 threads per Block. Only two blocks could fit into an SM for Volta, so only $1800/2048$ of the SM thread capacity would be utilized.

Comparison of NVIDIA Data Center GPUs

GPU Features	NVIDIA Pascal P100	NVIDIA Volta V100	NVIDIA A100
GPU Codename	GP100	GV100	GA100
GPU Architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
GPU Board Form Factor	SXM	SXM2	SXM4
SMs	56	80	108
TPCs	28	40	54
FP32 Cores/ SM	64	64	64
FP32 Cores/ GPU	3584	5120	6912
FP64 Cores/ SM (excl. Tensor)	32	32	32
FP64 Cores/ GPU (excl. Tensor)	1792	2560	3456
INT32 Cores / SM	NA	64	64
INT32 Cores / GPU	NA	5120	6912
Tensor Cores / SM	NA	8	4 ²
Tensor Cores / GPU	NA	640	432
GPU Boost Clock	1480 MHz	1530 MHz	1410 MHz
Memory Interface	4096-bit HBM2	4096-bit HBM2	5120-bit HBM2
Memory Size	16 GB	32 GB / 16 GB	40 GB
Memory Data Rate	703 MHz DDR	877.5 MHz DDR	1215 MHz DDR
Memory Bandwidth	720 GB/sec	900 GB/sec	1555 GB/sec
L2 Cache Size	4096 KB	6144 KB	40960 KB
Shared Memory Size / SM	64 KB	Configurable up to 96 KB	Configurable up to 164 KB
Register File Size / SM	256 KB	256 KB	256 KB
Register File Size / GPU	14336 KB	20480 KB	27648 KB
TDP	300 Watts	300 Watts	400 Watts
Transistors	15.3 billion	21.1 billion	54.2 billion
GPU Die Size	610 mm ²	815 mm ²	826 mm ²
TSMC Manufacturing Process	16 nm FinFET+	12 nm FFN	7 nm N7

Comparison of Compute Capability: GP100 vs GV100 vs GA100

GPU features	NVIDIA Pascal P100	NVIDIA Volta V100	NVIDIA A100
GPU codename	GP100	GV100	GA100
GPU architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
Compute capability	6.0	7.0	8.0
Threads / warp	32	32	32
Max warps / SM	64	64	64
Max threads / SM	2048	2048	2048
Max thread blocks / SM	32	32	32
Max 32-bit registers / SM	65536	65536	65536
Max registers / block	65536	65536	65536
Max registers / thread	255	255	255
Max thread block size	1024	1024	1024
FP32 cores / SM	64	64	64
Ratio of SM registers to FP32 cores	1024	1024	1024
Shared Memory Size / SM	64 KB	Configurable up to 96 KB	Configurable up to 164 KB

Memory and Data Locality in depth

Material from The GPU Teaching Kit .

The GPU Teaching Kit is licensed by NVIDIA and the University of Illinois under the [Creative Commons Attribution-NonCommercial 4.0 International License.](#)

Objective

- To learn to effectively use the CUDA memory types in a parallel program
 - Importance of memory access efficiency
 - Registers, shared memory, global memory
 - Scope and lifetime

Review: Image Blur Kernel.

```
// Get the average of the surrounding 2xBLUR_SIZE x 2xBLUR_SIZE box
for(int blurRow = -BLUR_SIZE; blurRow < BLUR_SIZE+1; ++blurRow) {
    for(int blurCol = -BLUR_SIZE; blurCol < BLUR_SIZE+1; ++blurCol) {

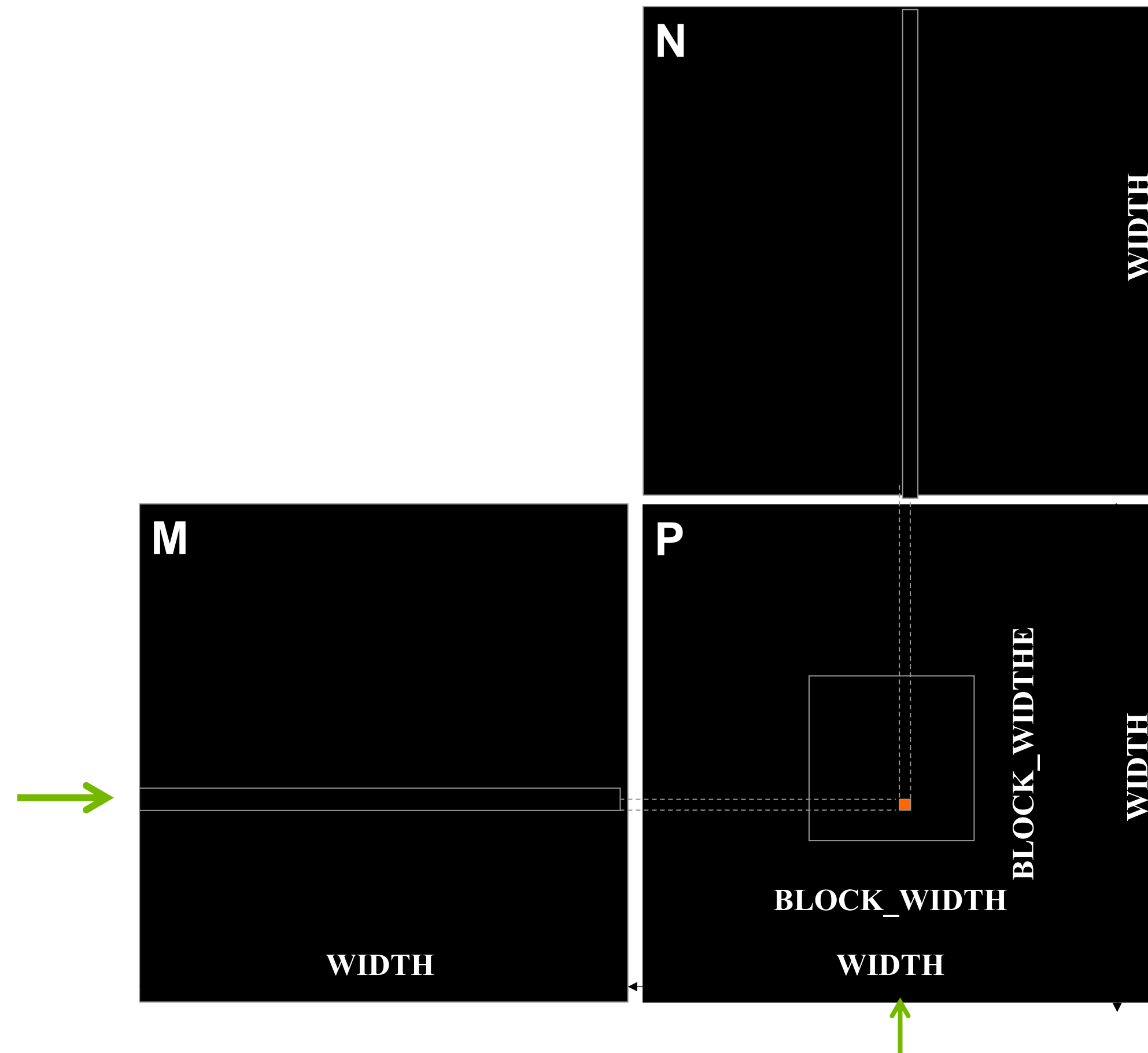
        int curRow = Row + blurRow;
        int curCol = Col + blurCol;
        // Verify we have a valid image pixel
        if(curRow > -1 && curRow < h && curCol > -1 && curCol < w) {
            ➡ pixVal += in[curRow * w + curCol];
            pixels++; // Keep track of number of pixels in the accumulated total
        }
    }
}

// Write our new pixel value out
out[Row * w + Col] = (unsigned char)(pixVal / pixels);
```

How about performance on a GPU

- All threads access global memory for their input matrix elements
 - One memory accesses (4 bytes) per floating-point addition
 - 4B/s of memory bandwidth/FLOPS
- Assume a GPU with
 - Peak floating-point rate 1,600 GFLOPS with 600 GB/s DRAM bandwidth
 - $4 \times 1,600 = 6,400$ GB/s required to achieve peak FLOPS rating
 - The 600 GB/s memory bandwidth limits the execution at 150 GFLOPS
- This limits the execution rate to 9.3% ($150/1600$) of the peak floating-point execution rate of the device!
- Need to drastically cut down memory accesses to get close to the 1,600 GFLOPS

Example – Matrix Multiplication



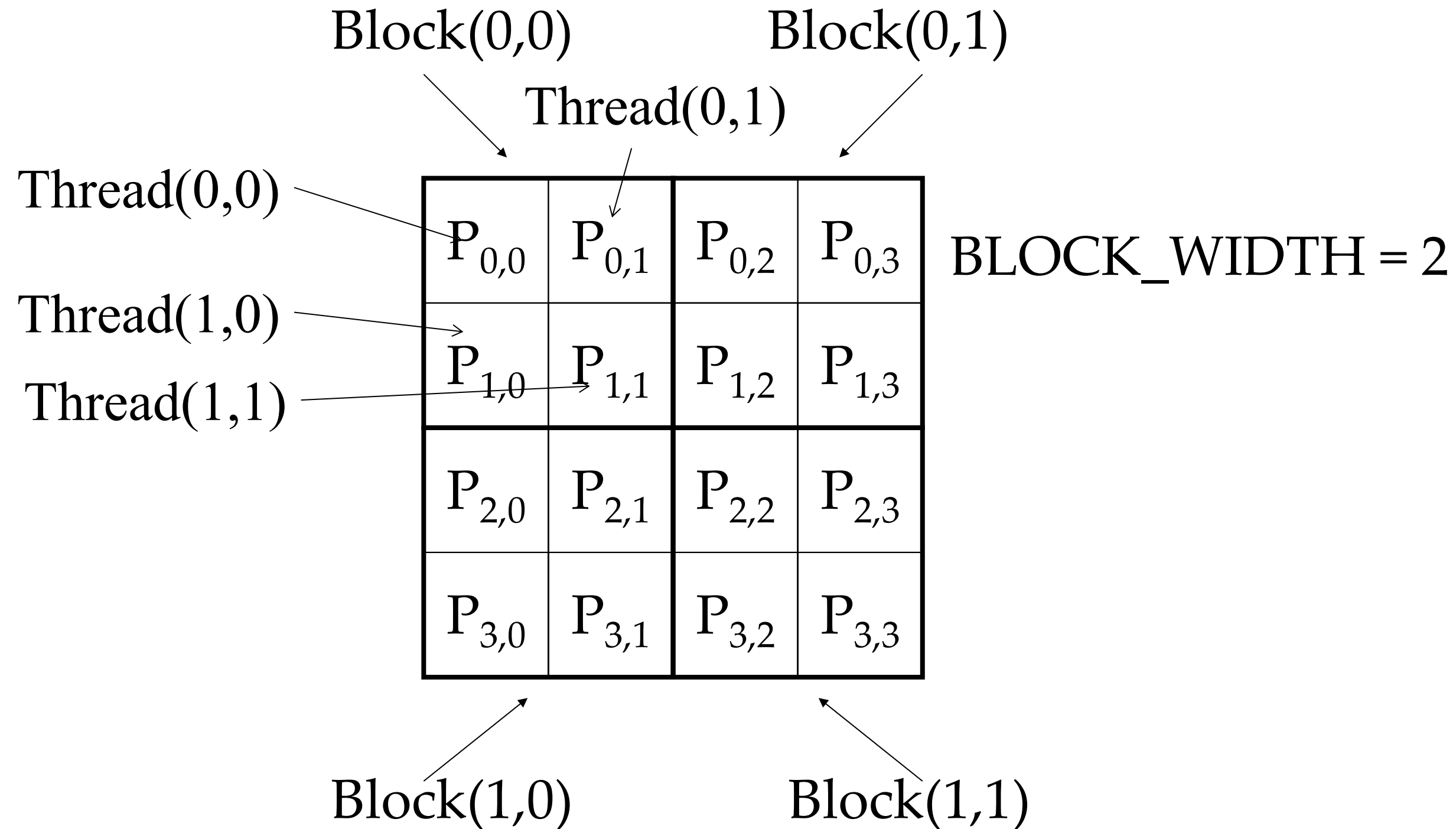
A Basic Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {  
  
    // Calculate the row index of the P element and M  
    int Row = blockIdx.y*blockDim.y+threadIdx.y;  
  
    // Calculate the column index of P and N  
    int Col = blockIdx.x*blockDim.x+threadIdx.x;  
  
    if ((Row < Width) && (Col < Width)) {  
        float Pvalue = 0;  
        // each thread computes one element of the block sub-matrix  
        for (int k = 0; k < Width; ++k) {  
            Pvalue += M[Row*Width+k]*N[k*Width+Col];  
        }  
        P[Row*Width+Col] = Pvalue;  
    }  
}
```

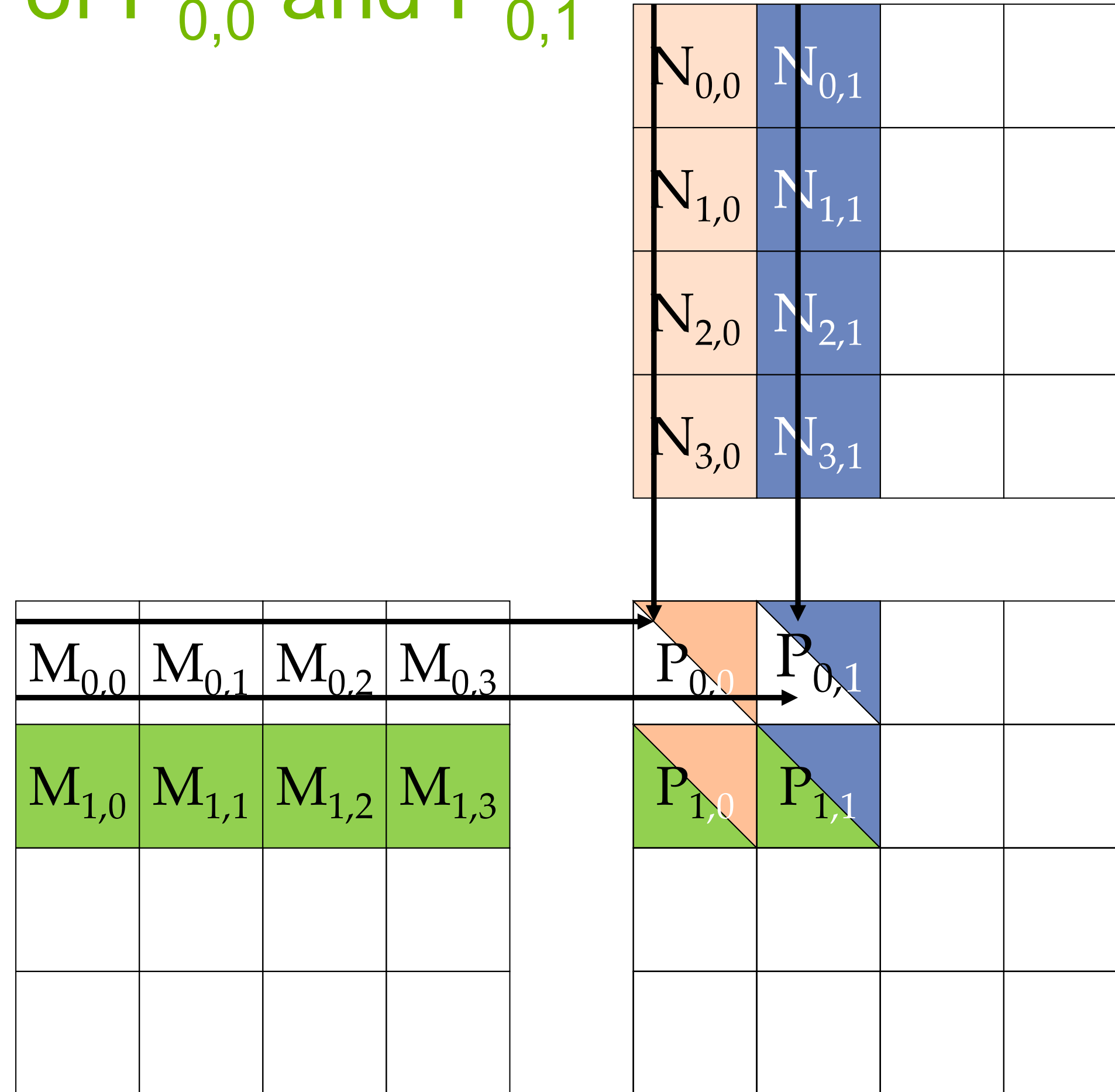
Example – Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {  
  
    // Calculate the row index of the P element and M  
    int Row = blockIdx.y*blockDim.y+threadIdx.y;  
  
    // Calculate the column index of P and N  
    int Col = blockIdx.x*blockDim.x+threadIdx.x;  
  
    if ((Row < Width) && (Col < Width)) {  
        float Pvalue = 0;  
        // each thread computes one element of the block sub-matrix  
        for (int k = 0; k < Width; ++k) {  
            Pvalue += M[Row*Width+k]*N[k*Width+Col];  
        }  
        P[Row*Width+Col] = Pvalue;  
    }  
}
```

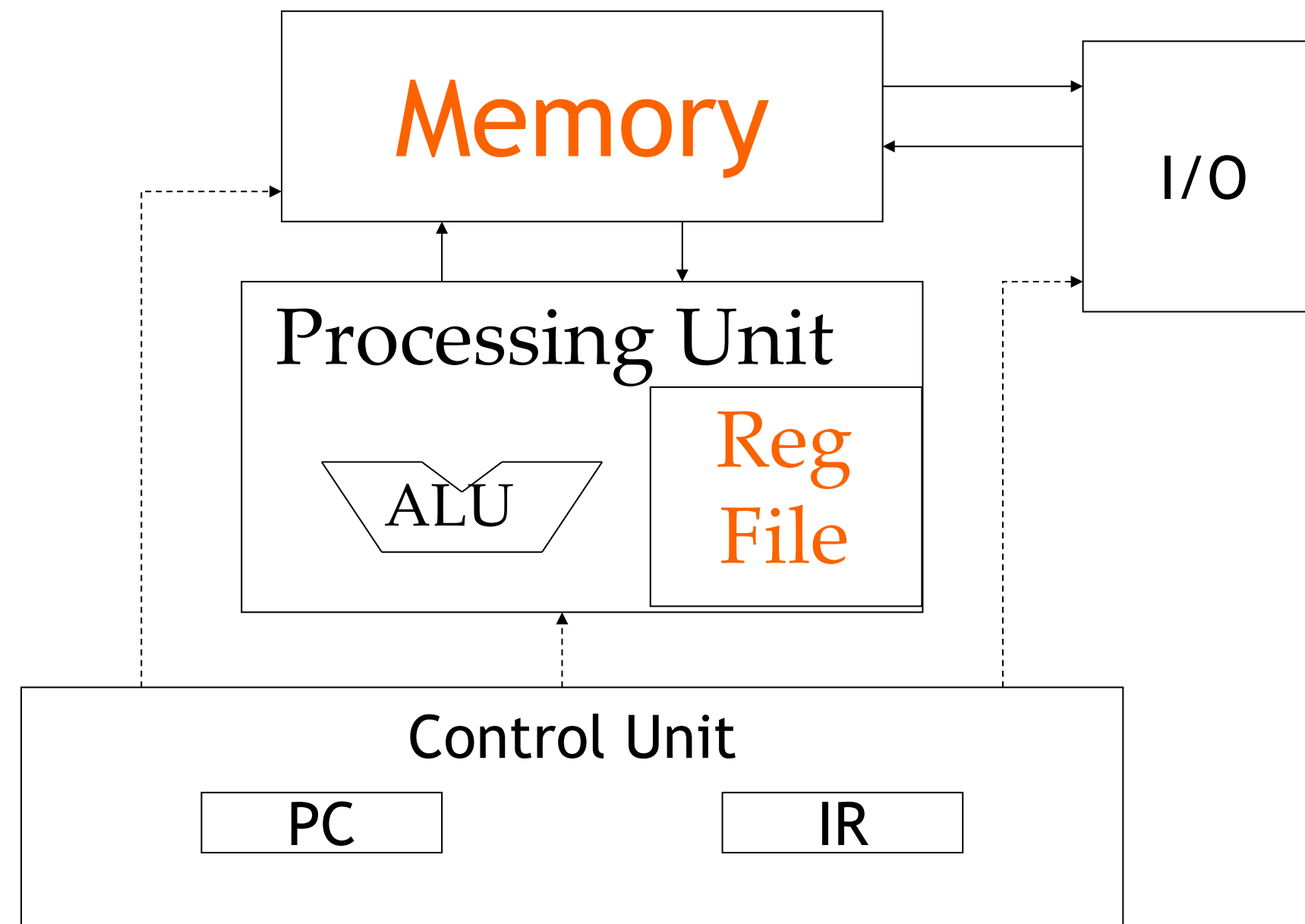
A Toy Example: Thread to P Data Mapping



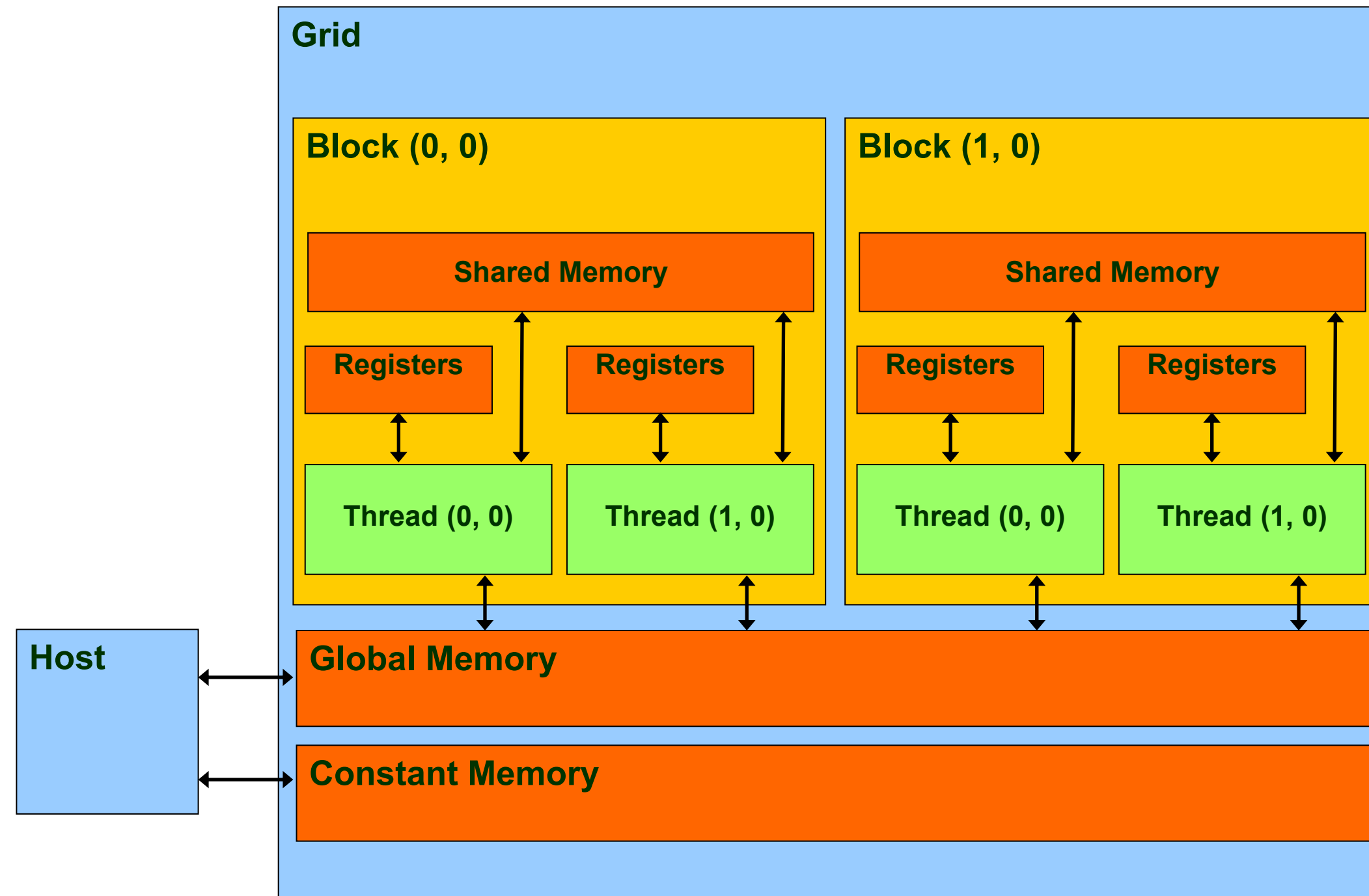
Calculation of $P_{0,0}$ and $P_{0,1}$



Memory and Registers in the Von-Neumann Model



Programmer View of CUDA Memories



Declaring CUDA Variables

Variable declaration	Memory	Scope	Lifetime
int LocalVar;	register	thread	thread
__device__ __shared__ int SharedVar;	shared	block	block
__device__ int GlobalVar;	global	grid	application
__device__ __constant__ int ConstantVar;	constant	grid	application

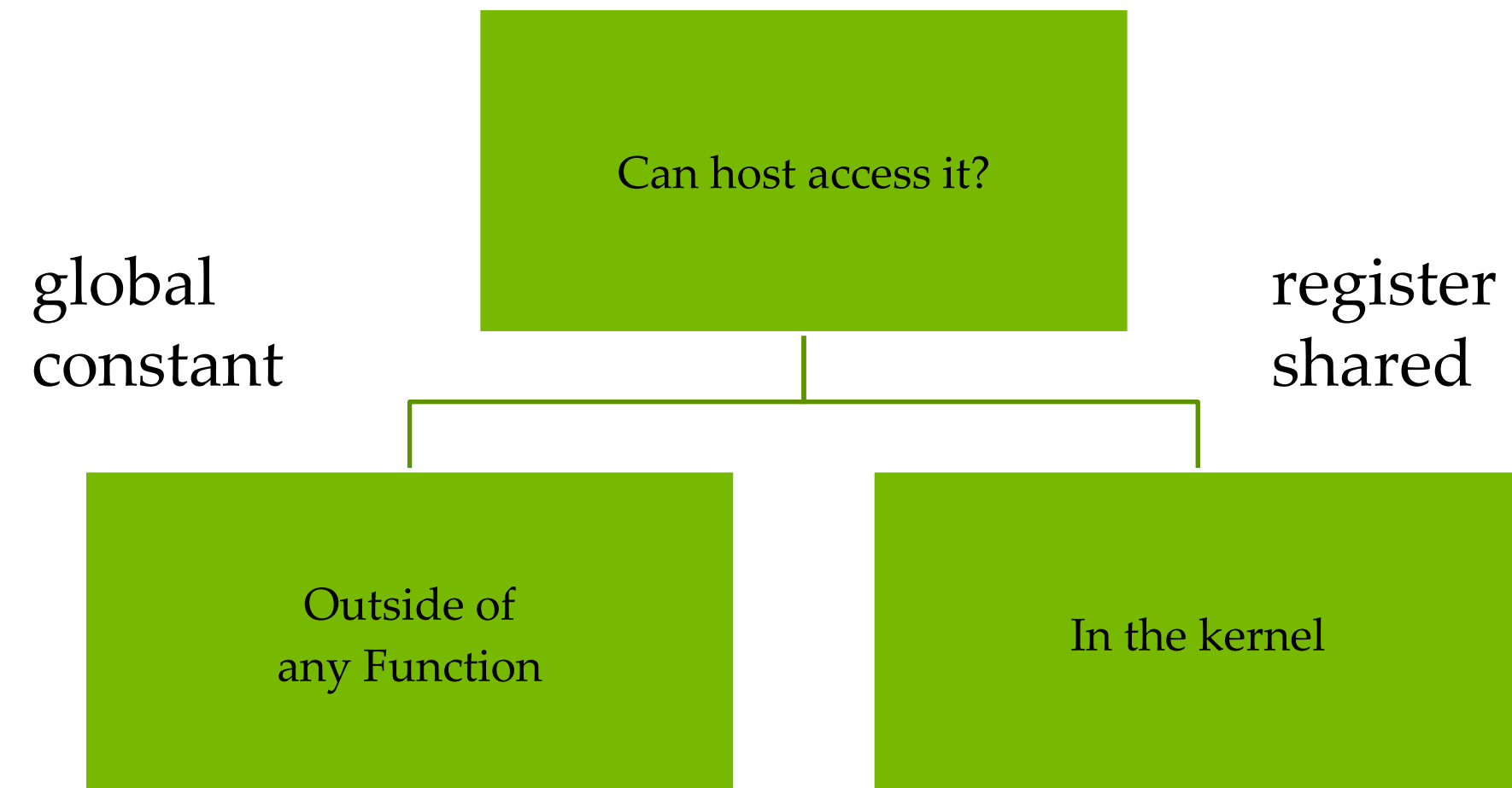
- **__device__** is optional when used with **__shared__**, or **__constant__**
- Automatic variables reside in a register
 - Except per-thread arrays that reside in global memory

Example:

Shared Memory Variable Declaration

```
void blurKernel(unsigned char * in, unsigned char * out, int w, int h)
{
    __shared__ float ds_in[TILE_WIDTH][TILE_WIDTH];
    ...
}
```

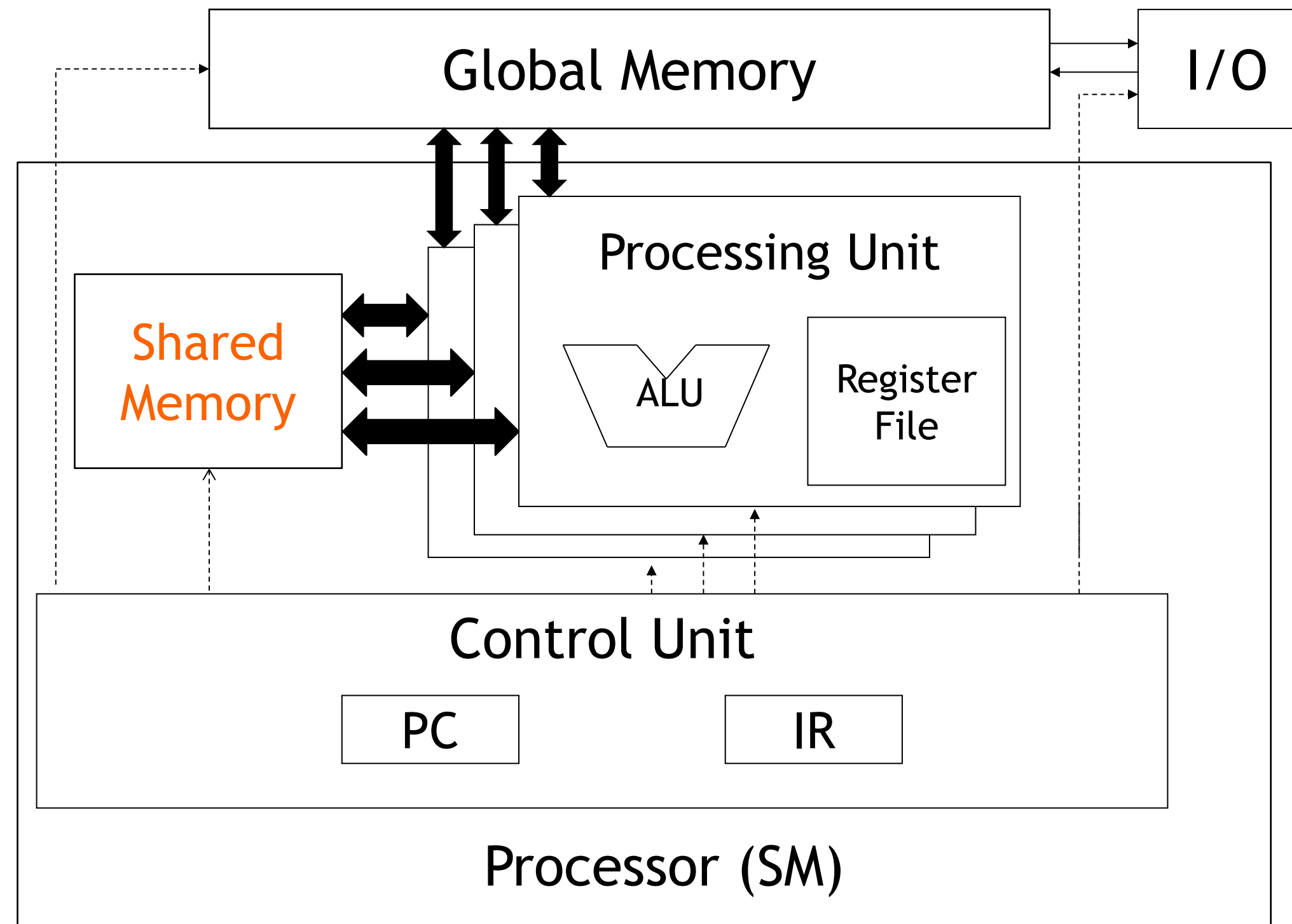
Where to Declare Variables?



Shared Memory in CUDA

- A special type of memory whose contents are explicitly defined and used in the kernel source code
 - One in each SM
 - Accessed at much higher speed (in both latency and throughput) than global memory
 - Scope of access and sharing - thread blocks
 - Lifetime – thread block, contents will disappear after the corresponding thread finishes terminates execution
 - Accessed by memory load/store instructions
 - A form of scratchpad memory in computer architecture

Hardware View of CUDA Memories

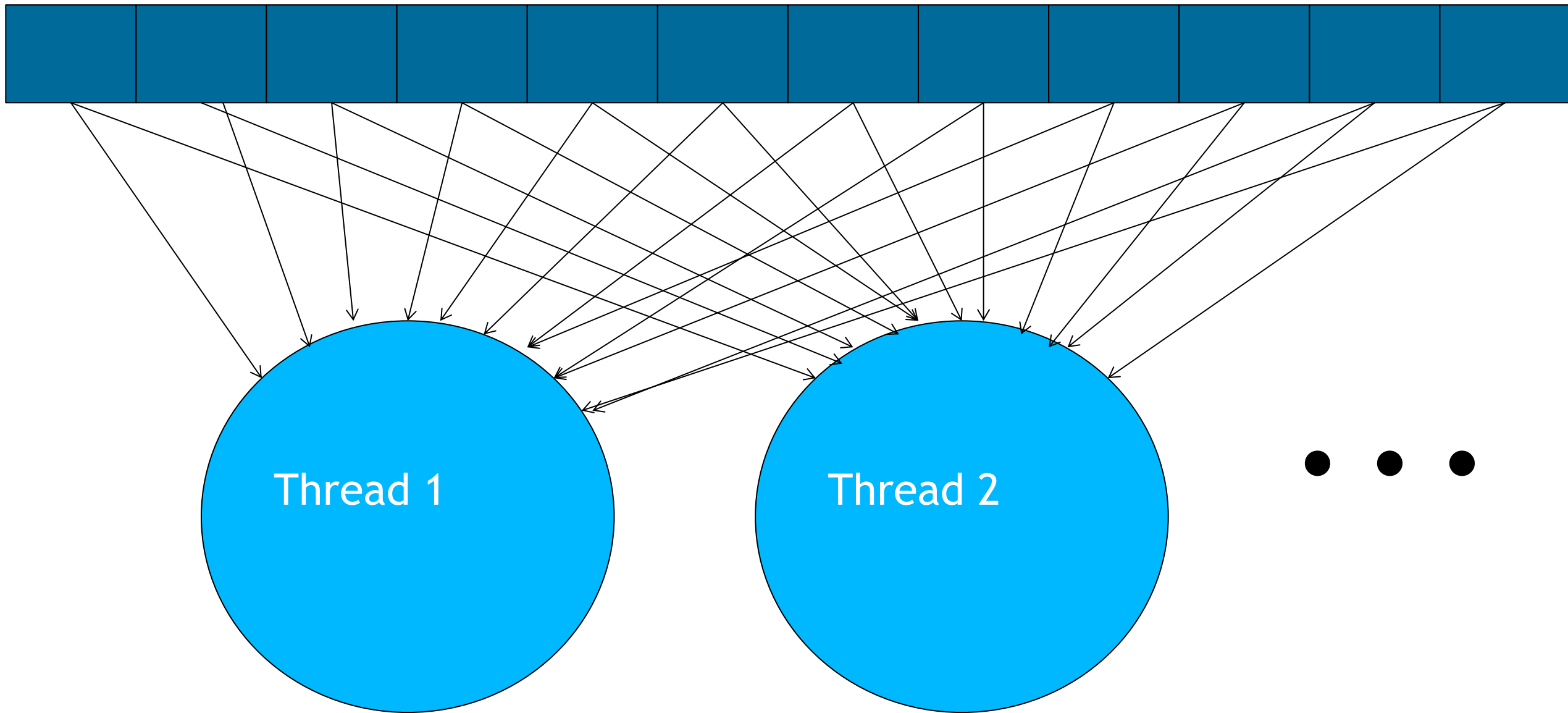


Objective

- To understand the motivation and ideas for tiled parallel algorithms
 - Reducing the limiting effect of memory bandwidth on parallel kernel performance
 - Tiled algorithms and barrier synchronization

Global Memory Access Pattern of the Basic Matrix Multiplication Kernel

Global Memory

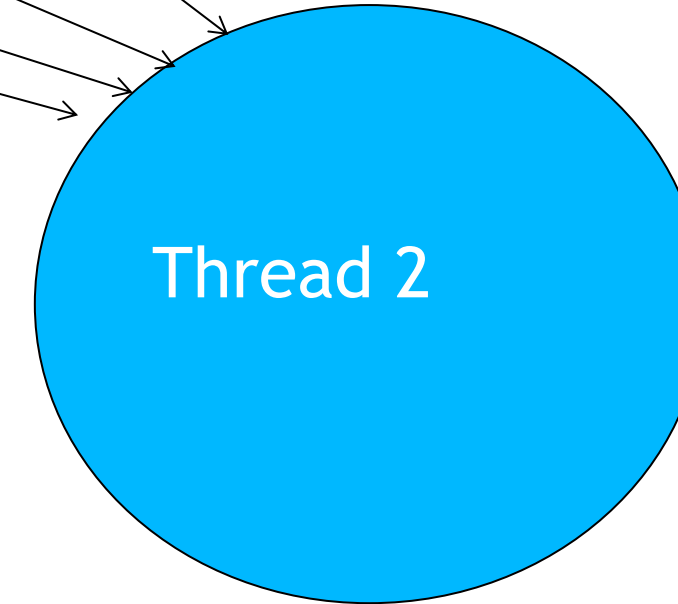
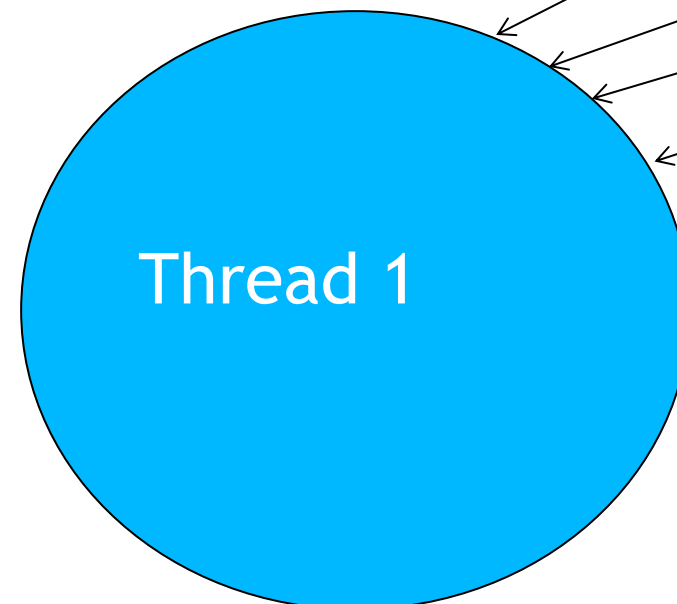
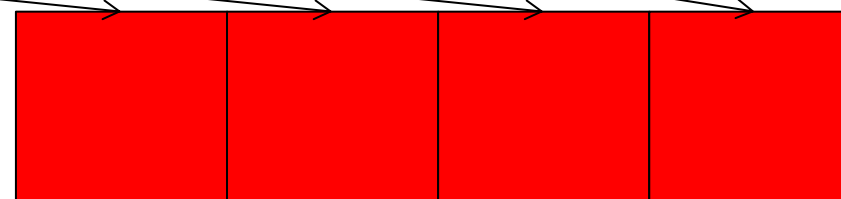


Tiling/Blocking - Basic Idea

Global Memory



On-chip Memory



...

Divide the global memory content into tiles

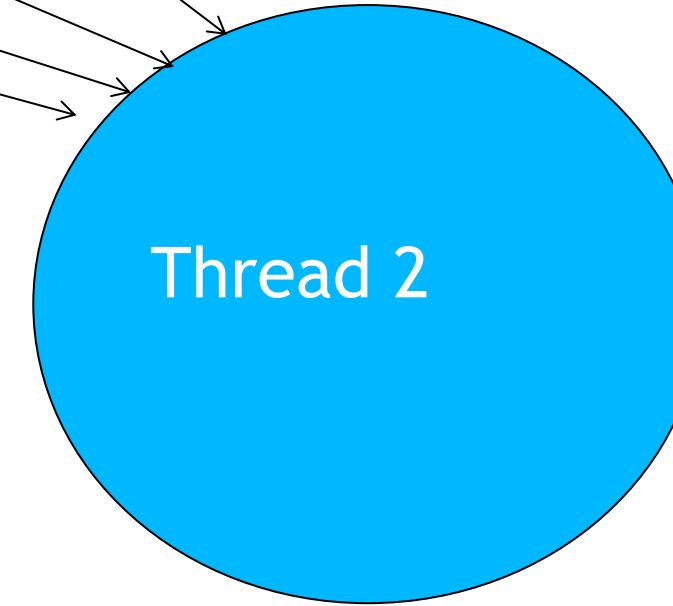
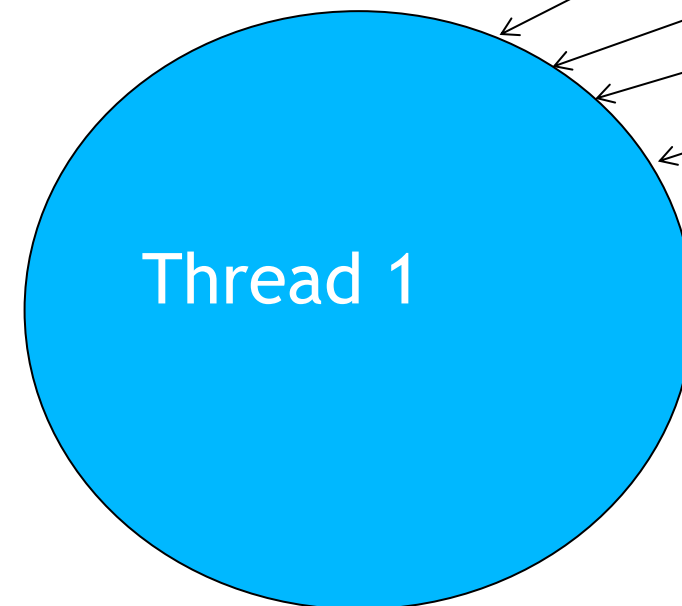
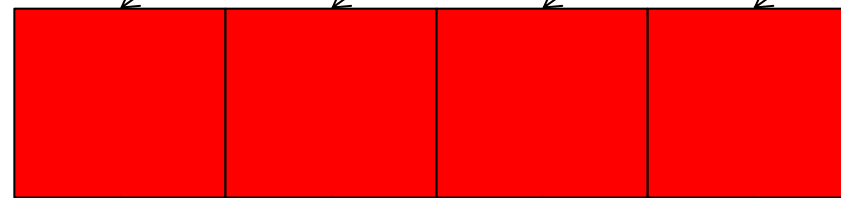
Focus the computation of threads on one or a small number of tiles at each point in time

Tiling/Blocking - Basic Idea

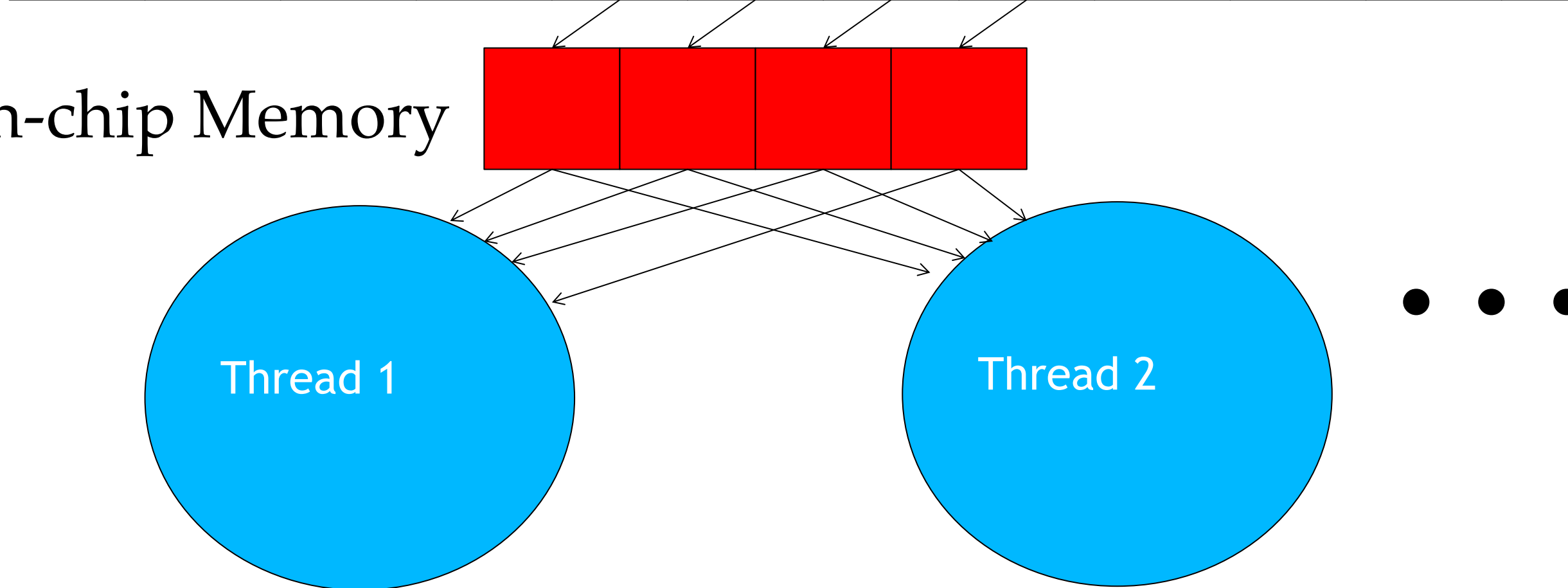
Global Memory



On-chip Memory



...



Basic Concept of Tiling

- In a congested traffic system, significant reduction of vehicles can greatly improve the delay seen by all vehicles
 - Carpooling for commuters
 - Tiling for global memory accesses
 - drivers = threads accessing their memory data operands
 - cars = memory access requests



Some Computations are More Challenging to Tile

- Some carpools may be easier than others
 - Car pool participants need to have similar work schedule
 - Some vehicles may be more suitable for carpooling
- Similar challenges exist in tiling



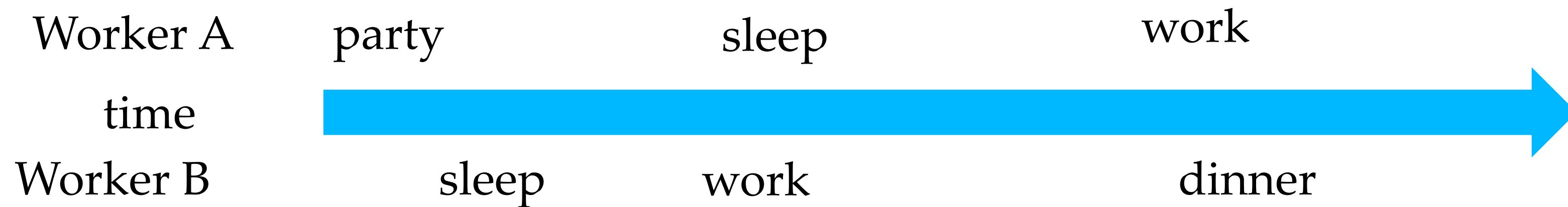
Carpools need synchronization.

- Good: when people have similar schedule



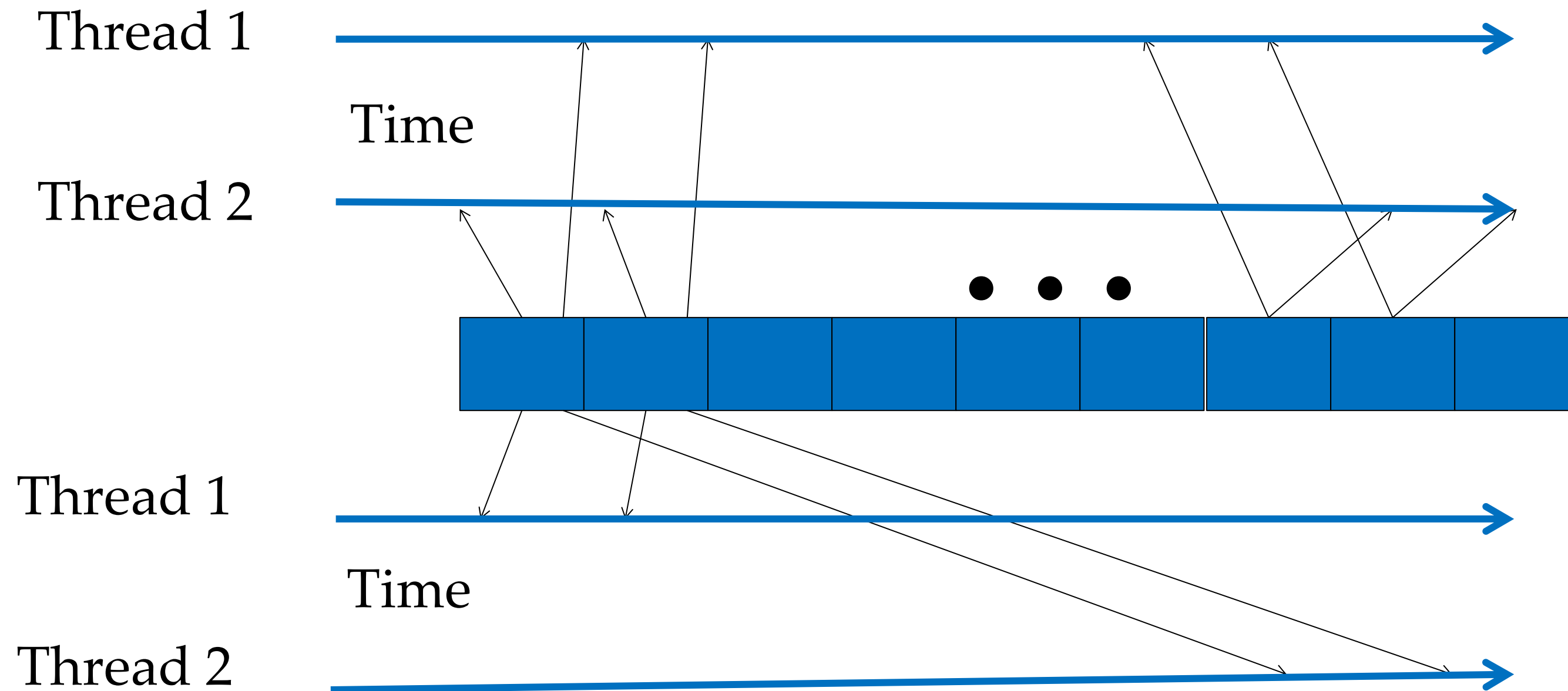
Carpools need synchronization.

- Bad: when people have very different schedule



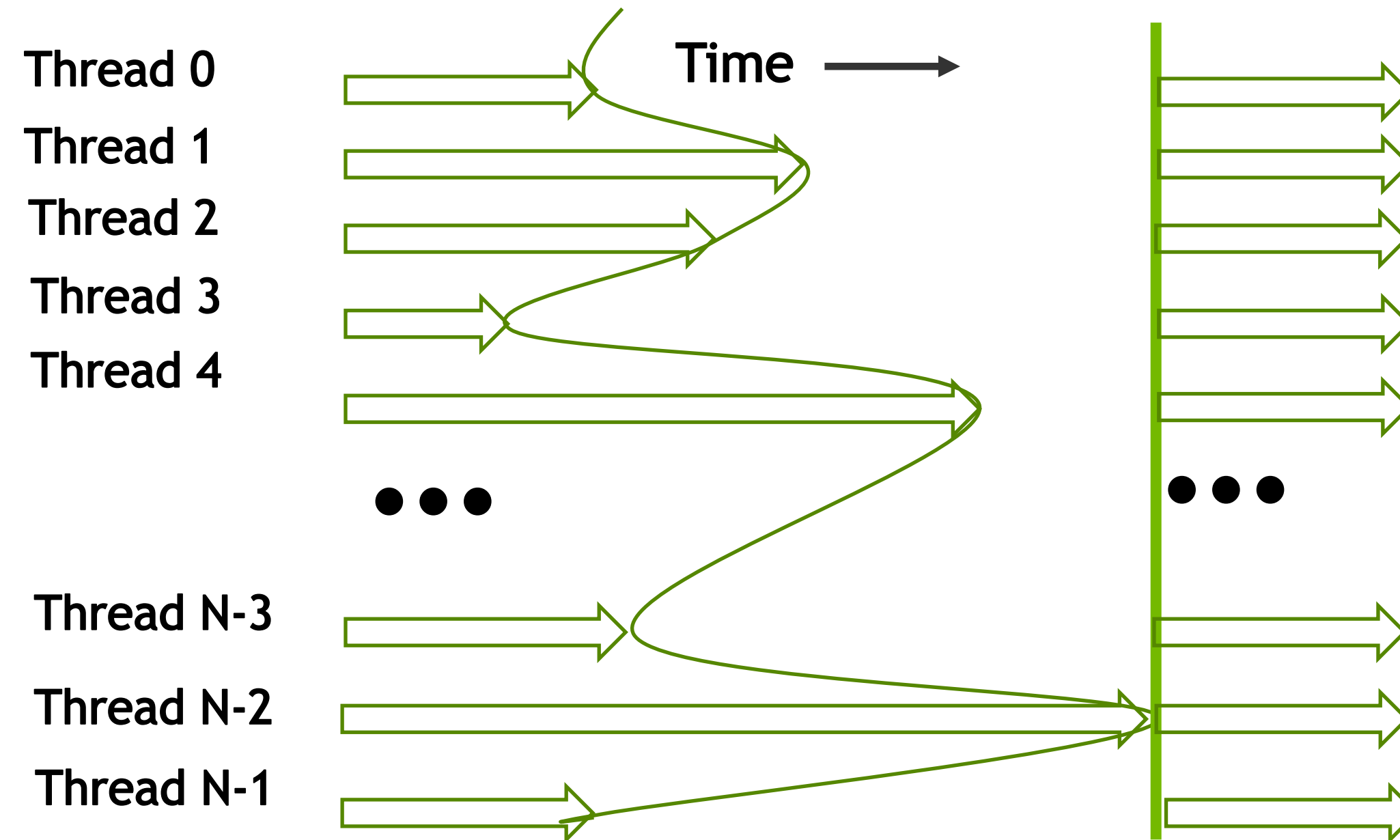
Same with Tiling

- Good: when threads have similar access timing



- Bad: when threads have very different timing

Barrier Synchronization for Tiling



Outline of Tiling Technique

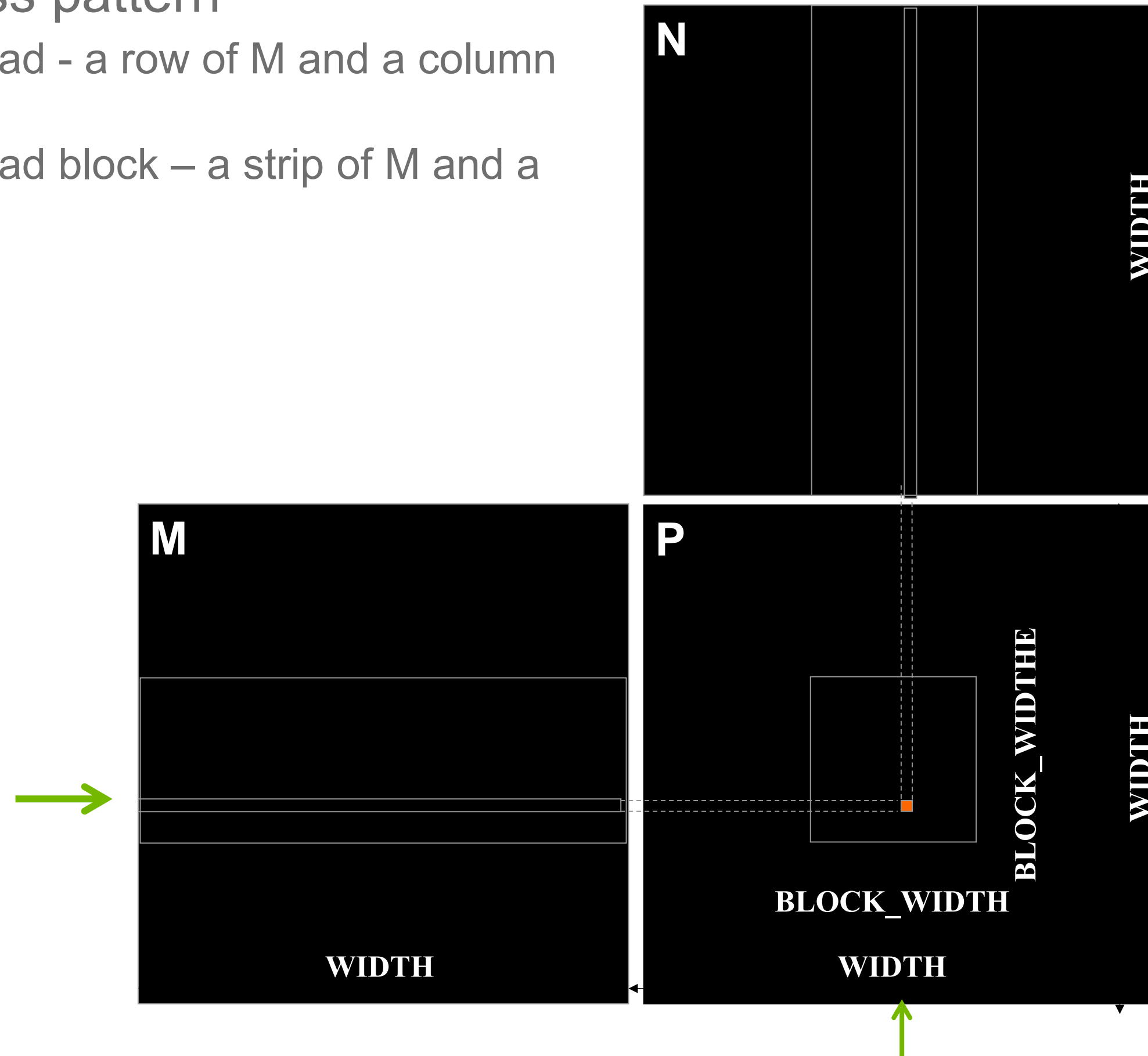
- Identify a tile of global memory contents that are accessed by multiple threads
- Load the tile from global memory into on-chip memory
- Use barrier synchronization to make sure that all threads are ready to start the phase
- Have the multiple threads to access their data from the on-chip memory
- Use barrier synchronization to make sure that all threads have completed the current phase
- Move on to the next tile

Objective

- To understand the design of a tiled parallel algorithm for matrix multiplication
 - Loading a tile
 - Phased execution
 - Barrier Synchronization

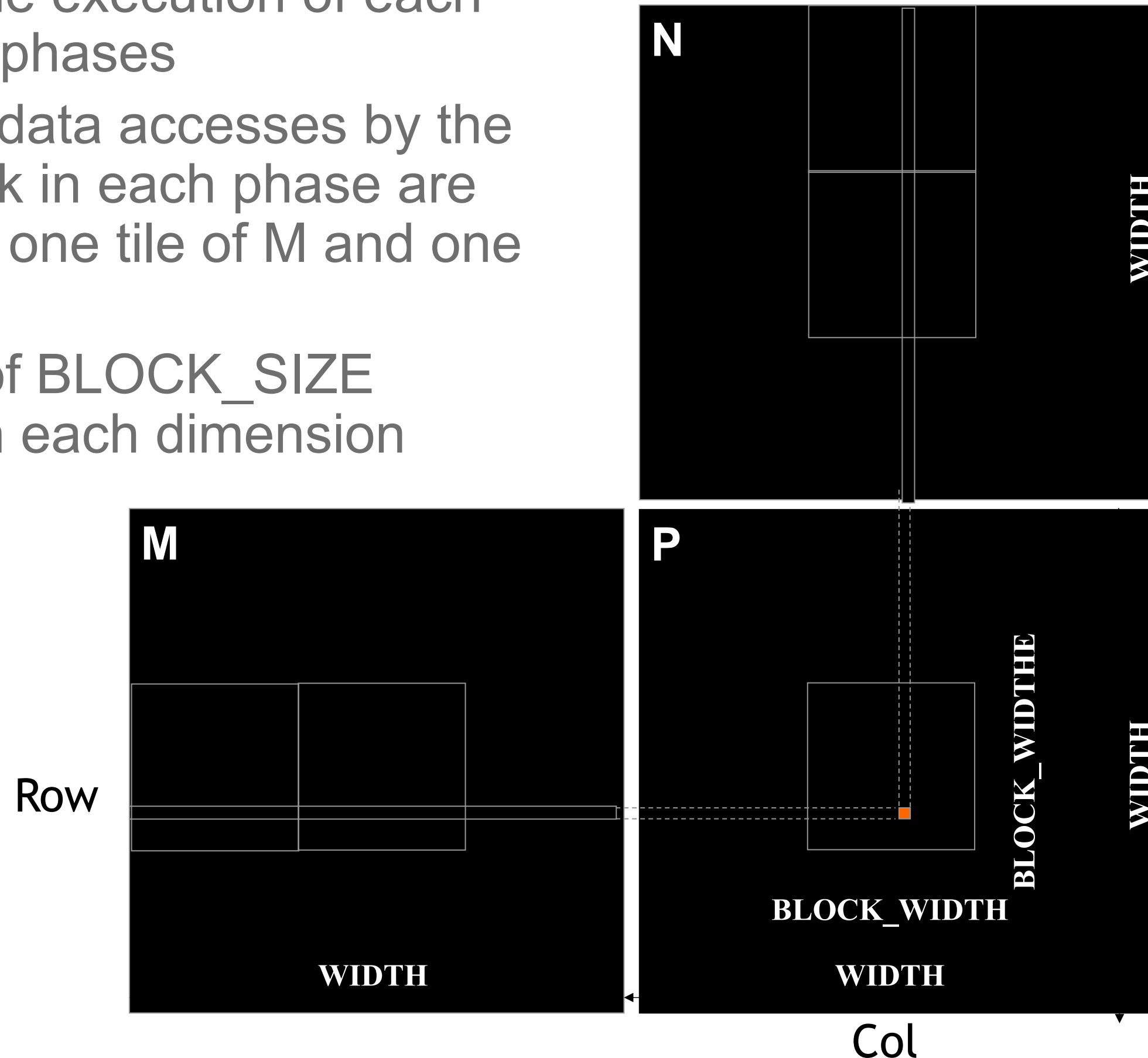
Matrix Multiplication

- Data access pattern
 - Each thread - a row of M and a column of N
 - Each thread block – a strip of M and a strip of N



Tiled Matrix Multiplication

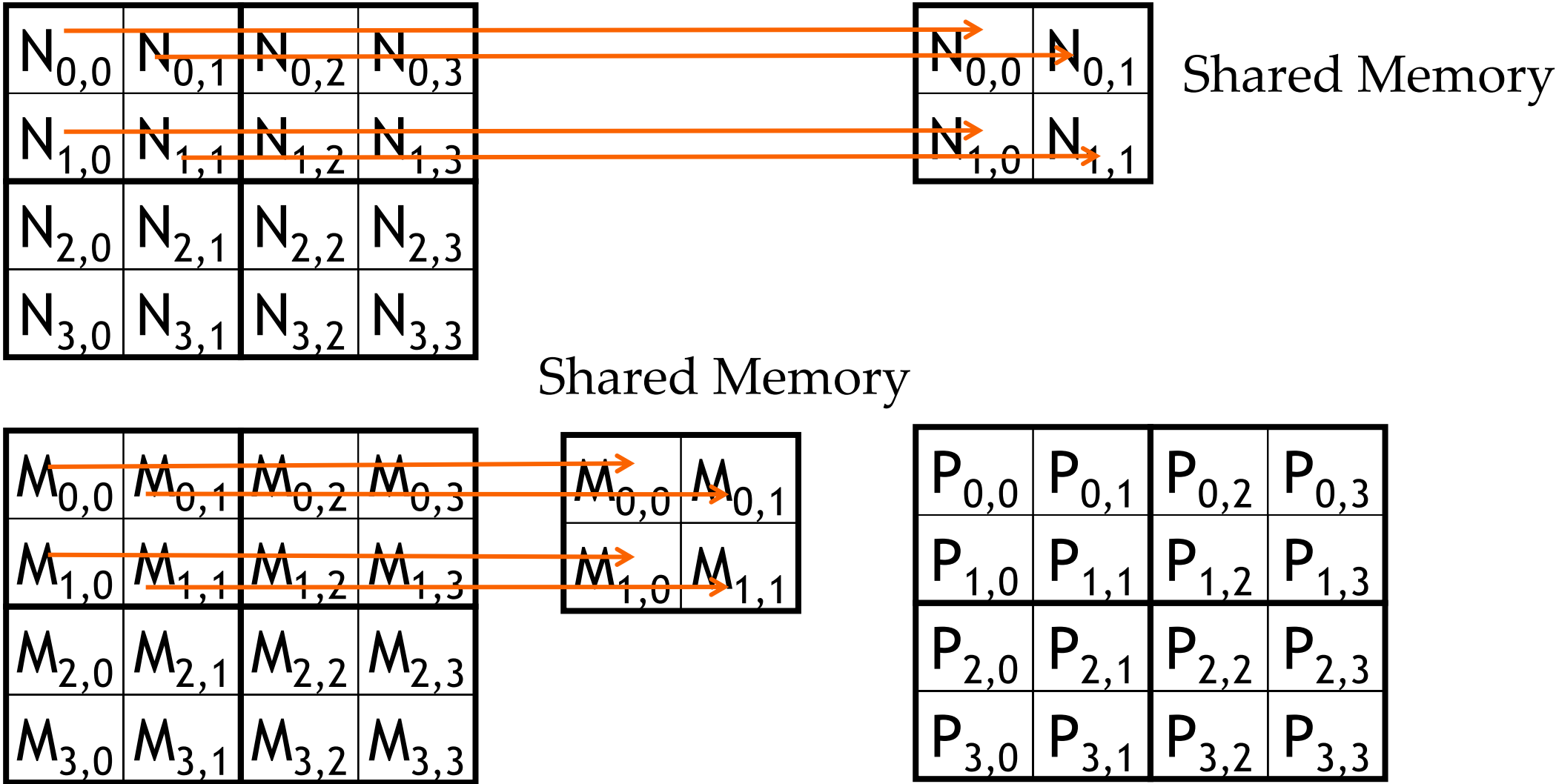
- Break up the execution of each thread into phases
- so that the data accesses by the thread block in each phase are focused on one tile of M and one tile of N
- The tile is of BLOCK_SIZE elements in each dimension



Loading a Tile

- All threads in a block participate
 - Each thread loads one M element and one N element in tiled code

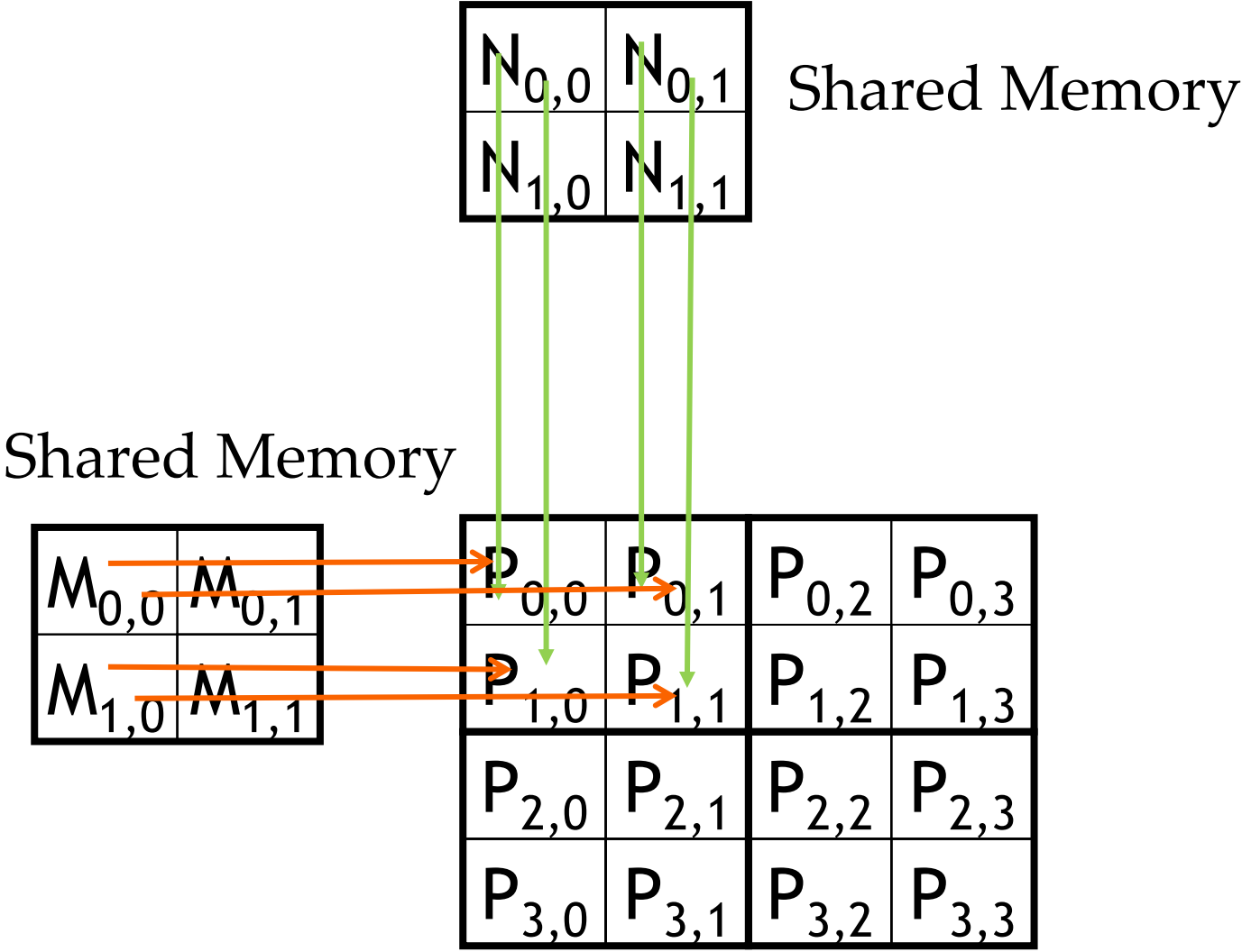
Phase 0 Load for Block (0,0)



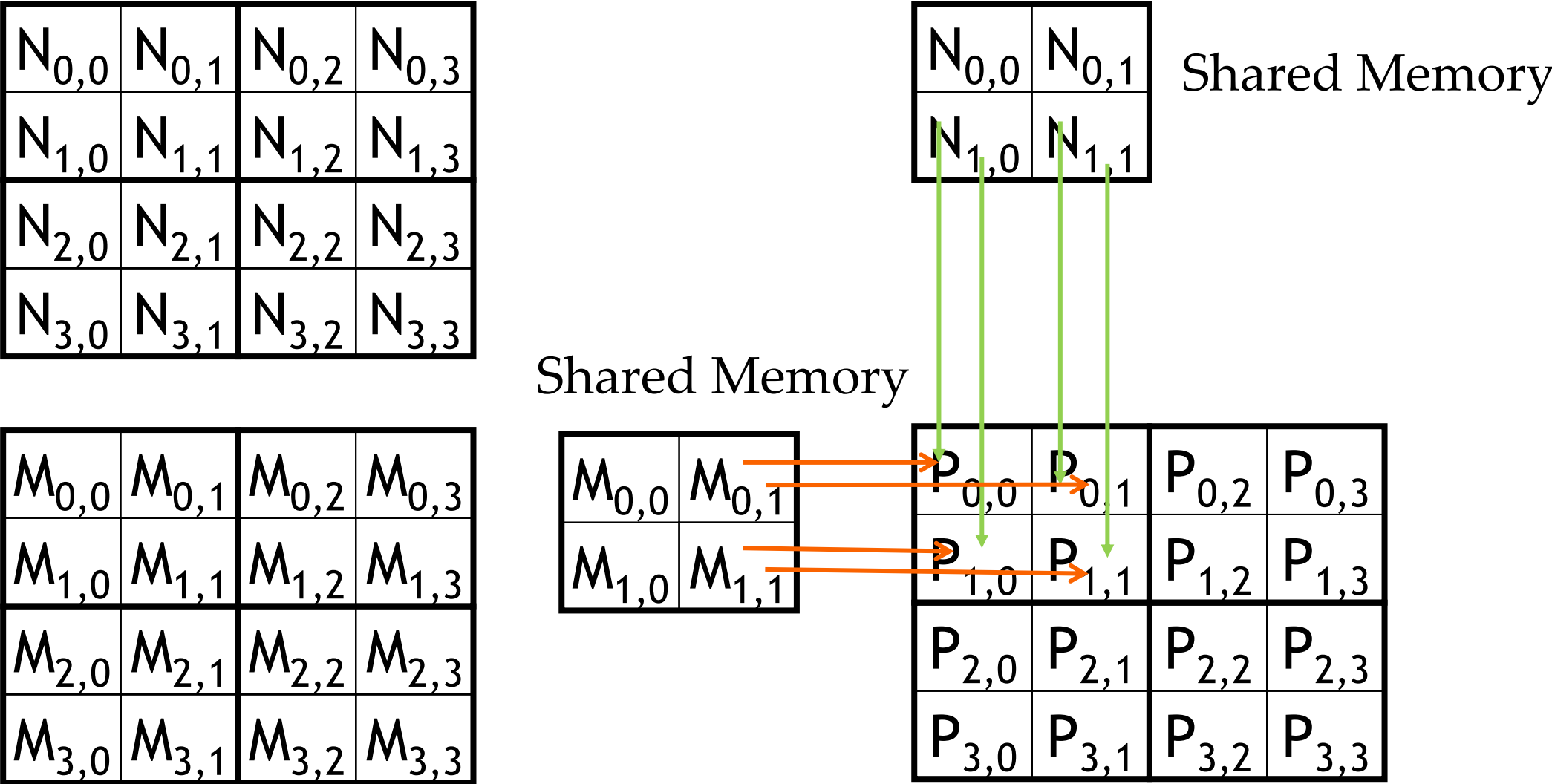
Phase 0 Use for Block (0,0) (iteration 0)

$N_{0,0}$	$N_{0,1}$	$N_{0,2}$	$N_{0,3}$
$N_{1,0}$	$N_{1,1}$	$N_{1,2}$	$N_{1,3}$
$N_{2,0}$	$N_{2,1}$	$N_{2,2}$	$N_{2,3}$
$N_{3,0}$	$N_{3,1}$	$N_{3,2}$	$N_{3,3}$

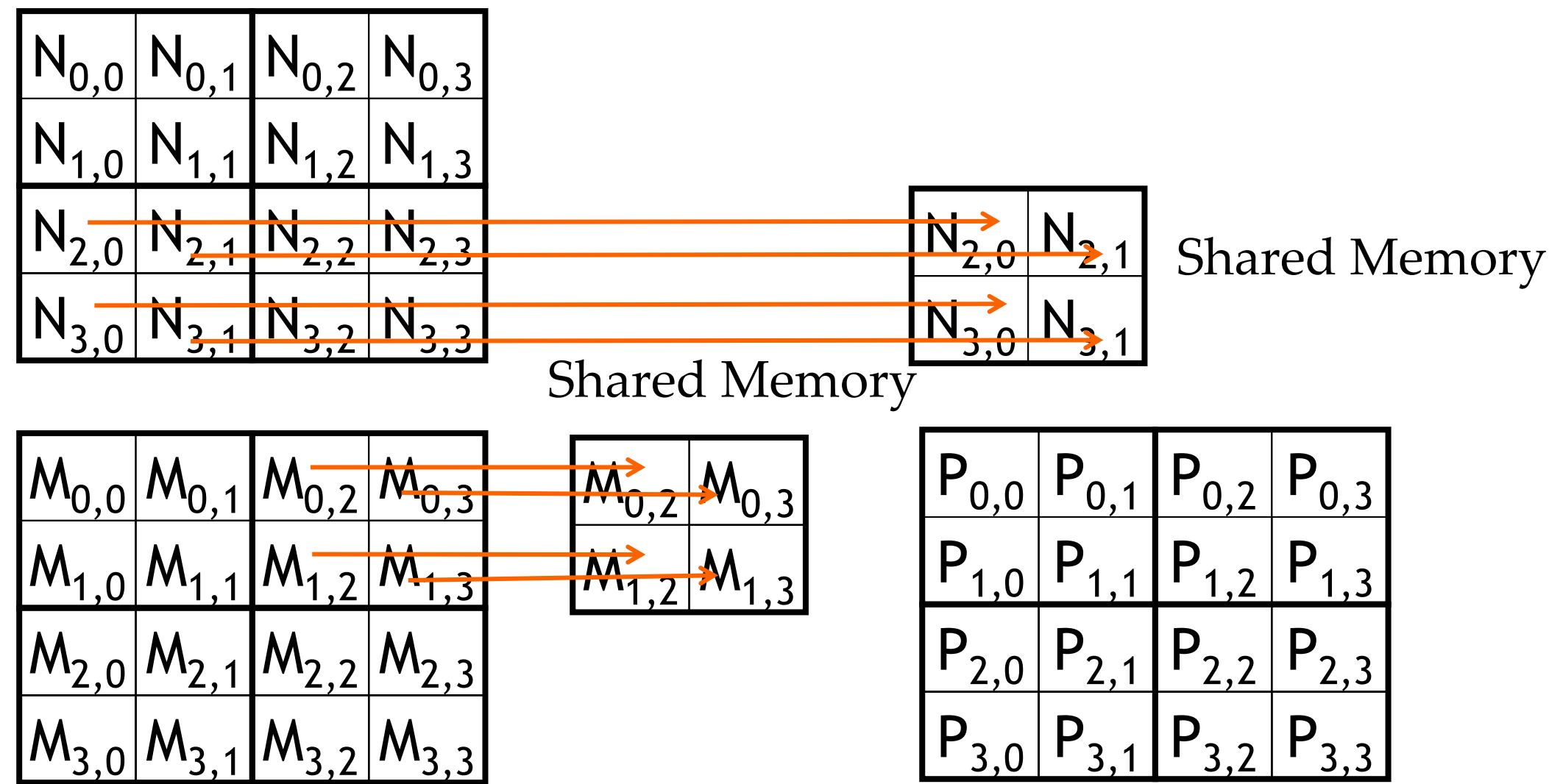
$M_{0,0}$	$M_{0,1}$	$M_{0,2}$	$M_{0,3}$
$M_{1,0}$	$M_{1,1}$	$M_{1,2}$	$M_{1,3}$
$M_{2,0}$	$M_{2,1}$	$M_{2,2}$	$M_{2,3}$
$M_{3,0}$	$M_{3,1}$	$M_{3,2}$	$M_{3,3}$



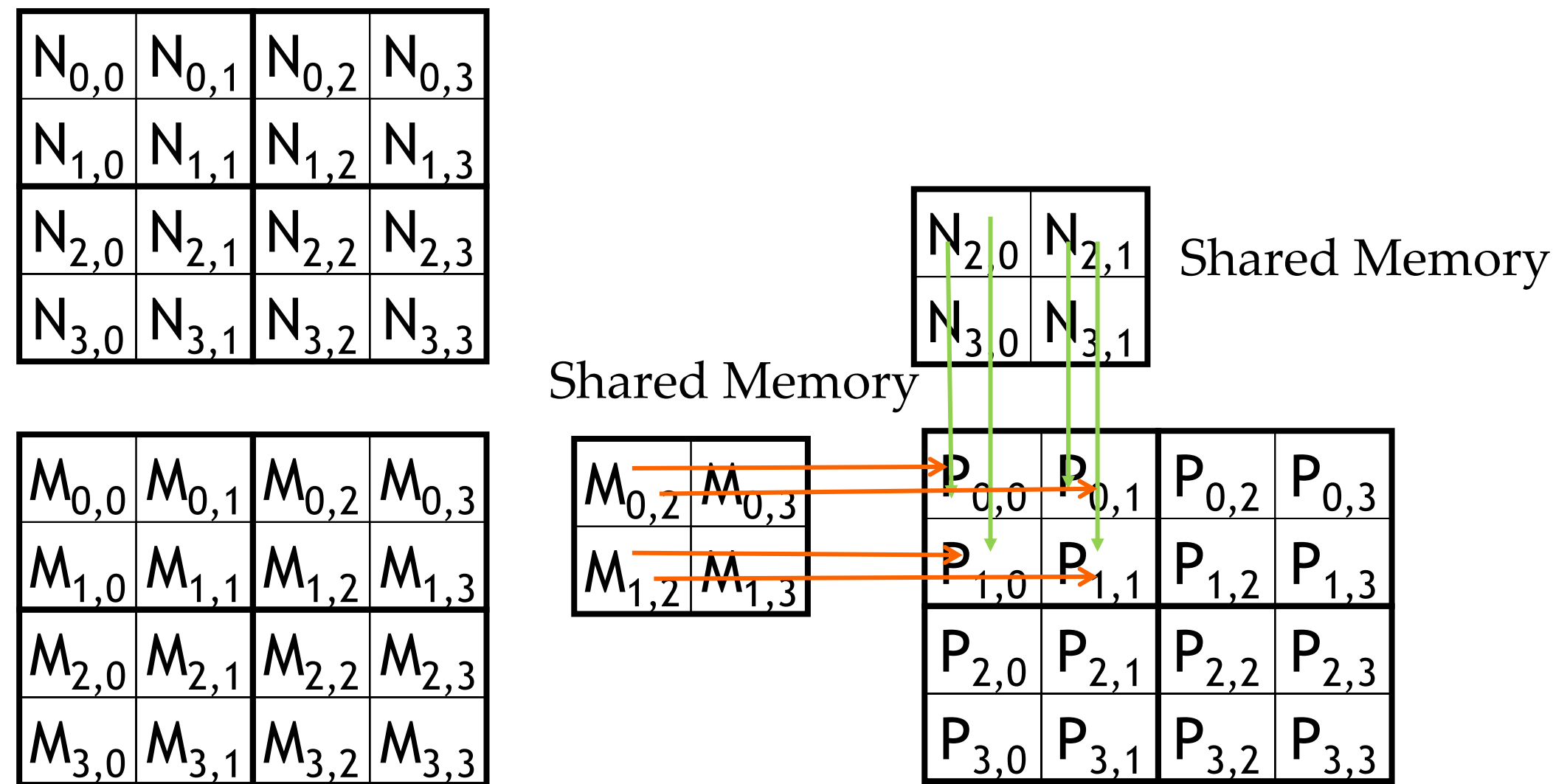
Phase 0 Use for Block (0,0) (iteration 1)



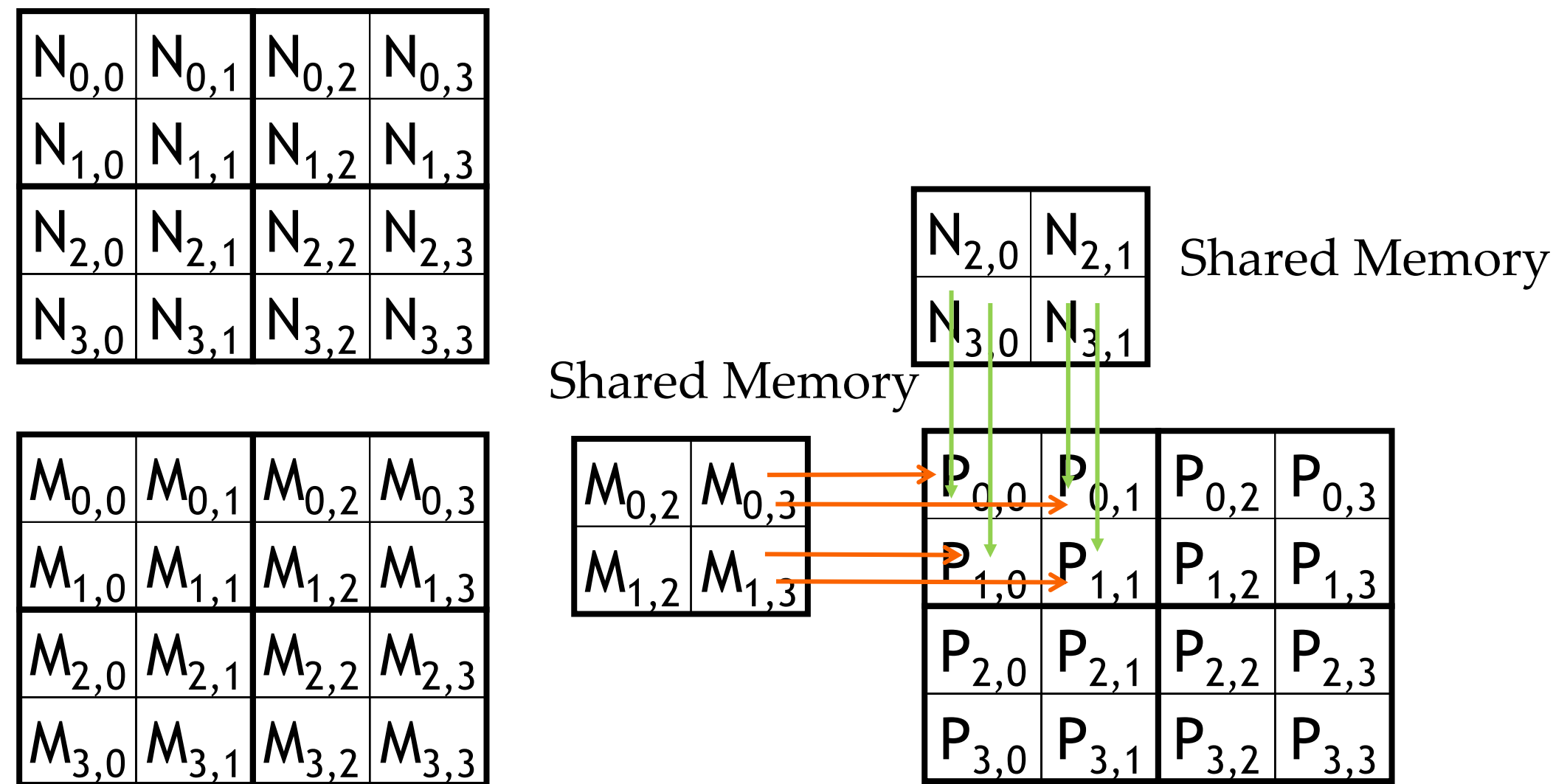
Phase 1 Load for Block (0,0)



Phase 1 Use for Block (0,0) (iteration 0)



Phase 1 Use for Block (0,0) (iteration 1)



Execution Phases of Toy Example

	Phase 0			Phase 1		
thread _{0,0}	M_{0,0} ↓ Mds _{0,0}	N_{0,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}	M_{0,2} ↓ Mds _{0,0}	N_{2,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}
thread _{0,1}	M_{0,1} ↓ Mds _{0,1}	N_{0,1} ↓ Nds _{1,0}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}	M_{0,3} ↓ Mds _{0,1}	N_{2,1} ↓ Nds _{0,1}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}
thread _{1,0}	M_{1,0} ↓ Mds _{1,0}	N_{1,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}	M_{1,2} ↓ Mds _{1,0}	N_{3,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}
thread _{1,1}	M_{1,1} ↓ Mds _{1,1}	N_{1,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}	M_{1,3} ↓ Mds _{1,1}	N_{3,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}

time 

Execution Phases of Toy Example (cont.)

	Phase 0			Phase 1		
thread _{0,0}	M_{0,0} ↓ Mds _{0,0}	N_{0,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}	M_{0,2} ↓ Mds _{0,0}	N_{2,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}
thread _{0,1}	M_{0,1} ↓ Mds _{0,1}	N_{0,1} ↓ Nds _{1,0}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}	M_{0,3} ↓ Mds _{0,1}	N_{2,1} ↓ Nds _{0,1}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}
thread _{1,0}	M_{1,0} ↓ Mds _{1,0}	N_{1,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}	M_{1,2} ↓ Mds _{1,0}	N_{3,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}
thread _{1,1}	M_{1,1} ↓ Mds _{1,1}	N_{1,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}	M_{1,3} ↓ Mds _{1,1}	N_{3,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}

time →

Shared memory allows each value to be accessed by multiple threads

Barrier Synchronization

- Synchronize all threads in a block
 - `__syncthreads()`
- All threads in the same block must reach the `__syncthreads()` before any of the them can move on
- Best used to coordinate the phased execution tiled algorithms
 - To ensure that all elements of a tile are loaded at the beginning of a phase
 - To ensure that all elements of a tile are consumed at the end of a phase

Objective

- To learn to write a tiled matrix-multiplication kernel
 - Loading and using tiles for matrix multiplication
 - Barrier synchronization, shared memory
 - Resource Considerations
 - Assume that Width is a multiple of tile size for simplicity

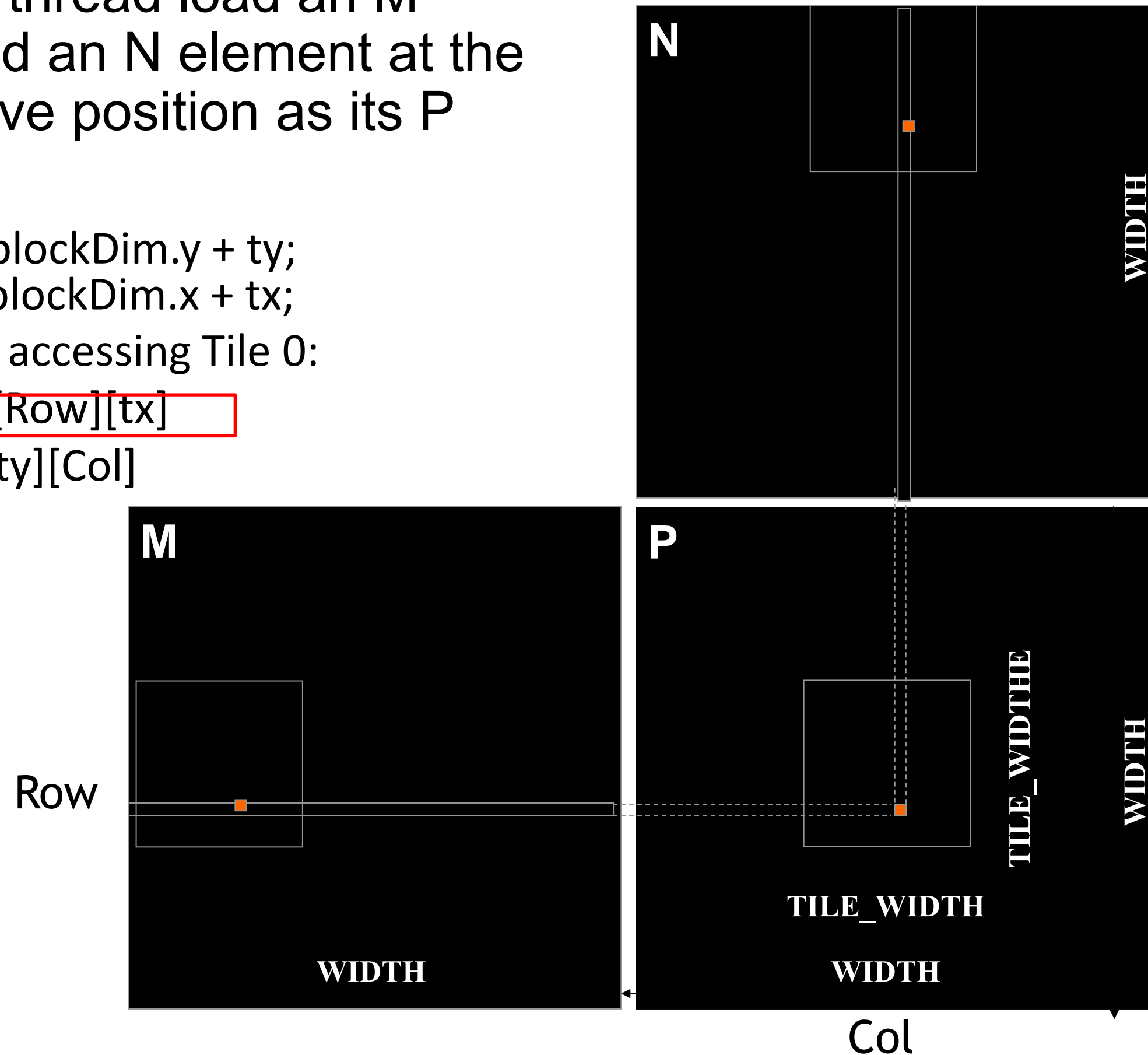
Loading Input Tile 0 of M (Phase 0)

- Have each thread load an M element and an N element at the same relative position as its P element.

```
int Row = by * blockDim.y + ty;  
int Col = bx * blockDim.x + tx;  
2D indexing for accessing Tile 0:
```

M[Row][tx]

N[ty][Col]



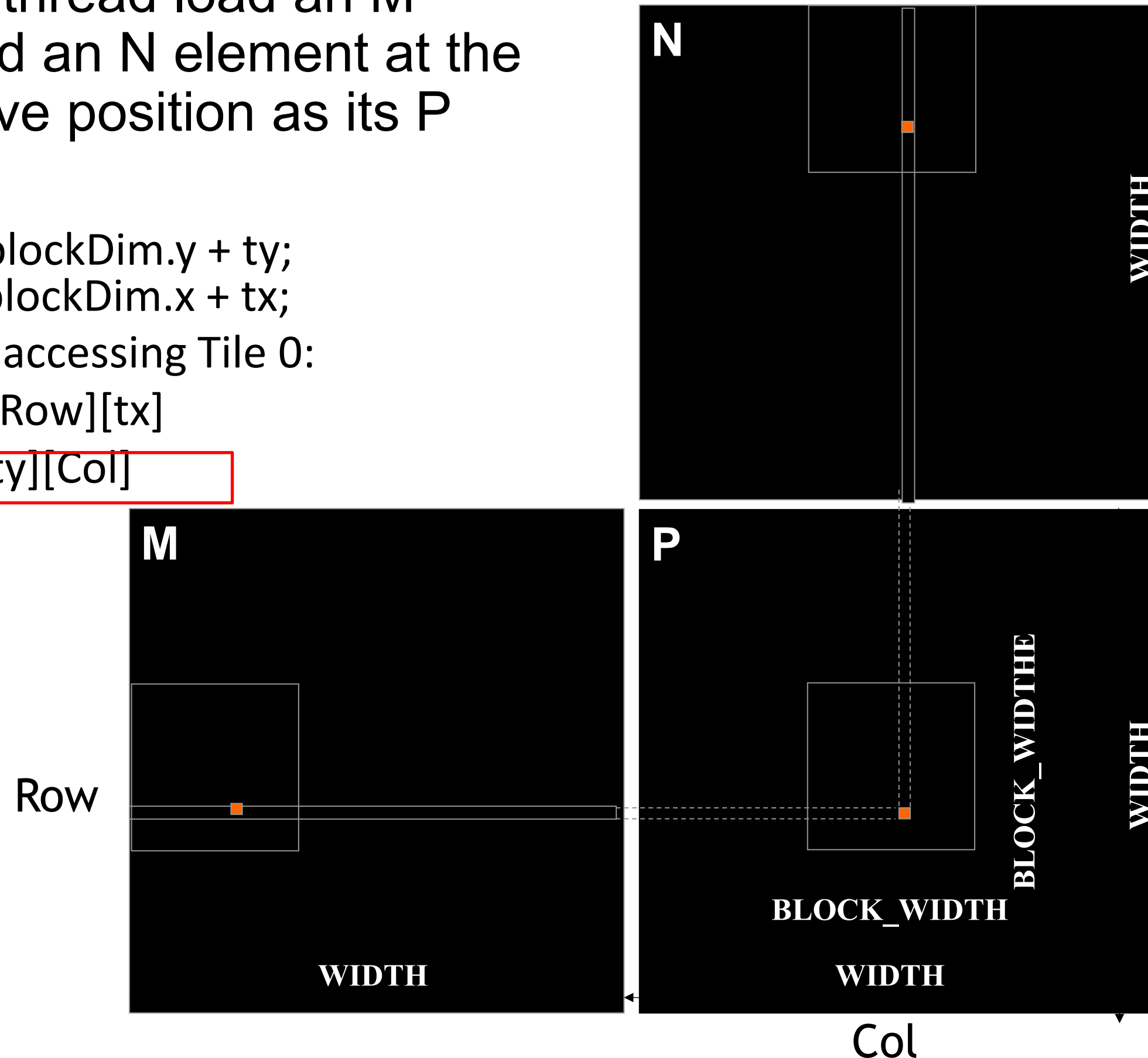
Loading Input Tile 0 of N (Phase 0)

- Have each thread load an M element and an N element at the same relative position as its P element.

```
int Row = by * blockDim.y + ty;  
int Col = bx * blockDim.x + tx;  
2D indexing for accessing Tile 0:
```

M[Row][tx]

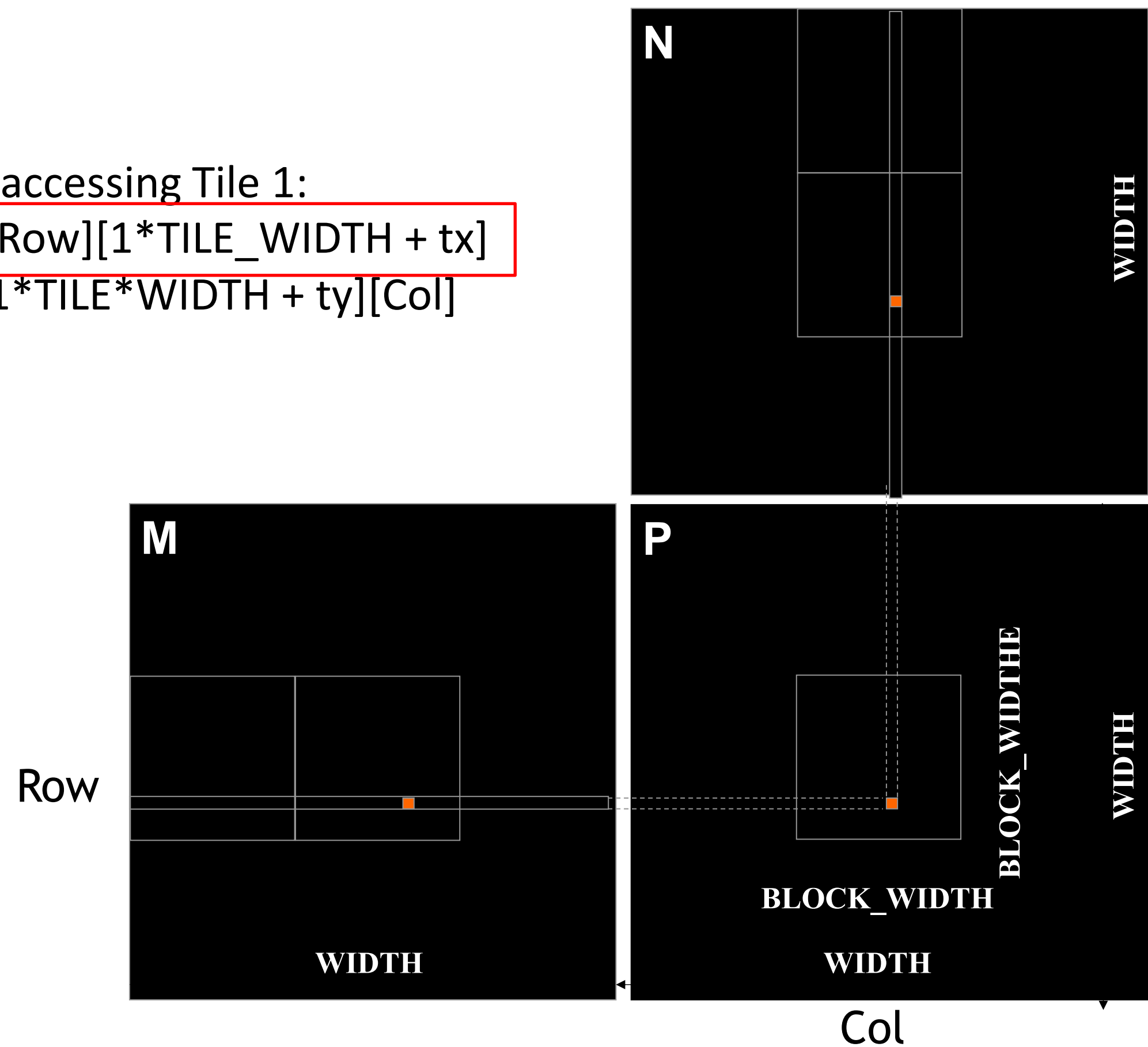
N[ty][Col]



Loading Input Tile 1 of M (Phase 1)

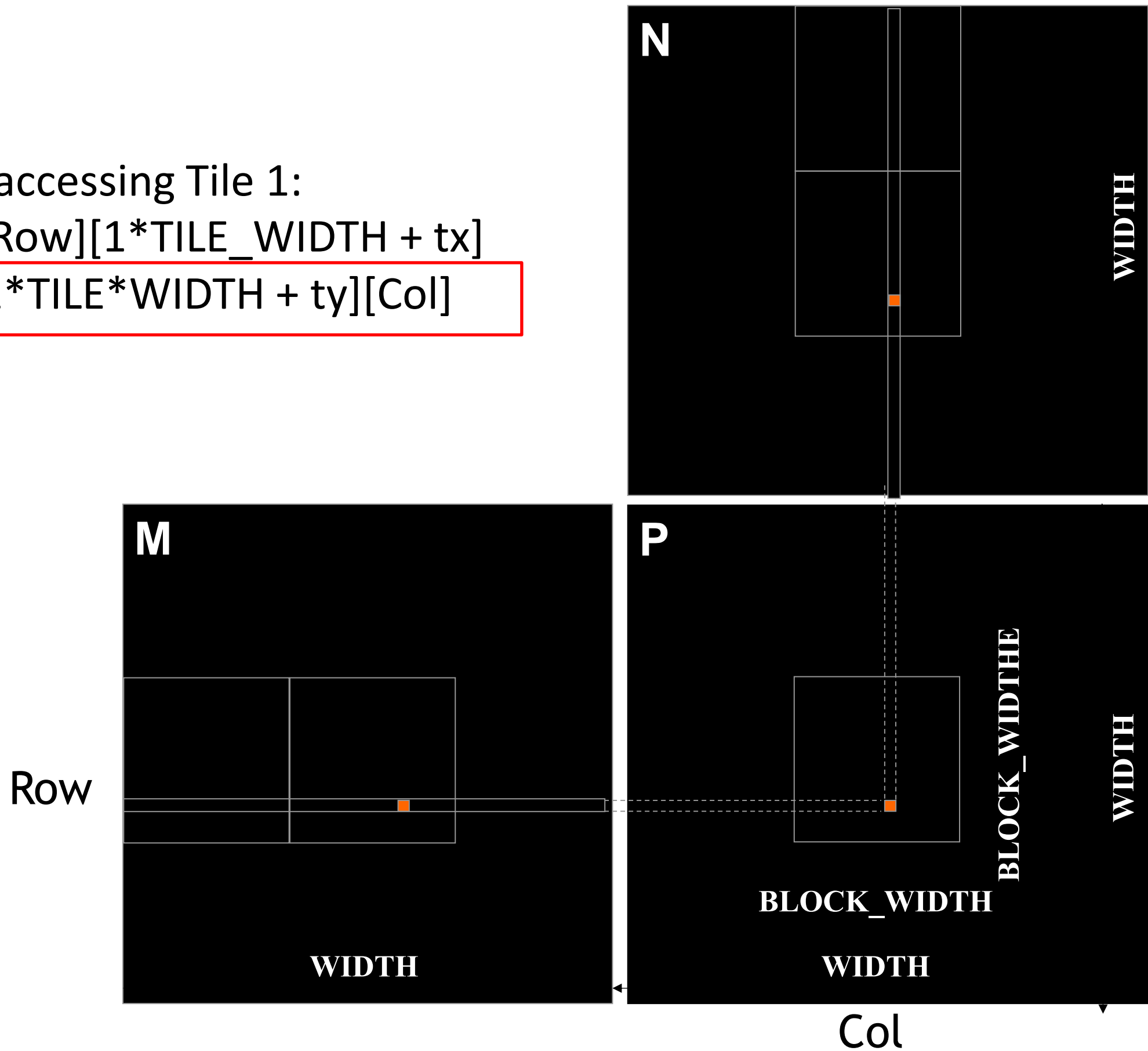
2D indexing for accessing Tile 1:

$$M[\text{Row}][1 * \text{TILE_WIDTH} + tx]$$
$$N[1 * \text{TILE} * \text{WIDTH} + ty][\text{Col}]$$



Loading Input Tile 1 of N (Phase 1)

2D indexing for accessing Tile 1:
M[Row][1*TILE_WIDTH + tx]
N[1*TILE*WIDTH + ty][Col]



M and N are dynamically allocated - use 1D indexing

→ $M[\text{Row}][p * \text{TILE_WIDTH} + tx]$
→ $M[\text{Row} * \text{Width} + p * \text{TILE_WIDTH} + tx]$

→ $N[p * \text{TILE_WIDTH} + ty][\text{Col}]$
→ $N[(p * \text{TILE_WIDTH} + ty) * \text{Width} + \text{Col}]$

where p is the sequence number of the current phase

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < n/TILE_WIDTH; ++p) {
        // Collaborative loading of M and N tiles into shared memory
        ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
        ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
        __syncthreads();

        for (int i = 0; i < TILE_WIDTH; ++i) Pvalue += ds_M[ty][i] * ds_N[i][tx];
        __syncthreads();
    }
    P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < n/TILE_WIDTH; ++p) {
        // Collaborative loading of M and N tiles into shared memory
        ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
        ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
        __syncthreads();

        for (int i = 0; i < TILE_WIDTH; ++i) Pvalue += ds_M[ty][i] * ds_N[i][tx];
        __syncthreads();
    }
    P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < n/TILE_WIDTH; ++p) {
        // Collaborative loading of M and N tiles into shared memory
        ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
        ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
        __syncthreads();

        for (int i = 0; i < TILE_WIDTH; ++i) Pvalue += ds_M[ty][i] * ds_N[i][tx];
        __syncthreads();
    }
    P[Row*Width+Col] = Pvalue;
}
```

Tile (Thread Block) Size Considerations

- Each **thread block** should have many threads
 - TILE_WIDTH of 16 gives $16 \times 16 = 256$ threads
 - TILE_WIDTH of 32 gives $32 \times 32 = 1024$ threads
- For 16, in each phase, each block performs $2 \times 256 = 512$ float loads from global memory for $256 \times (2 \times 16) = 8,192$ mul/add operations. (16 floating-point operations for each memory load)
- For 32, in each phase, each block performs $2 \times 1024 = 2048$ float loads from global memory for $1024 \times (2 \times 32) = 65,536$ mul/add operations. (32 floating-point operation for each memory load)

Shared Memory and Threading

- For an SM with 16KB shared memory
 - Shared memory size is implementation dependent!
 - For `TILE_WIDTH = 16`, each thread block uses $2 \times 256 \times 4\text{B} = 2\text{KB}$ of shared memory.
 - For 16KB shared memory, one can potentially have up to 8 thread blocks executing
 - This allows up to $8 \times 512 = 4,096$ pending loads. (2 per thread, 256 threads per block)
 - The next `TILE_WIDTH 32` would lead to $2 \times 32 \times 32 \times 4 \text{ Byte} = 8\text{K Byte}$ shared memory usage per thread block, allowing 2 thread blocks active at the same time
 - However, in a GPU where the thread count is limited to 1536 threads per SM, the number of blocks per SM is reduced to one!
- Each `__syncthread()` can reduce the number of active threads for a block
 - More thread blocks can be advantageous

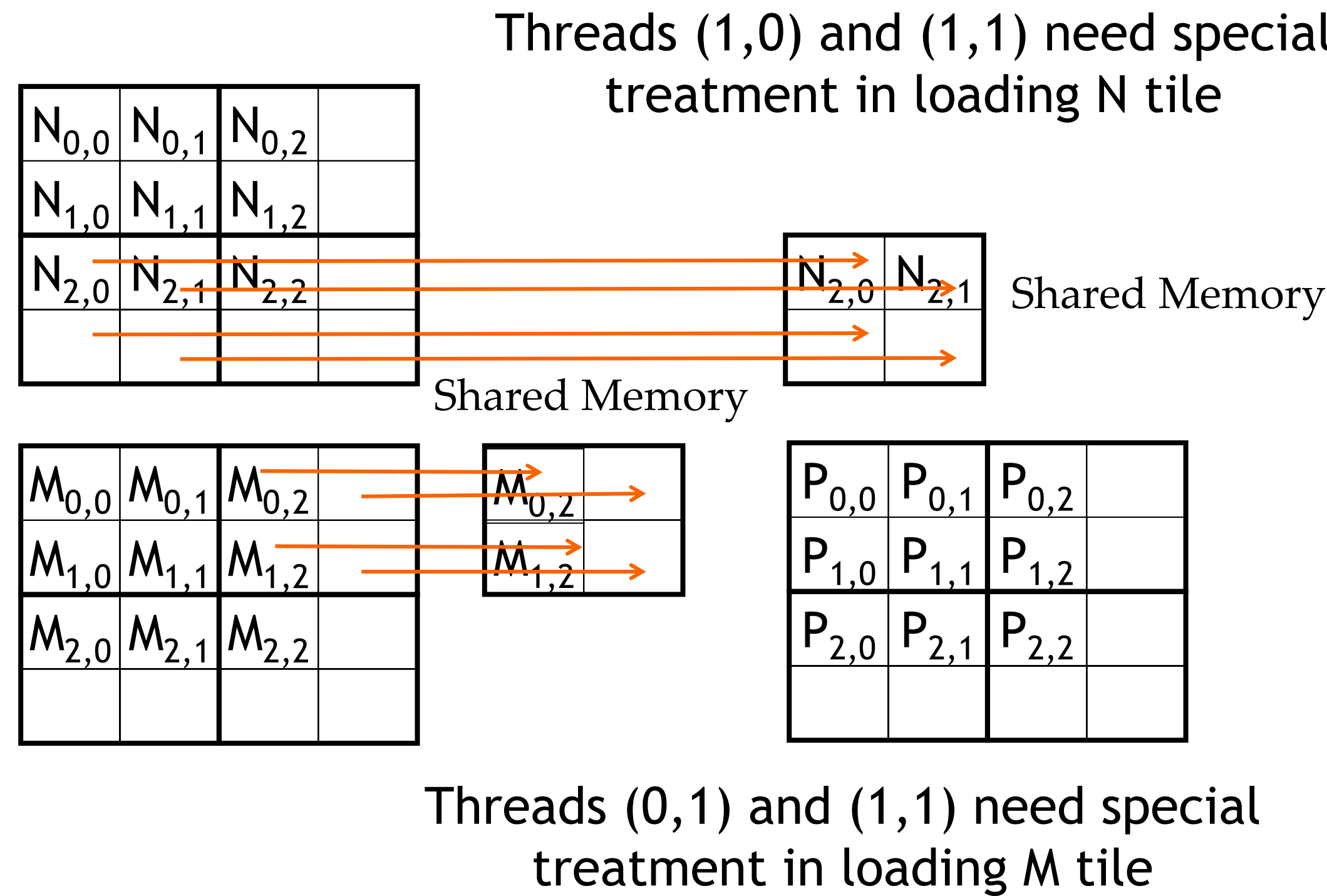
Objective

- To learn to handle arbitrary matrix sizes in tiled matrix multiplication
 - Boundary condition checking
 - Regularizing tile contents
 - Rectangular matrices

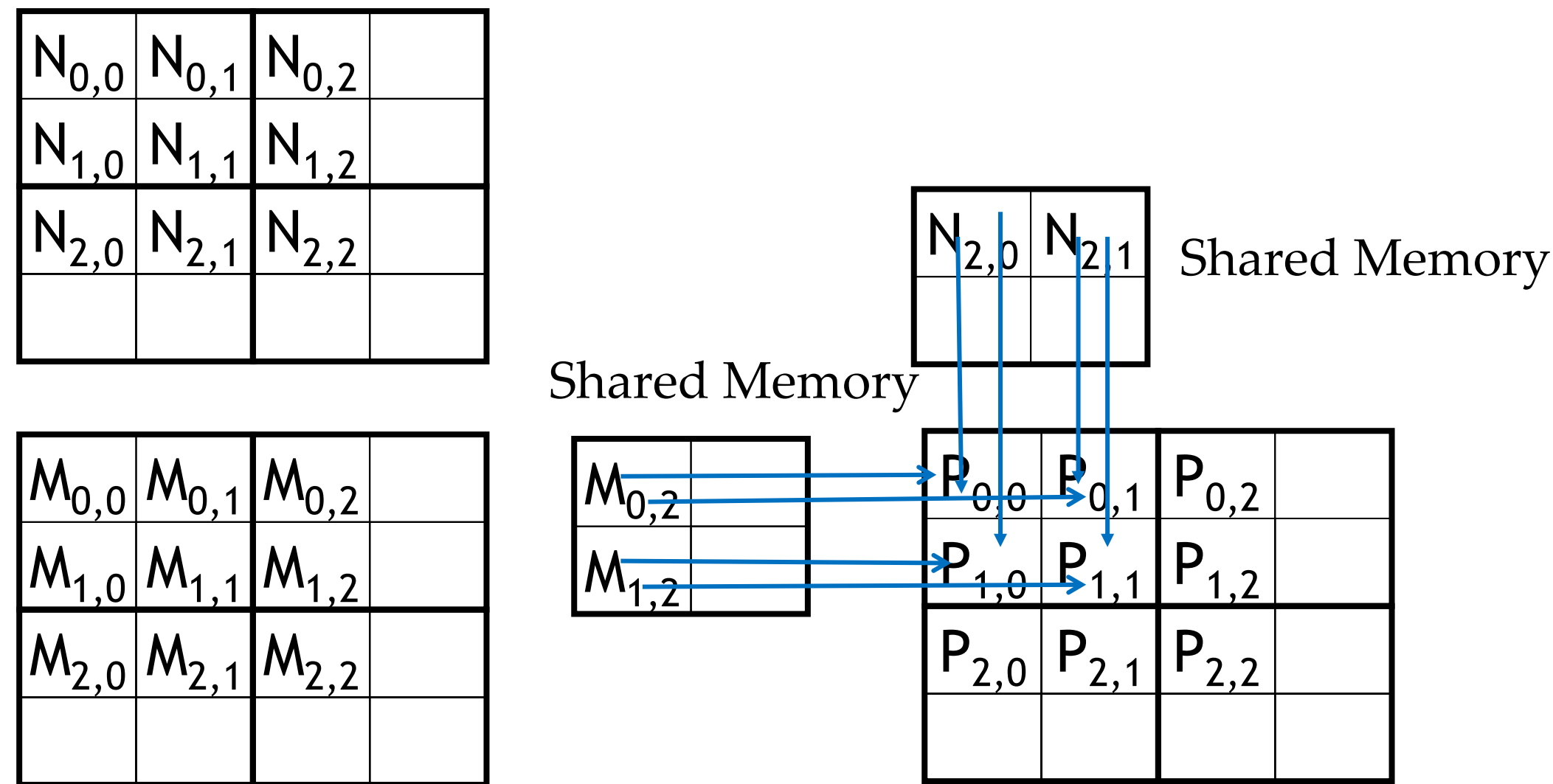
Handling Matrix of Arbitrary Size

- The tiled matrix multiplication kernel we presented so far can handle only square matrices whose dimensions (Width) are multiples of the tile width (TILE_WIDTH)
 - However, real applications need to handle arbitrary sized matrices.
 - One could pad (add elements to) the rows and columns into multiples of the tile size, but would have significant space and data transfer time overhead.
- We will take a different approach.

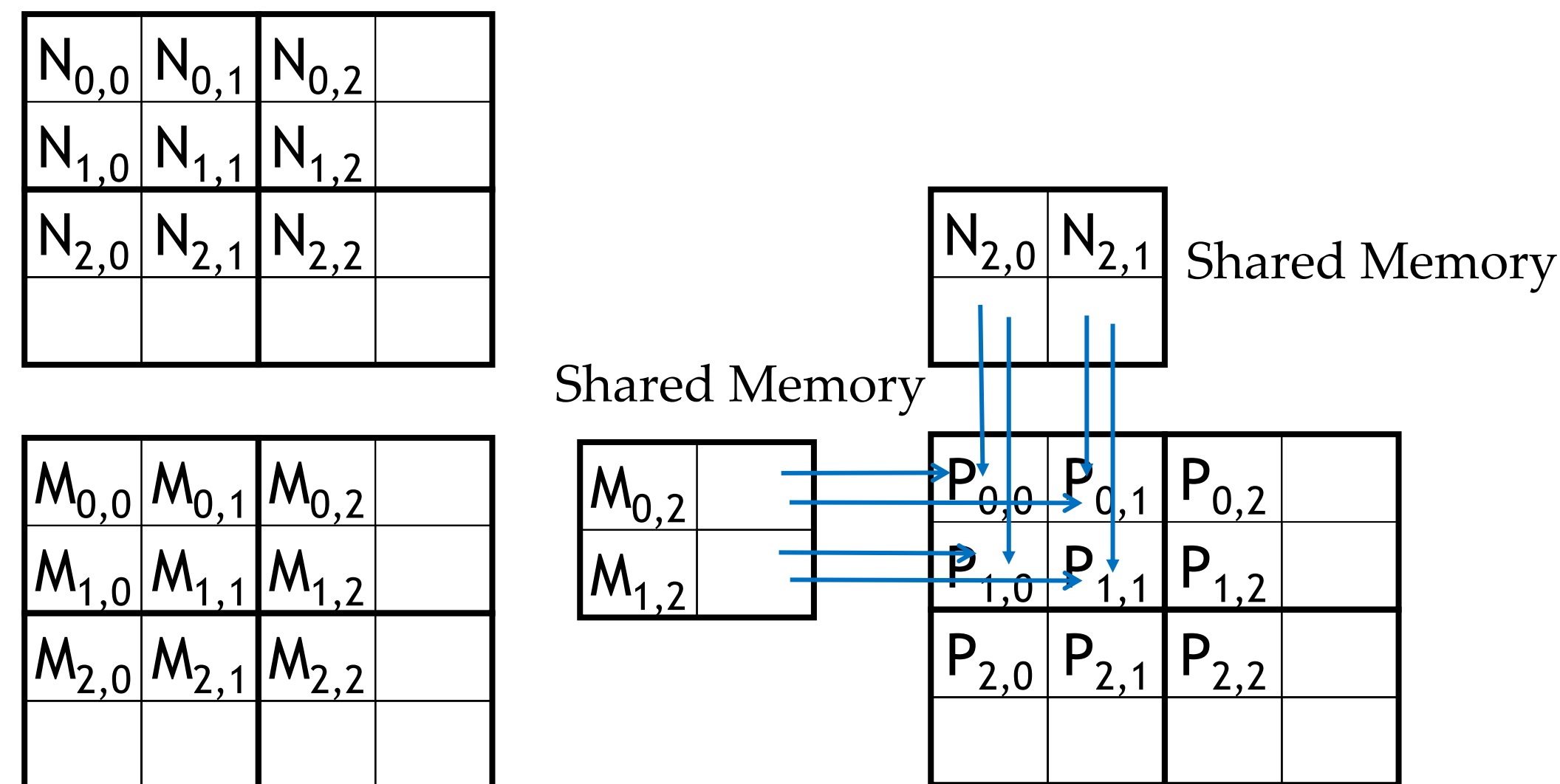
Phase 1 Loads for Block (0,0) for a 3x3 Example



Phase 1 Use for Block (0,0) (iteration 0)



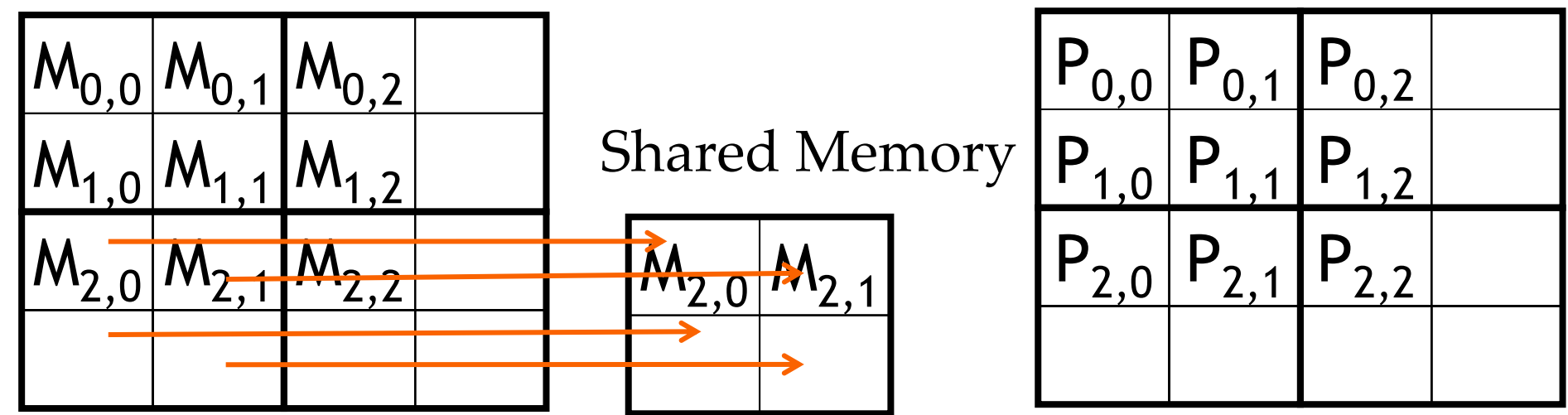
Phase 1 Use for Block (0,0) (iteration 1)



All Threads need special treatment.
None of them should introduce
invalidate contributions to their P
elements.

Phase 0 Loads for Block (1,1) for a 3x3 Example

Threads (0,1) and (1,1) need special treatment in loading N tile



Threads (1,0) and (1,1) need special treatment in loading M tile

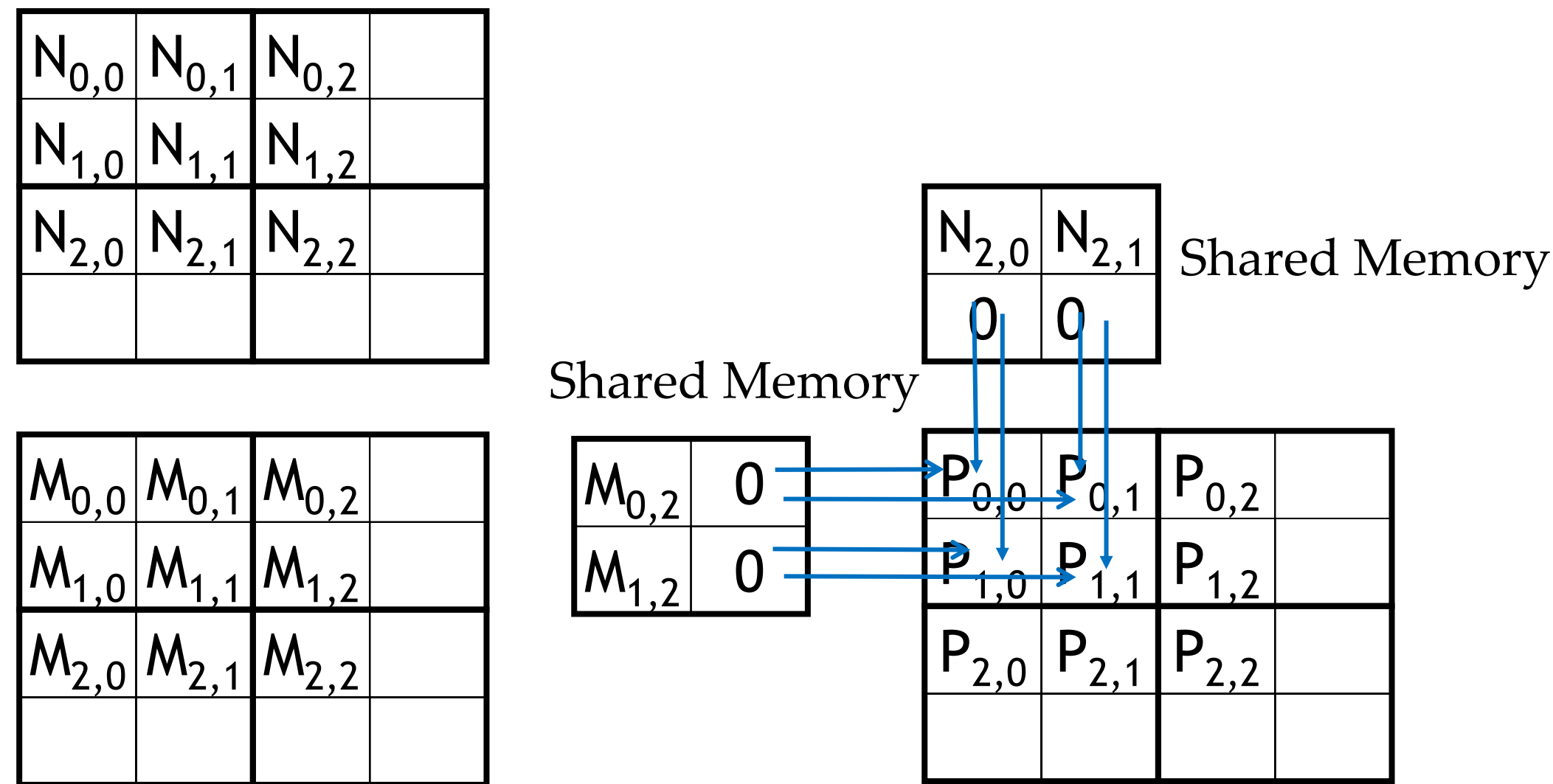
Major Cases in Toy Example

- Threads that do not calculate valid P elements but still need to participate in loading the input tiles
 - Phase 0 of Block(1,1), Thread(1,0), assigned to calculate non-existent $P[3,2]$ but need to participate in loading tile element $N[1,2]$
- Threads that calculate valid P elements may attempt to load non-existing input elements when loading input tiles
 - Phase 0 of Block(0,0), Thread(1,0), assigned to calculate valid $P[1,0]$ but attempts to load non-existing $N[3,0]$

A “Simple” Solution

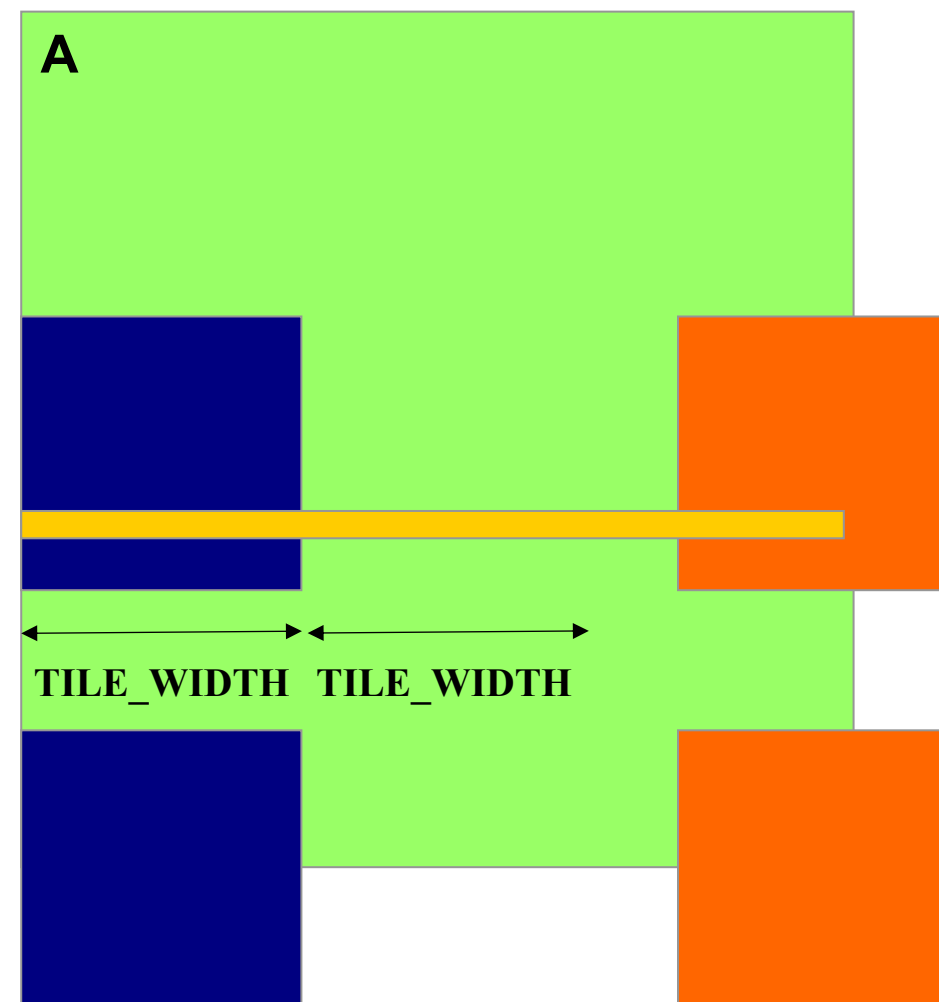
- When a thread is to load any input element, test if it is in the valid index range
 - If valid, proceed to load
 - Else, do not load, just write a 0
- Rationale: a 0 value will ensure that the multiply-add step does not affect the final value of the output element
- The condition tested for loading input elements is different from the test for calculating output P element
 - A thread that does not calculate valid P element can still participate in loading input tile elements

Phase 1 Use for Block (0,0) (iteration 1)



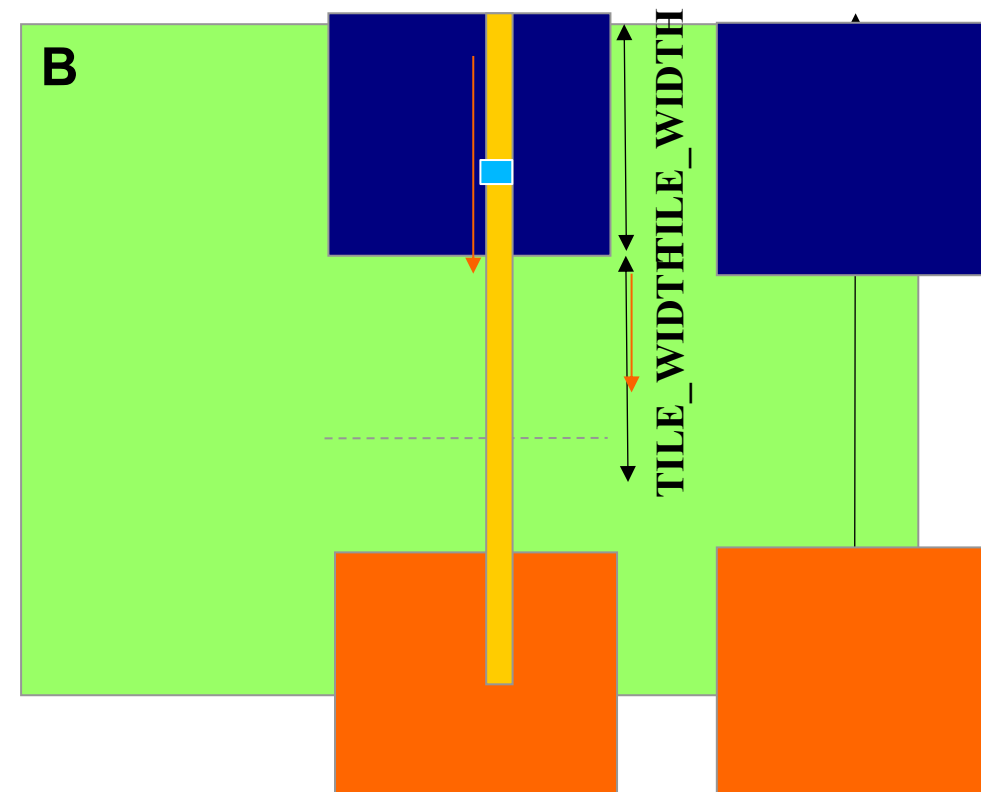
Boundary Condition for Input M Tile

- Each thread loads
 - $M[\text{Row}][p \cdot \text{TILE_WIDTH} + tx]$
 - $M[\text{Row} \cdot \text{Width} + p \cdot \text{TILE_WIDTH} + tx]$
- Need to test
 - $(\text{Row} < \text{Width}) \ \&\& \ (p \cdot \text{TILE_WIDTH} + tx < \text{Width})$
 - If true, load M element
 - Else , load 0



Boundary Condition for Input N Tile

- Each thread loads
 - $N[p \cdot \text{TILE_WIDTH} + ty][\text{Col}]$
 - $N[(p \cdot \text{TILE_WIDTH} + ty) \cdot \text{Width} + \text{Col}]$
- Need to test
 - $(p \cdot \text{TILE_WIDTH} + ty < \text{Width}) \ \&\& \ (\text{Col} < \text{Width})$
 - If true, load N element
 - Else , load 0



Loading Elements – with boundary check

```
– 8  for (int p = 0; p < (Width-1) / TILE_WIDTH + 1; ++p) {
–
–  ++    if(Row < Width && p * TILE_WIDTH+tx < Width) {
–  9      ds_M[ty][tx] = M[Row * Width + p * TILE_WIDTH + tx];
–  ++    } else {
–  ++      ds_M[ty][tx] = 0.0;
–  ++    }
–  ++    if (p*TILE_WIDTH+ty < Width && Col < Width) {
–  10      ds_N[ty][tx] = N[(p*TILE_WIDTH + ty) * Width + Col];
–  ++    } else {
–  ++      ds_N[ty][tx] = 0.0;
–  ++    }
–  11  __syncthreads();
–
```

Inner Product – Before and After

```
– ++  if(Row < Width && Col < Width) {  
– 12      for (int i = 0; i < TILE_WIDTH; ++i)  
– 13          Pvalue += ds_M[ty][i] * ds_N[i][tx];  
–      }  
– 14  __syncthreads();  
– 15 } /* end of outer for loop */  
– ++  if (Row < Width && Col < Width)  
– 16      P[Row*Width + Col] = Pvalue;  
– } /* end of kernel */
```

Some Important Points

- For each thread the conditions are different for
 - Loading M element
 - Loading N element
 - Calculating and storing output elements
- The effect of control divergence should be small for large matrices

Handling General Rectangular Matrices

- In general, the matrix multiplication is defined in terms of rectangular matrices
 - A $j \times k$ M matrix multiplied with a $k \times l$ N matrix results in a $j \times l$ P matrix
- We have presented square matrix multiplication, a special case
- The kernel function needs to be generalized to handle general rectangular matrices
 - The Width argument is replaced by three arguments: j, k, l
 - When Width is used to refer to the height of M or height of P, replace it with j
 - When Width is used to refer to the width of M or height of N, replace it with k
 - When Width is used to refer to the width of N or width of P, replace it with l